

UNIVERSITY OF COPENHAGEN
DEPARTMENT OF ECONOMICS



Escaping the black box: Feedforward Neural Networks for Pricing the Danish Cross-Section

Master's Thesis

Aske Engmark and Johan Ølgaard

Advisor: Anders Rahbek

Handed in: June 2nd, 2025

Keystrokes 146.400

ECTS: 30 points

[This page is intentionally left blank]

Escaping the black box: Feedforward Neural Networks for Pricing the Danish Cross-Section

Aske Engmark[†] and Johan Ølgaard[‡]

Advisor: Anders Rahbek

June 2025

University of Copenhagen

Department of Economics

Abstract

This thesis investigates the applicability and performance of feedforward neural networks (FFNNs) for forecasting excess returns in the Danish equity market. We set forth to demystify feedforward neural networks by proposing a generalized FFNN, exemplifying how they are calculated, and highlighting their use cases. We construct a dataset of firm-level predictors and market factors, and assess the forecast accuracy of FFNNs compared to the traditional methods OLS and LASSO regressions. To better capture directional accuracy in forecasts, we propose a novel loss function, the Augmented Mean Absolute Directional Loss (AMADL), designed to penalize incorrect sign predictions while retaining sensitivity to forecast accuracy. We find that FFNNs outperform linear methods when forecasting; however, they perform on par with a naïve mean forecast in point accuracy. When applying the FFNNs' forecasts in a net-zero H-L portfolio, they outperform their linear peers and the OMXC25 index, though the highest monthly excess returns do not exceed 0.6%.

All scripts required to reproduce the simulations and results described in this thesis are available at <https://github.com/johanoelgaard/MastersThesis>

Responsibilities:

Jointly: 1, 5, and 6

Aske Engmark: 2.1, 2.1.1, 2.1.3, 2.1.5, 2.2, 2.3.2, 2.4.1, 3, 3.1, 3.2, 4.1, 4.3

Johan Ølgaard: 2.1.2, 2.1.4, 2.1.6, 2.3.1, 2.4, 2.4.2 2.5, 3.3, 4.2, 4.4

[†]Student ID: cfh708

[‡]Student ID: jlh601

Contents

1	Introduction	3
1.1	Literature review	4
2	Neural networks for forecasting	6
2.1	Feedforward neural networks	7
2.1.1	FFNN architecture	7
2.1.2	Activation functions	9
2.1.3	Loss functions	10
2.1.4	Norm penalization	12
2.1.5	Network Training	14
2.1.6	Regularization techniques in training	18
2.2	FFNN exemplified	20
2.3	Linear models	27
2.3.1	OLS estimator	27
2.3.2	LASSO estimator	28
2.4	What is good forecasting	29
2.4.1	Forecasting for trading purposes	30
2.4.2	Portfolio performance metrics	33
2.5	Model simulations	34
3	Data	37
3.1	Firm-level trading and accounting data	38
3.2	Macroeconomic variables	39
3.3	Feature engineering and preprocessing	39
4	Empirical study on Danish equities	41
4.1	Model specification	41
4.2	Hyperparameter tuning	43
4.3	Out-of-sample forecasting	52
4.3.1	Which predictors matter?	54
4.4	Machine learning portfolios	59
5	Discussion	63
6	Concluding remarks	66
	Bibliography	69

A	Appendix	73
A.1	Pseudocode for gradient descent	73
A.2	Pseudocode for early stopping	75
A.3	Deriving gradient descent	75
A.4	Forward pass calculations	77
A.5	Gradient calculations	79
A.6	RMSE calculations	81
A.7	Deriving the OLS estimator	83
B	Additional Tables and Figures	84

1 Introduction

There exists a rich literature on predicting the cross-section of stock returns. Since the asset pricing models of Sharpe (1964), Lintner (1965), and Black (1972), many professionals and academics have contributed to the collective understanding of what factors play an important role in pricing stocks. Dubbed the "factor zoo" in Cochrane (2011), modern literature has collectively found several hundred factors to measure asset risk premia. In a forest of factors, defining a model to predict risk premia in the cross-section is not a trivial task. The econometrics literature presents numerous alternative estimation techniques, each with strengths and caveats. Since the early 90s, the use of machine learning has become prevalent in the economics and finance literature. Gu, Kelly, and Xiu (2020), one of the most cited papers on machine learning in finance,¹ define machine learning as; (a) a diverse collection of high-dimensional models for statistical prediction, combined with (b) so-called "regularization" methods for model selection and mitigation of overfit, and (c) efficient algorithms for searching among a vast number of potential model specifications. Machine learning turns around the approach of classical econometrics. Instead of approaching the problem with a predefined idea of the functional form and underlying data generating process (DGP), machine learning attempts to learn the DGP through the patterns that present themselves in the data. Machine learning methods are complex tools, and in the literature, they often take the shape of black boxes. Hence, the objective of this paper is to:

(i) demystify neural networks by explaining their inner workings, and through simulation show their use cases, (ii) deepen the understanding of neural network specification and evaluation in a financial context, and (iii) apply this method of estimation on a large factor model on Danish equities, and assess the usefulness of factors in this stock universe.

In doing so, we hope to make neural networks a more accessible tool for others. Furthermore, most literature is made using US data. In answering objective (iii), we follow the approach of Gu, Kelly, and Xiu (ibid.) by comparing neural networks to other more well-known regression methods, but on Danish data, to see if we can use some of the factors presented in academia to predict asset risk premia in a new stock universe.

The paper is structured as follows: Section 2 delves into feedforward neural networks. Here, we provide a generalization of the model and explain the intricacies of the applied algorithms, functions, and techniques. Additionally, we introduce

¹Gu, Kelly, and Xiu (2020) have 2592 citations according to [Google Scholar](#) as of 30/05/2025

Ordinary Least Squares (OLS) and the Least Absolute Shrinkage and Selection Operator (LASSO). We also reflect on how to assess risk premia forecasts and conduct simulations to enhance understanding of when feedforward neural networks may be beneficial. In Section 3, we introduce the dataset we synthesize for the factor model in a Danish stock universe. Like Gu, Kelly, and Xiu (2020), all the factors we gather are a subset of the factors presented in Green, Hand, and Zhang (2017), which all have some degree of academic evidence behind them. For macroeconomic variables, we use the definitions set forth by Welch and Goyal (2008). In Section 4, we specify the models and tune hyperparameters. We then assess the forecasting performance of each model, and lastly see if the forecasts can provide an edge when used to backtest simple trading strategies on Danish equities. In Section 5, we discuss and contextualize our findings, and in Section 6, we conclude this paper.

1.1 Literature review

This paper sits at the intersection of finance and machine learning. In this section, we first aim to gain a common understanding of factor models and the current state of the literature on measuring asset risk premia. Afterwards, we dive into the use of neural networks in asset pricing.

Factor models: measuring risk premia

Let us first set the scene for asset pricing and factor models in general. The first major breakthrough in asset pricing was the Capital Asset Pricing Model (CAPM) by Sharpe (1964), Lintner (1965), and Black (1972), which earned Sharpe the Nobel prize in 1990. CAPM provided a simple and elegant formula: the expected return of an asset is linear in its covariance with the market portfolio. This one-factor model treated the market’s excess return as the sole source of risk premia, predicting a neat trade-off: higher beta $\rightarrow$ higher expected return, compensating the investor for taking on excess risk, and is mean-variance efficient as described in Markowitz (1959). CAPM quickly became the cornerstone of finance theory, used in everything from cost of capital estimates to performance evaluation. However, CAPM suffers from empirical weaknesses. The beta and average return relationship is flatter or more tenuous than CAPM predicts. If you are structurally able to generate alpha,² the model is most likely misspecified. This poor empirical support, perhaps due to, among other things, difficulties in proxying the true market portfolio, poses the question of what other factors were at play (Fama and French 2004).

To challenge the simple approach of CAPM, Ross (1976) presented the Arbitrage Theory of Capital Asset Pricing. Ross builds on CAPM by deviating from the market

²Abnormal returns

portfolio as a single factor and argues that assets should be priced by allowing for multiple risk factors, such that no arbitrage is left in the market. This more flexible framework is where multi-factor models for pricing the cross-section of stock returns take off. This approach not only becomes prevalent for the cross section of stock returns, but prior to Ross (1976) we also see work like Black and Scholes (1973) using multi-factor models for pricing European options. This leaves us with an intriguing question: What factors are relevant for pricing capital?

Basu (1977) provided the first academic evidence for a value factor. An idea that was first made popular by David Dodd and Benjamin Graham in their book *Security Analysis* from 1934 (Graham and Dodd 1934). In short, the value effect implies that stocks with low price-to-earnings ratios tend to be undervalued and vice versa. A few years later, Banz (1981) introduced the "size effect". The size effect is the tendency for firms with relatively smaller market capitalization to earn a higher risk-adjusted return. These two factors, together with the market factor, laid the foundation for modern factor models. Fama and French (1992) and Fama and French (1993), incorporated these ideas, present empirical evidence, and lay out the so-called Fama-French 3-factor model, which, to this day, is likely the most well-known factor model and marks the beginning of the modern factor model approach. Following the contribution of Fama-French, the empirical finance literature takes off on a long run of empirical exploration into the complexities of the return-generation process. Jegadeesh and Titman (1993) showed that buying the stocks that recently performed well and shorting the worst-performing stocks earned structurally abnormal returns for up to a year. The so-called "momentum factor." Carhart (1997) incorporated it into the framework of Fama-French, expanding it to a four-factor model. By now, we have covered some of the most well-known factors. If we had to cover all factors that researchers have argued possess some explanatory power, this would be a very, very long literature review. It is not without merit that Cochrane (2011) called it the *factor zoo*, and Harvey, Liu, and Zhu (2015) showed that the number of documented factors was well above 300, and claim most of these results cannot be replicated outside the original samples used in each study.

Neural networks in finance

Neural networks, in the shape and form we know them today, were first formalized in neuroscience literature by Rosenblatt (1958), building on the work of McCulloch and Pitts (1943) and Hebb (1949). The idea was to mimic the way the brain works, through a network built up by layers, consisting of multiple neurons. However, it was not before White (1988) neural networks made their way into stock return forecasting. However, White (ibid.) did not find a single hidden layer neural network to be able

to beat a linear model in forecasting IBM common stock daily returns. As White puts it, "finding evidence against efficient markets with such simple networks is not going to be easy," referring to the efficient market hypothesis of Eugene Fama in Fama (1970). He follows up by exploring the use of a backpropagation algorithm in H. White (1989). Cybenko (1989) and Hornik, Stinchcombe, and H. White (1989) give us the *universal approximation theorem*, proving that any continuous function can be learned with arbitrary precision. One of the first successful implementations in the pricing literature is Hutchinson, Lo, and Poggio (1994), estimating the pricing formula of a derivative asset using neural networks. Since then, machine learning and finance have become progressively more intertwined, with a plethora of publications using neural networks Gu, Kelly, and Xiu (2020) pit feedforward networks against multiple regression alternatives on large-scale US data, and finds the neural networks to earn a Sharpe ratio of 1.35 in a value-weighted 10-1 long-short portfolio, more than doubling the performance of leading regression-based strategies from the literature. The popularity of this paper probably has something to do with the impressive out-of-sample results they present. In this paper, when conducting our empirical analysis, we take our starting point in Gu, Kelly, and Xiu (ibid.) and use the paper to put our findings in relief.

2 Neural networks for forecasting

In this section, we aim to shed light on what neural networks are and why they might be interesting to use in the context of forecasting in finance. To do this, we will begin by introducing feedforward neural networks (FFNNs) in Section 2.1. We do this to establish a foundational understanding of the actual mechanics behind this method of estimation. Unfortunately, neural nets often take the shape of "black boxes," where it is not clear what is going on, and thus what one can take away from using them. In Section 2.2, we attempt to show how a basic neural network works by calculating it by hand. We hope to demystify what neural networks are, and provide a common ground for understanding why they might be useful in a finance context, and, as importantly, what limitations these otherwise seemingly magical models have. We then briefly define the more well-known OLS and LASSO regression methods in Section 2.3.1 and 2.3.2. Then we run simulations comparing OLS, LASSO, and FFNNs on different data generating processes to exemplify where they differ in their use cases. For this to be meaningful, we will first inquire into what good forecasting is, and how to compare the performance of the models we present in Section 2.4. This provides a stepping stone into Section 2.5 where we run the simulations.

2.1 Feedforward neural networks

We take our theoretical starting point in Peña and Tsay (2021), but propose a generalized model for FFNN. A neural network attempts to approximate a response function for a given set of explanatory variables. It does this by searching for approximations of functions of linear combinations of all input variables. Cybenko (1989) and Hornik, Stinchcombe, and H. White (1989) proved that any continuous function on a closed and bounded input domain can be “learned” arbitrarily well by a fully connected feedforward neural network with just one hidden layer, provided you give it enough neurons and use a non-constant, non-polynomial activation function. While this does hold true, one will often see this theorem misused to mean “neural nets can do almost everything.” While the fact that the universal approximation theorem says such a function *can* be approximated, it says nothing about *how* to do it. The typical use case for neural networks is characterised by being highly non-convex in nature, and one will often find oneself getting stuck at local minima when optimizing these types of networks. Nevertheless, we will dive into how neural networks are structured and how they function.

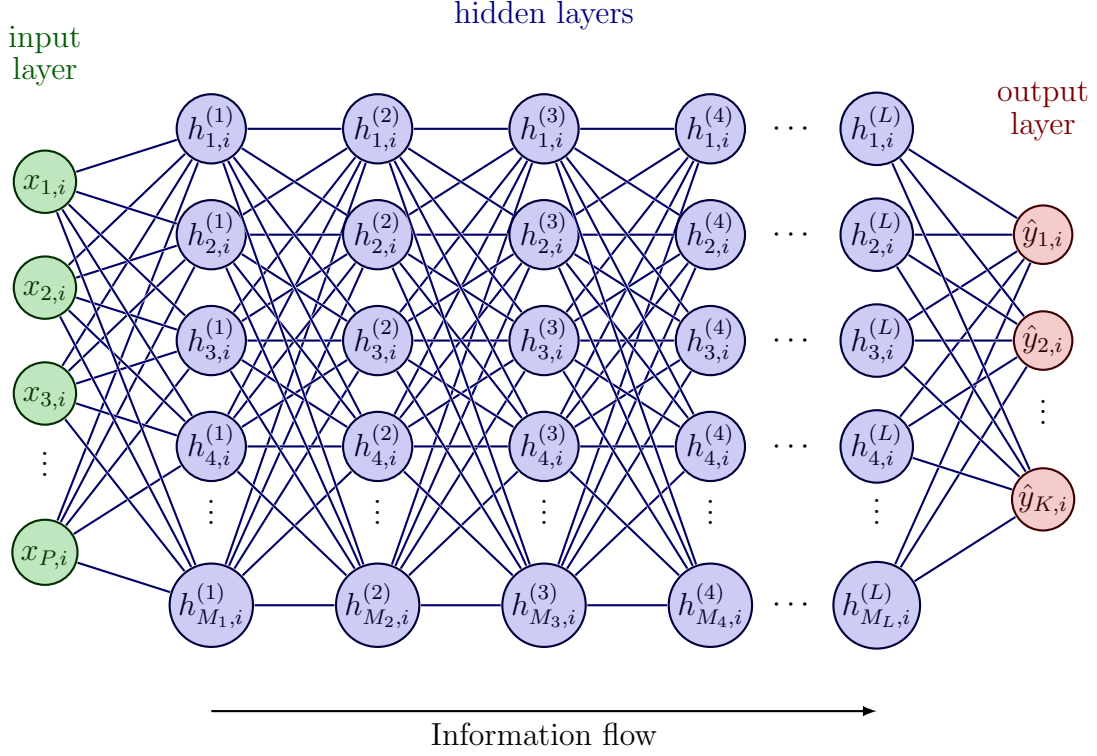
2.1.1 FFNN architecture

A feedforward neural network is a specific type of network called a perceptron. It is defined by the *control strategy* of the network, i.e., the direction of the information flow in the network. Information flows from the *input layer* consisting of explanatory variables $x_{p,i}$, indexed $p = 1, 2, \dots, P$ for observations $i = 1, 2, \dots, N$, and then forward through the layers in the middle of the network called *hidden layers*. A network with no hidden layers is called a single-layer perceptron (SLP),³ as it has only one trainable layer, the *output layer* comprising the predictions of the network. Usually, one will encounter FFNNs with at least one hidden layer, in which case we call it a multi-layer perception (MLP).⁴ The hidden layers are indexed $l = 1, 2, \dots, L$ and have $m = 1, 2, \dots, M_l$ amount of neurons, with $h_{m,i}^{(l)}$ being the output for the m th neuron at layer l . For feedforward networks, information flowing backwards to previous layers or to a neuron from itself is not permitted, as the network would then be a Recursive Neural Network. For FFNNs, each neuron only receives information once per forward pass. The last layer of the network, the output layer, produces the output of the network, be it one or more $\hat{y}_{k,i}$ indexed $k = 1, 2, \dots, K$. Oftentimes,

³We use the convention that an SLP is a network with no hidden layers. Some places you will see people using SLP for an FFNN with one hidden layer.

⁴A two-layer MLP will have 1 hidden layer and one output layer, as we typically only refer to trainable layers. This is not universally agreed, but this is the naming convention we follow in this paper.

Figure 2.1: General Feedforward Network Topology Diagram



NOTE: Topology diagram for a generalized FFNN. $x_{p,i}$ are the input features indexed $p = 1, \dots, P$, $h_{m,i}^{(l)}$ being the output for m th neuron with $m = 1, \dots, M_l$ at layer $l = 1, \dots, L$, and $\hat{y}_{k,i}$ being the model predictions indexed $k = 1, \dots, K$.

one will encounter what is called a fully connected FFNN, meaning all input nodes are connected to all neurons in the first hidden layer, and all neurons from one layer are connected to all neurons in the next layer, and so forth. An architecture diagram for a neural network of this type can be seen in Figure 2.1, also called a network topology diagram.

Each neuron processes information in two steps: (i) an affine transformation of all incoming inputs, often including a bias. This is referred to as the pre-activation, followed by (ii) a non-linear activation function wrapped around the affine transformation. The only exception will typically be at the output layer, where, when modeling continuous variables, the linear identity activation is used. We can now formally set up the network. For input vectors $\mathbf{x}_i = (x_{1,i}, x_{2,i}, \dots, x_{P,i})^\top$, output vectors from the hidden layers $\mathbf{h}_i^{(l)} = (h_{1,i}^{(l)}, h_{2,i}^{(l)}, \dots, h_{M_l,i}^{(l)})^\top$ and output vectors $\hat{\mathbf{y}}_i = (\hat{y}_{1,i}, \hat{y}_{2,i}, \dots, \hat{y}_{K,i})^\top$

the model of the network can be described by the equations:

$$\begin{aligned}
u_{m,i}^{(1)} &= \mathbf{w}_m^{(1)\top} \mathbf{x}_i + w_{m,0}^{(1)}, & \text{for } m = 1, 2, \dots, M_1, \\
u_{m,i}^{(l)} &= \mathbf{w}_m^{(l)\top} \mathbf{h}_i^{(l-1)} + w_{m,0}^{(l)}, & \text{for } m = 1, 2, \dots, M_l \text{ and } l = 2, 3, \dots, L, \\
h_{m,i}^{(l)} &= g_{l,m}(u_{m,i}^{(l)}), & \text{for } m = 1, 2, \dots, M_l \text{ and } l = 1, 2, \dots, L, \\
q_{k,i} &= \boldsymbol{\beta}_k^\top \mathbf{h}_i^L + \beta_{k,0}, & \text{for } k = 1, 2, \dots, K, \\
\hat{y}_{k,i} &= f_k(\mathbf{x}_i) = g_k(\mathbf{q}_i).
\end{aligned}$$

Here $\mathbf{w}_m^{(l)} = (w_{m,1}^{(l)}, w_{m,2}^{(l)}, \dots, w_{m,M_{(l-1)}}^{(l)})^\top$ and $w_{m,0}^{(l)}$ are the weights and biases of the m th node in the l th layer with $M_{(l-1)}$ being the number of neurons in the previous layer. Here, we define $M_{(1-1=0)} = P$ and thus each hidden layer will have $(M_{l-1} + 1) \cdot M_l$ parameters, while the output layer has $(M_L + 1) \cdot K$. Further, $\boldsymbol{\beta}_k = (\beta_{k,1}, \beta_{k,2}, \dots, \beta_{k,M_L})^\top$ and $\beta_{k,0}$ are the weights and bias output layer. $u_{m,i}^{(l)}$ and $q_{k,i}$ are the pre-activations of the hidden layer and the output layer, respectively. Lastly, $g_{l,m}(\cdot)$ is the activation function at the m th neuron at the l th layer, and $g_k(\cdot)$ is the activation function for the output layer. Although one has the flexibility to use a unique activation function for each layer and neuron, it is common to use the same activation function across all hidden layers and neurons.

2.1.2 Activation functions

Activation functions come in many shapes and forms. The purpose of the activation function is to introduce non-linearities to the network. If one did not have a non-linear transformation function, the whole network would collapse into a poor linear regression model. With this said, the output layer will, when modeling continuous variables in a regression network, always be identity activation such that $\hat{y}_{k,i} = g_k(\mathbf{q}_i) = q_{k,i}$. This is the only place one will see a linear activation function in practice. For classification, you might use a sigmoid or softmax function at the output layer; as we aim to do regression in this paper, we will be using identity activation at the output layer.

The activation function for the hidden neurons has a large selection to choose from. Some of the more popular options are as follows. In Section 4.2, we inquire into the importance of activation choice.

Sigmoid (σ)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Often used for smaller neural networks and classification tasks, a sigmoid (logistic) function has smooth gradients, and the output can be interpreted as probabilities.

There are two main challenges, the first being saturation for large $|x|$, causing vanishing gradients, and the second being the lack of computational efficiency when applied to deep learning.

Hyperbolic Tangent (tanh)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Quite similar to the sigmoid function, hyperbolic tangent helps with training as it has stronger gradients near 0 than sigmoid. Like sigmoid, it also saturates for large $|x|$ and is not the most computationally efficient.

Rectified Linear Unit (ReLU)

$$\text{ReLU}(x) = \max(0, x)$$

For deep learning (DL), ReLU is by far the most popular and well-known function. It is simple, and very computationally efficient, and mitigates vanishing gradient for $x > 0$. While the efficiency in computation makes it ideal for DL, it does have its limitations. It can have "dead" neurons when inputs are never positive.

Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0, \\ \alpha x, & x < 0, \end{cases}$$

To mitigate the issue of dead neurons, one can use the Leaky ReLU function, as it does not have zero gradient for $x < 0$, letting some of the signal "leak" through. Often, α will be a very small number, or can even be a parameter of the model to be trained, in which case it becomes a PReLU (parameteric ReLU) as per He et al. (2015). The family of ReLU activation functions is often referred to as rectifiers.

While there are no formalized guidelines for when to use what activation function, they each have their pros and cons. In general terms, these types of networks are often used for either regression or classification. In regression, for continuous variables, if used for DL, as we expect to do in this paper, some form of ReLU for the hidden neurons, as computational efficiency is desired to minimize wall time.

2.1.3 Loss functions

As for most types of parametric regression, neural networks are trained by minimizing a loss function. In this section, we will cover the most popular loss functions for regression using machine learning, MSE, MAE, and Huber loss functions, as they are the ones deployed in some capacity in this paper, following the theory of Terven et al.

(2025). One might also encounter Log-Cosh, Quantile, and Poisson loss functions in the literature on machine learning regression, and for a detailed overview of these, we refer the reader to Terven et al. (2025).

The loss function is the criterion with which we evaluate the fit of the model to the training data. Thus, this is where we impose an objective on the model. Generally, a regression loss function can be defined $\mathcal{L}$ over dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ for N samples where $\mathbf{x}_i \in \mathbb{R}^P$ is a P -dimensional vector of explanatory variables, often called an input feature vector in machine learning, and $y_i \in \mathbb{R}$ is the continuous target variable. Define a model f_θ of parameters θ that produce predictions $\hat{y}_i = f_\theta(\mathbf{x}_i)$. The goal is to obtain the parameter vector θ that minimizes the loss:

$$\min_{\theta} \mathcal{L}(\{\mathbf{x}_i, y_i\}_{i=1}^N, \theta).$$

Mean Squared Error (MSE)

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - f_\theta(\mathbf{x}_i))^2 \quad (2.1)$$

Mean Squared Error, or L2 loss, is one of the most common loss functions for regression. It is smooth and differentiable, which facilitates gradient-based optimization. It is convex in predictions, and for the linear case, also convex in the parameters. For DL, however, the problems often turn highly non-convex due to non-linear activation functions. As errors are squared, it is always positive, with 0 being a perfect fit. Squaring the errors also makes MSE quite sensitive to outliers. MSE can be a good choice if you are working with errors that follow a Gaussian distribution instead of heavier-tailed distributions.

Mean Absolute Error (MAE)

$$\mathcal{L}_{\text{MAE}}(\theta) = \frac{1}{N} \sum_{i=1}^N |y_i - f_\theta(\mathbf{x}_i)| \quad (2.2)$$

MAE, or L1 loss, is very closely related to MSE, but more robust to outliers. It is the average of the absolute deviation between the true value and the predicted value. Errors are, in contrast to MSE, penalized linearly and are thus less sensitive to outliers. It is also convex in the predictions, but for DL, most likely not in the parameters. As the absolute value function $|\cdot|$ is not differentiable at 0, this requires algorithms to adaptively handle such discontinuities. This can be done in

multiple ways, one of which is subgradients. Generally, using MAE is slightly more computationally taxing than MSE. In contrast to MSE, MAE might be desirable if the error distribution has heavier tails.

Huber

$$\xi_{\text{Huber},i}(\theta, \psi) = \begin{cases} \frac{1}{2}e_i^2, & |e_i| \leq \psi, \\ \psi(|e_i| - \frac{1}{2}\psi), & |e_i| > \psi, \end{cases} \quad e_i \equiv y_i - f_\theta(\mathbf{x}_i),$$

$$\mathcal{L}_{\text{Huber}}(\theta, \psi) = \frac{1}{N} \sum_{i=1}^N \xi_{\text{Huber},i}(\theta, \psi),$$

The Huber loss behaves like MSE for small residuals (smooth and quadratic) and transitions to a linear penalty like MAE for large residuals, offering a tunable balance via the threshold δ . It is, like the others, convex in the predictions, but likely not in the parameters. The Huber loss function is a good choice for regression where outliers or noise are present, but you want to choose when to penalize linearly or quadratically, getting the best of both worlds. While this offers flexibility, it also introduces yet another tunable hyperparameter to the model.

These three loss functions, MSE, MAE, and Huber, span a spectrum of robustness and smoothness properties. In practice, one selects among them based on the expected noise distribution and presence of outliers in the data.

2.1.4 Norm penalization

In many models, especially highly parameterized and flexible ones like neural networks, it's very common to add a *regularization* penalty to the loss function to control model complexity. Regularization can help solve two issues: it can 1) introduce sparsity when you have overparameterization, and 2) introduce shrinkage to avoid overfitting to the specific training sample. In this section, we follow the theory of Hastie, Tibshirani, and Friedman (2009). The overall training objective becomes:

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N} \sum_{i=1}^N \xi(y_i, f_\theta(\mathbf{x}_i))}_{\text{loss function}} + \underbrace{R(\theta)}_{\text{regularizer}},$$

where $\xi(\cdot)$ is your base loss for each sample (MSE, MAE, Huber, etc.) and $R(\theta)$ is a penalty on the parameters. Here, we will cover the regularization we use later on.

LASSO (ℓ_1)

$$R(\theta) = \lambda_{\ell_1} \|\theta\|_1.$$

The ℓ_1 penalty, introduced by Tibshirani (1996), will promote sparsity in parameters via an absolute-value penalty and shrinkage. LASSO corresponds to placing a Laplace (double-exponential) prior on the parameters. In n -dimensional space, the iso-density contour takes the shape of a cross-polytope (a diamond in two-dimensional space). This will often lead to the coefficient being penalized to exactly zero, while others will just be shrunk.

Ridge (ℓ_2)

$$R(\theta) = \lambda_{\ell_2} \|\theta\|_2^2.$$

Also known in the world of neural networks as *weight decay*, formalized by Hoerl and Kennard (1970), is a very common regularizer. The ℓ_2 penalty encourages small weights, thereby providing a smoother learned function. The penalty boundaries of the Ridge regularizer are ellipsoids, and the solution to the penalized minimization never vanishes completely, but shrinks the coefficients toward zero, as opposed to LASSO, which provides both shrinkage and sparsity in parameters.

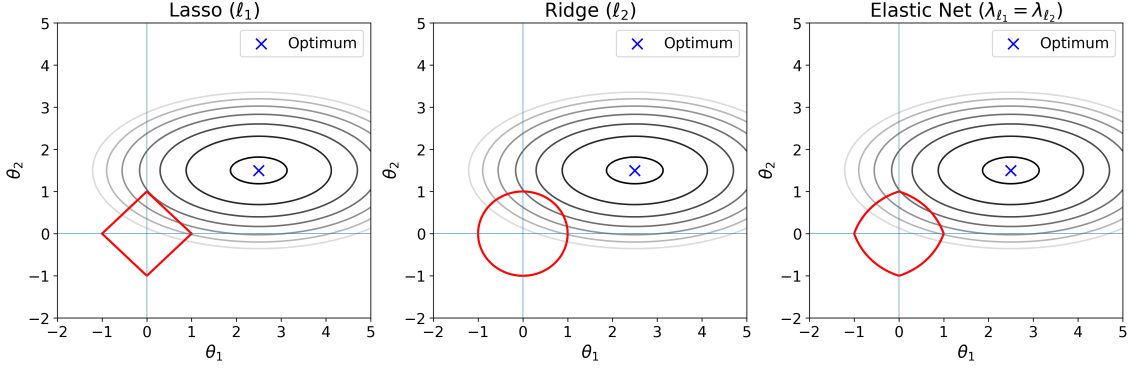
Elastic Net

$$R(\theta) = \lambda_{\ell_1} \|\theta\|_1 + \lambda_{\ell_2} \|\theta\|_2^2.$$

Where Ridge will only provide shrinkage, and LASSO often will introduce sparsity, the elastic net, as proposed by Zou and Hastie (2005), combines ℓ_1 and ℓ_2 penalties to balance sparsity and smoothness. This provides flexibility, which can be very desirable in certain situations, but we also now get two tunable hyperparameters.

To better understand the crucial differences among these regularizers, Figure 2.2 shows a geometric illustration in two-dimensional parameter space with contour plots for Ridge, LASSO, and Elastic Net. The optimal regularized solution is the point of tangency where the smallest iso-loss contour (the ellipses) intersects the penalty boundary. From Figure 2.2, we see how, due to the kink in the absolute value function of the LASSO regularizer, the tangent point with the loss contours will often be at a parameter value of zero, thus introducing sparsity. In contrast, the smooth penalty boundary of Ridge will have a tangency point where parameters are shrunk towards zero, while the elastic net is a linear combination of the two.

Figure 2.2: Contour plots for two-dimensional Ridge, LASSO & Elastic Net



NOTE: Contour-plots for ℓ_1 , ℓ_2 , and Elastic Net ($\lambda_{\ell_1} = \lambda_{\ell_2}$). Blue X denotes the optimal unregularized solution, and the surrounding ellipses are iso-loss contours. The penalty boundary of the constraint regions is shown in red. Optimal penalized solutions will be at the tangency point of the penalty boundary and the iso-loss ellipses.

2.1.5 Network Training

By now, we have a well-defined network. We have a structure, activation functions, and a loss function. Training the networks happens in two steps: forward propagation and backpropagation. The goal is to estimate the weights and biases of the network accurately according to the loss function, until the network is sufficiently trained to capture the explainable variance, without overfitting to the training sample. The theory of this section is from Peña and Tsay (2021).

Sampling

In practice, we begin by partitioning our entire dataset into two major subsets: one for training, which we use to fit the weights and biases, and one for *out-of-sample* (OOS) testing, the *test set*, which we set aside entirely to obtain an unbiased estimate of the model's performance on new data. To avoid overfitting and to guide the choice of architecture and hyperparameters, we typically further carve the training set into two parts. A *training set* on which the network is trained, and a *validation set*, used for model selection and tuning. During training, we monitor performance on the validation set to decide when to stop learning (early stopping) and to compare different network designs and hyperparameter configurations. With the data subdivided, we can now begin training the network.

Forward propagation

Until now, we have primarily focused on the forward flow of information in the networks, called forward propagation. This involves calculating predictions of the

network based on the input data, architecture, and parameters of the model. In the forward pass, each input $\mathbf{x}_i$ is propagated through the layers of the network to produce an output prediction $\hat{y}_i = f_{\theta}(\mathbf{x}_i)$ given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$. The size of the loss, or how well the network fits the training data, depends on the weights and biases in the network. But how do we find these?

Initialization

To initialize the network, one has to randomly draw initial weights and biases and set the hyperparameters of the model. There are multiple approaches to doing so, with Peña and Tsay (2021) simply suggesting drawing parameters uniformly at random from a compact interval. The literature generally has little emphasis on the importance of the initial values of weights and biases, and often simply draws random small weights close to zero. However, the shape of the activation function can be shown to be of some importance when choosing initial weights and biases. Later in this paper, we will be using rectifiers for the activations. There is a general consensus that rectifiers should be initialized according to He et al. (2015). The idea is to scale the variance of the distribution from which you draw to avoid reducing or magnifying the magnitudes of input signals. The weights should be drawn $\mathbf{w}_{mx} \sim \mathcal{N}\left(0, \frac{2}{(1+\alpha^2)M_{l-1}}\right)$, where α is the slope of the Leaky ReLU for $x < 0$, where for ReLU α defaults to 0.

Hyperparameters can be picked arbitrarily within a reasonable compact set, or optimized on the validation set, and for each hyperparameter, there are guidelines in the literature on how to set the hyperparameter. Given the high number of hyperparameters and their mutual dependency, it is generally speaking pointless to try to perfectly optimize the hyperparameterization of the model. One can tune the model by cross-validating a limited set of reasonable guesses on the validation set. We will cover our exact approach for hyperparameterization during the implementation in section 4.1.

Backpropagation

After initialization and the first forward pass, it is time to *train* the network. There are a few approaches to doing so, and often you will hear about first-order *gradient descent*. We will be implementing an algorithm called *Adaptive Moment Estimation (ADAM)* (Kingma and Ba 2017). Although many alternatives exist, including Newton or quasi-Newton methods, in which you also invoke the Hessian to some degree, and population-based methods such as genetic algorithms or other evolutionary algorithms. Because gradient-based methods are computationally efficient and scale well to large parameter sets, we adopt them here, even though they lack some of

the rapid convergence properties of second-order methods. We will now cover the basics of gradient-based algorithms and then build towards ADAM, gradually adding layers of understanding. The pseudocode for all the mentioned backpropagation methods in this section can be found in Appendix A.1. Full-batch updates can be memory-intensive on large datasets.

To do gradient-based optimization, one must compute the gradient $\nabla_{\theta}\mathcal{L}$ by applying the chain rule layer by layer. These gradients can now be used to update the weights and biases of the model, subject to the chosen *learning rate*, η . In full-batch gradient descent, you compute $\nabla_{\theta}\mathcal{L}$ over all N training samples and then update:

$$\theta_t \leftarrow \theta_{t-1} - \eta \nabla_{\theta} \mathcal{L}(\theta_{t-1}).$$

Each time the algorithm sees the entire training set once, we call it an epoch. There are two common alternatives that mitigate this issue. The first is *stochastic gradient descent (SGD)*, where you update the parameters for every sample:

$$\theta_t \leftarrow \theta_{t-1} - \eta \nabla_{\theta} \xi(y_i, f_{\theta}(\mathbf{x}_i)),$$

and thus update N times pr. epoch. The second is *mini-batch gradient descent*, where you subdivide the training sample into batches of size b ,⁵ looping through each batch B_t indexed $t = 1, 2, \dots, T$ for each epoch, with $T = \lceil \frac{N}{b} \rceil$ iterations per epoch:

$$\theta_t \leftarrow \theta_{t-1} - \eta \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \xi(y_i, f_{\theta}(\mathbf{x}_i)),$$

In stochastic and mini-batch gradient descent, you need a simple, unbiased way to pick which samples drive each update. A popular choice is to shuffle and split: at the start of each epoch, randomly permute all N training samples, then split them into consecutive mini-batches of size b (with $b = 1$ reducing to pure SGD). This guarantees every sample contributes exactly one gradient step per epoch, while the random order helps decorrelate successive updates. As an alternative, you can perform online updates by drawing one (or b) sample(s) with replacement at each iteration. Here, one may revisit the same data point before having seen the full set. Both approaches produce unbiased estimates of the full-batch gradient (Goodfellow, Bengio, and Courville 2016).

As these algorithms sometimes get stuck in suboptimal local minima, some algorithms are designed to attempt to escape such minima. Building on mini-batch

⁵If b does not evenly divide N , the last batch contains fewer than b items.

gradient descent, an important-to-know modification is *momentum* as in Sutskever et al. (2013):

$$v_t = \phi v_{t-1} + (1 - \phi) \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \xi(y_i, f_{\theta}(\mathbf{x}_i)),$$

$$\theta_t \leftarrow \theta_{t-1} - \eta v_t.$$

Here, v_t acts as a velocity term, an exponential moving average of recent gradients, controlled by the momentum coefficient ϕ . By mixing in a fraction of the previous velocity with the current gradient, we smooth out noisy updates and build up speed in directions where the loss consistently decreases. The parameters θ are then pushed along this velocity vector. Momentum smooths and accelerates the descent by building up inertia, but it still relies on a global learning rate for every parameter. There are multiple ways of tuning the global learning rate, but in this case, we might want to have a learning rate that adapts to the learning we are doing. We might see noisy directions where a large fixed step overshoots, and shallow or sparse directions where the same step barely moves. Kingma and Ba (2017) introduces one solution to this, ADAM. ADAM combines the aforementioned momentum gradient descent with what is called *Root Mean Square Propagation (RMSProp)*. Here we propose a mini-batch implementation:

$$g_t \equiv \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \xi(y_i, f_{\theta}(\mathbf{x}_i)),$$

$$m_t = \phi_1 m_{t-1} + (1 - \phi_1) g_t,$$

$$v_t = \phi_2 v_{t-1} + (1 - \phi_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \phi_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \phi_2^t},$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

ADAM addresses the issue by coupling momentum with an adaptive per-parameter scaling: it tracks both the first moment and the second moment of past gradients, then divides the bias-corrected first moment by the square root of the bias-corrected second moment. This automatically shrinks updates in high-variance directions and magnifies them in low-variance ones, reducing oscillation, improving convergence speed, and removing much of the pain of hand-tuning learning rates.

2.1.6 Regularization techniques in training

One of the most important, but also quite difficult things in all types of machine learning is to avoid overfitting. Beyond norm penalties on the parameters you can use in conjunction with the loss, there are several techniques that can be applied during training. There are a lot of options, and we will only cover the ones we employ in some capacity later on. In this section, we follow the theory of Goodfellow, Bengio, and Courville (2016).

Early stopping

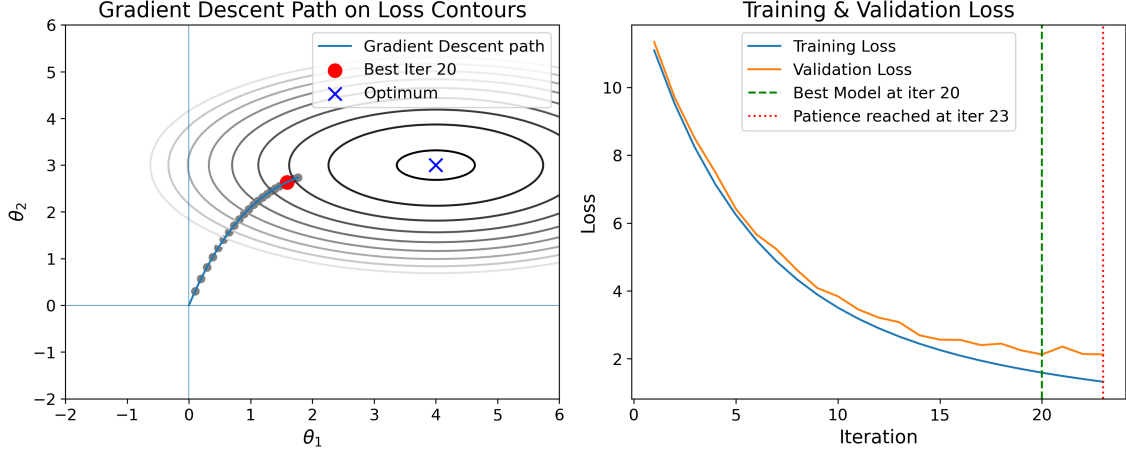
The first, and most commonly seen technique used when training neural networks is called *early stopping*. Early stopping is exactly as it sounds; you stop the algorithm "early." In this context, this means stopping when we can't improve the validation loss anymore. We save a copy of the model parameters each time the loss on the validation set improves, called θ^* . *Patience*, ι , is the maximum allowed number of times we check for improvement in the loss on the validation set, without seeing improvements. We will later implement early stopping, where we check the validation loss after $n = \lceil \frac{N}{b} \rceil$ iterations of weight updating. Thus, patience is denoted in the number of epochs. Once the patience is reached, training stops, and we return θ^* , *not* the latest trained model. The pseudocode for the implementation used in this paper can be found in Appendix A.2.

To illustrate how early stopping works, we have plotted the gradient descent path and training, and validation loss for a hypothetical neural network over parameter $\theta \in \mathbb{R}^2$ in Figure 2.3 with patience, $\iota = 3$. From Figure 2.3, it is evident that for the first 20 iterations, the validation loss generally improves with the training loss. However, after the 20th iteration, the validation loss does not improve for three consecutive periods, triggering the early stopping.

Ensemble methods

Another important technique is *model averaging*. The idea is to reduce the generalization error by combining the output of several trained models. For machine learning in general, we call this strategy model averaging, and the techniques that employ this strategy are called *ensemble methods*. You train an ensemble of models and average the predictions. Most machine learning competitions are won using this technique, as almost all machine learning models benefit from ensembling, often combining more than a dozen models. There are many ways of doing this. You can train models with different activations, loss functions, layers, etc., with the most prominent method being *bagging*, where you sample your data with replacement into smaller bootstrapping data sets, and train models on these. Due to the, for

Figure 2.3: Illustration of early stopping



NOTE: Illustration of early stopping over the $\theta \in \mathbb{R}^2$, where θ is initialized at $\{0,0\}$. The left plot shows the optimal solution for a fully converged network marked by the blue X . The ellipses are iso-loss contours, the gray path is the gradient descent path with all 23 iterations plotted, and the best model is marked with red. On the right plot, the loss for each iteration of gradient descent is shown, with the orange line being the validation loss, the blue line being the training loss, the dashed red line marking the final iteration with after a patience of 3, and the dashed green line marking the minimum validation-loss model achieved during training.

machine learning standards, small amount of data we are working with in this paper, this form of model averaging is not ideal. Instead, we train models using different random initializations, thus potentially leading us to reach different local minima when optimizing. If we, for illustrative purposes, define the prediction of a trained model to be $f_j(\mathbf{x}_i)$ trained with initialization j , the prediction of the ensemble is:

$$\hat{y}_i^{(ensemble)} = \frac{1}{\mathcal{E}} \sum_{j=1}^{\mathcal{E}} f_j(\mathbf{x}_i),$$

where $\mathcal{E}$ is the number of models in the ensemble. In some literature, you will see models contributing to the prediction with different weights; however, in this paper, we use a simple average. Do note that we must take the average of the predictions, which is *not* synonymous with averaging the parameters of the network.

Dropout

The last regularization technique we consider in this paper is dropout. Simply put, dropout involves randomly zeroing out some non-output units in the network. There are two reasons why employing dropout is desirable. First, introducing dropout acts as a regularizer that helps prevent overfitting by randomly turning off neurons. Second,

by doing so, the network automatically becomes easier to train, as a substantial number of neurons are zeroed out, making training less costly both in terms of runtime and memory usage. This makes it feasible to train a larger ensemble of models for model averaging.

More formally, for mini-batch gradient descent, when loading each training sample, we randomly sample a Bernoulli mask vector, $\boldsymbol{\mu}^{(l)}$, for each layer $l = 0, \dots, L$, where $l = 0$ is the input layer:

$$\boldsymbol{\mu}^{(l)} = [\mu_1^{(l)}, \dots, \mu_{M_l}^{(l)}]^\top, \quad \mu_m^{(l)} \stackrel{\text{i.i.d.}}{\sim} \text{Bernoulli}(\chi), \quad \chi = 1 - \rho$$

where ρ is the dropout rate. The mask for each neuron is sampled independently, and ρ is a predefined hyperparameter of the model. We use element-wise multiplication of the mask on the pre-dropout activations and inverted dropout scale during training, so we do not have to scale during testing:

$$\tilde{\mathbf{h}}_i^{(l)} = \frac{\boldsymbol{\mu}^{(l)} \odot \mathbf{h}_i^{(l)}}{\chi},$$

and the next layer's pre-dropout activations become

$$h_{m,i}^{(l+1)} = g_{l+1,m}(\mathbf{w}_m^{(l+1)\top} \tilde{\mathbf{h}}_i^{(l)} + w_{m,0}^{(l+1)}).$$

The probability of zeroing out a unit can be upwards of $\rho = 0.5$, although dropout on the input layer should preferably, as a rule of thumb, not be much higher than $\rho = 0.2$.

2.2 FFNN exemplified

We will now provide an example of a neural network, calculated by hand, to show how the algorithm functions using simple assumptions about the architecture of the network. We will calculate through one pass of forward propagation and one pass of backward propagation of a simple neural network. Even though you would most likely not use an FFNN to solve this kind of task, we can, for this exercise, assume that the data generating process follows:

$$y_i = 2x_{1,i} - 3x_{2,i} + 0.5x_{3,i} + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$$

From this process, we generate the data as seen in table 2.1, setting $\sigma = 0.1$. We set up a simple 2-layer MLP architecture. The network has three nodes in the input layer, corresponding to the three observed variables. For ease of calculation, we

Table 2.1: Five training observations and three out-of-sample test observations

	No.	x_1	x_2	x_3	y
Training	1	-3	2	1	-11.4934
	2	0	0	3	1.6127
	3	-3	1	-2	-9.9532
	4	-3	0	3	-4.5859
	5	2	2	2	-0.9631
Test	6	2	0	-3	2.4041
	7	2	0	0	4.0878
	8	-1	-2	3	5.4950

NOTE: Inputs are drawn i.i.d. as $x_1, x_2, x_3 \sim \text{Unif}\{-3, \dots, 3\}$. The target is $y_i = 2x_{1i} - 3x_{2i} + 0.5x_{3i} + \varepsilon_i$ with additive Gaussian noise $\varepsilon \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 0.1^2)$.

include three nodes in the hidden layer, and as we are only trying to predict y_i , one node in the output layer, as illustrated in the topology diagram in Figure 2.4. Hence, we have input vectors $\mathbf{x}_i = (x_{1,i}, x_{2,i}, x_{3,i})^\top$, output vector from the hidden layer $\mathbf{h}_i = (h_{1,i}, h_{2,i}, h_{3,i})^\top$ and output $\hat{y}_i$. The model of the network becomes:

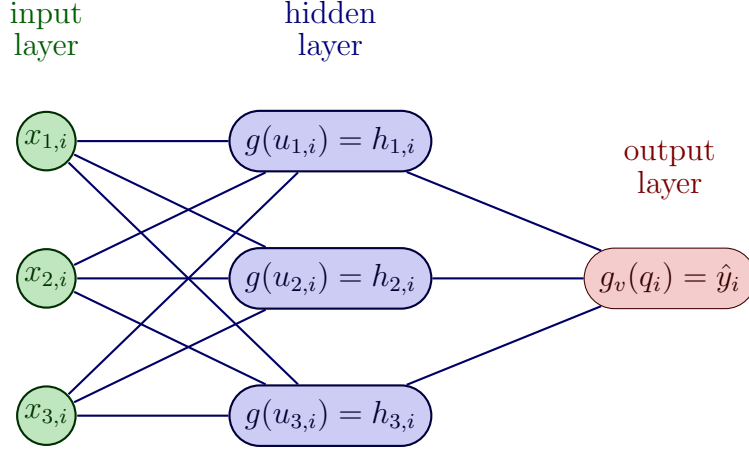
$$\begin{aligned}
 u_{m,i} &= \mathbf{w}_m^\top \mathbf{x}_i, & m &= 1, 2, 3, \\
 h_{m,i} &= g(u_{m,i}), \\
 q_i &= \boldsymbol{\beta}^\top \mathbf{h}_i, \\
 \hat{y}_i &= f(x_i) = g_v(q_i),
 \end{aligned}$$

where $\mathbf{w}_m = (w_{m,1}, w_{m,2}, w_{m,3})^\top$ are the weights of the m th node in the hidden layer, $\boldsymbol{\beta} = (\beta_1, \beta_2, \beta_3)^\top$ are the weights of the output layer with $u_{m,i}$ and q_i being the pre-activation of the hidden and output layer, respectively. We optimize over the parameter space $\Theta = (\mathbb{R}^3)^4$. To simplify this example, and without loss of generality, we have not introduced any biases. $g(\cdot)$ and $g_v(\cdot)$ are the activation functions of the hidden layer and output layer, respectively. For the output layer, we use identity activation such that $\hat{y}_i = g_v(q_i) = q_i$, for reasons previously described. For the hidden layer, we will use a Leaky ReLU function:

$$g(u_{m,i}) = \begin{cases} u_{m,i}, & u_{m,i} \geq 0, \\ \alpha u_{m,i}, & u_{m,i} < 0, \end{cases} \quad g'(u_{m,i}) = \begin{cases} 1, & u_{m,i} \geq 0, \\ \alpha, & u_{m,i} < 0. \end{cases} \quad (2.3)$$

This is used to avoid the issue of "dead" neurons associated with the typical ReLU where $\alpha = 0$. For this exercise, we will choose $\alpha = 0.1$. We also need a well-defined

Figure 2.4: Fully Connected 3-3-1 MLP Topology Diagram



NOTE: Topology diagram of the model showing the three input features $\mathbf{x}_i = (x_{1,i}, x_{2,i}, x_{3,i})^\top$, the hidden layer of three neurons with post-activation $\mathbf{h}_i = (h_{1,i}, h_{2,i}, h_{3,i})^\top$, and the output layer with prediction $\hat{y}_i$. $u_{m,i}$ and q_i are the pre-activations of the hidden layer and the output layer, respectively. $g(\cdot)$ and $g_v(\cdot)$ are the activation functions of the hidden layer and the output layer, respectively.

loss function, so we can calculate the derivatives for gradient descent and set a learning rate. For this, we will use the MSE loss function:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

Here we scaled the loss with $\frac{1}{2}$ to simplify the derivatives. We can now calculate the gradients used for backpropagation. To do this, we take the partial derivative with respect to the weights using the chain rule. Calculations can be found in Appendix A.3. For β we have equation (A.1):

$$\nabla_{\beta} \mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \mathbf{h}_i, \quad (2.4)$$

and for $\mathbf{w}_m$ we have equation (A.2):

$$\nabla_{\mathbf{w}_m} \mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \beta_m \mathbf{x}_i g'(u_{m,i}), \quad g'(u_{m,i}) = \begin{cases} 1, & u_{m,i} \geq 0, \\ 0.1, & u_{m,i} < 0. \end{cases} \quad (2.5)$$

With learning rate η , we update all weights once per epoch for batch gradient descent:

$$\mathbf{w}_m \leftarrow \mathbf{w}_m - \eta \nabla_{\mathbf{w}_m} \mathcal{L}_{\text{MSE}}, \quad m = 1, 2, 3, \quad (2.6)$$

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \eta \nabla_{\boldsymbol{\beta}} \mathcal{L}_{\text{MSE}}. \quad (2.7)$$

Where we, for ease of notation, have not introduced a loop indexation, as we will only be doing one pass. The gradients are calculated in the initial weights, and then we simply "overwrite" $\mathbf{w}_m$ and $\boldsymbol{\beta}$ with the new values. In this case, we use learning rate $\eta = 0.1$. The last thing we need before we can start doing a pass of forward propagation is the starting values of the weights. In this case, we use He/Kaiming initialization following He et al. (2015). For the hidden layer, we use activation Leaky ReLU, and therefore weights are drawn $\mathbf{w}_{m,x} \sim \mathcal{N}(0, \frac{2}{(1+\alpha^2)|\mathbf{x}_i|})$. For the output layer, we use identity activation, which corresponds to a leaky ReLU with $\alpha = 1$. Therefore weights $\boldsymbol{\beta}$ are drawn $\boldsymbol{\beta}_m \sim \mathcal{N}(0, \frac{1}{|\mathbf{h}_i|})$. Thus, we draw initial weights:

$$\mathbf{w}_1 = \begin{bmatrix} -0.1502 \\ -0.5532 \\ 0.9932 \end{bmatrix}, \mathbf{w}_2 = \begin{bmatrix} -0.1255 \\ -0.3480 \\ -0.2861 \end{bmatrix}, \mathbf{w}_3 = \begin{bmatrix} 0.4325 \\ 0.2969 \\ 0.3353 \end{bmatrix}, \boldsymbol{\beta} = \begin{bmatrix} 0.2487 \\ 1.2365 \\ -0.2346 \end{bmatrix}.$$

We are now ready to begin forward propagation. We will conduct one pass, here we will only show the calculations related to the first observation. For the remaining observations, calculations can be found in Appendix A.4. For the first observation, we have predictors, and for reference, the target:

$$\mathbf{x}_1^\top = [-3, 2, 1], \quad y_1 = -11.4934,$$

Thus the prediction becomes:

$$\begin{aligned} u_{1,1} &= \mathbf{w}_1^\top \mathbf{x}_1 = (-0.1502)(-3) + (-0.5532)(2) + (0.9932)(1) = 0.3374, \\ u_{2,1} &= \mathbf{w}_2^\top \mathbf{x}_1 = (-0.1255)(-3) + (-0.3480)(2) + (-0.2861)(1) = -0.6056, \\ u_{3,1} &= \mathbf{w}_3^\top \mathbf{x}_1 = (0.4325)(-3) + (0.2969)(2) + (0.3353)(1) = -0.3684, \\ h_{1,1} &= g(u_{1,1}) = u_{1,1} = 0.3374, \\ h_{2,1} &= g(u_{2,1}) = 0.1u_{2,1} = 0.1(-0.6056) = -0.0606, \\ h_{3,1} &= g(u_{3,1}) = 0.1u_{3,1} = 0.1(-0.3684) = -0.0368, \\ q_1 &= \boldsymbol{\beta}^\top \mathbf{h}_1 = (0.2487)(0.3374) + (1.2365)(-0.0606) + (-0.2346)(-0.0368) \\ &= 0.0177 \end{aligned}$$

$$\hat{y}_1 = g_v(q_1) = q_1 = 0.0177. \quad (2.8)$$

Including the calculations found in Appendix A.4 we get the output predictions, $\hat{\mathbf{y}}_{train}$ as:

$$\hat{\mathbf{y}}_{train} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \hat{y}_4 \\ \hat{y}_5 \end{bmatrix} = \begin{bmatrix} 0.0177 \\ 0.3989 \\ 0.7300 \\ 0.8004 \\ -0.5433 \end{bmatrix}.$$

From here, we can now show a pass of backward propagation. Using the predictions and equations (2.4) and (2.5) to calculate the gradients. for β :

$$\begin{aligned} \nabla_{\beta} \mathcal{L}_{MSE} &= \frac{1}{5} \sum_{i=1}^5 (\hat{y}_i - y_i) \mathbf{h}_i \\ &= \frac{1}{5} \left[(0.0177 - (-11.4934)) \begin{pmatrix} 0.3374 \\ -0.0606 \\ -0.0368 \end{pmatrix} \right. \\ &\quad + (0.3989 - 1.6127) \begin{pmatrix} 2.9796 \\ -0.0858 \\ 1.0059 \end{pmatrix} \\ &\quad + (0.7300 - (-9.9532)) \begin{pmatrix} -0.2089 \\ 0.6007 \\ -0.1671 \end{pmatrix} \\ &\quad + (0.8004 - (-4.5859)) \begin{pmatrix} 3.4302 \\ -0.0482 \\ -0.0292 \end{pmatrix} \\ &\quad \left. + (-0.5433 - (-0.9631)) \begin{pmatrix} 0.5796 \\ -0.1519 \\ 2.1294 \end{pmatrix} \right] \\ &= \frac{1}{5} \begin{pmatrix} 16.7631 \\ 5.5032 \\ -2.6928 \end{pmatrix} = \begin{pmatrix} 3.3526 \\ 1.1006 \\ -0.5386 \end{pmatrix}. \end{aligned}$$

For the gradients for $\mathbf{w}_m$ for $m = 1, 2, 3$, we will show the calculations for $m = 1$ here, and the rest can be found in Appendix A.5:

$$\begin{aligned}
\nabla_{\mathbf{w}_1} \mathcal{L}_{\text{MSE}} &= \frac{1}{5} \sum_{i=1}^5 (\hat{y}_i - y_i) \beta_1 g'(u_{1,i}) \mathbf{x}_i \\
&= \frac{1}{5} \left[(0.0177 + 11.4934) \cdot 0.2487 \cdot 1 \begin{pmatrix} -3 \\ 2 \\ 1 \end{pmatrix} \right. \\
&\quad + (0.3989 - 1.6127) \cdot 0.2487 \cdot 1 \begin{pmatrix} 0 \\ 0 \\ 3 \end{pmatrix} \\
&\quad + (0.7300 + 9.9532) \cdot 0.2487 \cdot 0.1 \begin{pmatrix} -3 \\ 1 \\ -2 \end{pmatrix} \\
&\quad + (0.8004 + 4.5859) \cdot 0.2487 \cdot 1 \begin{pmatrix} -3 \\ 0 \\ 3 \end{pmatrix} \\
&\quad \left. + (-0.5433 + 0.9631) \cdot 0.2487 \cdot 1 \begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix} \right] \\
&= \frac{1}{5} \begin{pmatrix} -13.1954 \\ 6.2000 \\ 5.6534 \end{pmatrix} = \begin{pmatrix} -2.6391 \\ 1.2400 \\ 1.1307 \end{pmatrix},
\end{aligned}$$

From equation (A.6):

$$\nabla_{\mathbf{w}_2} \mathcal{L}_{\text{MSE}} = \begin{pmatrix} -9.1587 \\ 3.2321 \\ -4.6689 \end{pmatrix},$$

and from equation (A.7):

$$\nabla_{\mathbf{w}_3} \mathcal{L}_{\text{MSE}} = \begin{pmatrix} 0.3488 \\ -0.1975 \\ 0.1019 \end{pmatrix}.$$

These can now be used to update the 12 parameters of the model according to equations (2.6) and (2.7):

$$\begin{aligned}
\mathbf{w}_1 &\leftarrow \mathbf{w}_1 - 0.1 \nabla_{\mathbf{w}_1} \mathcal{L}_{\text{MSE}} = \begin{pmatrix} -0.1502 \\ -0.5532 \\ 0.9932 \end{pmatrix} - 0.1 \begin{pmatrix} -2.6391 \\ 1.2400 \\ 1.1307 \end{pmatrix} = \begin{pmatrix} 0.1137 \\ -0.6772 \\ 0.8802 \end{pmatrix}, \\
\mathbf{w}_2 &\leftarrow \mathbf{w}_2 - 0.1 \nabla_{\mathbf{w}_2} \mathcal{L}_{\text{MSE}} = \begin{pmatrix} -0.1255 \\ -0.3480 \\ -0.2861 \end{pmatrix} - 0.1 \begin{pmatrix} -9.1587 \\ 3.2321 \\ -4.6689 \end{pmatrix} = \begin{pmatrix} 0.7904 \\ -0.6712 \\ 0.1808 \end{pmatrix}, \\
\mathbf{w}_3 &\leftarrow \mathbf{w}_3 - 0.1 \nabla_{\mathbf{w}_3} \mathcal{L}_{\text{MSE}} = \begin{pmatrix} 0.4325 \\ 0.2969 \\ 0.3353 \end{pmatrix} - 0.1 \begin{pmatrix} 0.3488 \\ -0.1975 \\ 0.1019 \end{pmatrix} = \begin{pmatrix} 0.3976 \\ 0.3167 \\ 0.3251 \end{pmatrix}, \\
\boldsymbol{\beta} &\leftarrow \boldsymbol{\beta} - 0.1 \nabla_{\boldsymbol{\beta}} \mathcal{L}_{\text{MSE}} = \begin{pmatrix} 0.2487 \\ 1.2365 \\ -0.2346 \end{pmatrix} - 0.1 \begin{pmatrix} 3.3526 \\ 1.1006 \\ -0.5386 \end{pmatrix} = \begin{pmatrix} -0.0866 \\ 1.1264 \\ -0.1807 \end{pmatrix}.
\end{aligned}$$

We have now successfully gone through one pass of backward propagation. This leaves us with new weights for the model. Since our model uses full-batch updating, we have updated the weights after exactly one epoch, meaning the model has "seen" our training data exactly once. In reality, we would keep training this model to get a better fit while, of course, keeping in mind that we should not overfit with too many epochs. There is a trade-off between the number of epochs and the learning rate. A higher learning rate is not compatible with many epochs, as this will lead to overfitting. Likewise, a very low learning rate and few epochs will lead to underfitting.

For the sake of this exercise, we can now showcase that doing gradient descent actually improved the model. Thus, we will now calculate and compare the root mean squared error (RMSE) of the original model, where all weights were drawn randomly, with the RMSE of the new model with updated weights after one pass of backward propagation. To do this, we will use the test data in table 2.1. Forward propagation is done as for equation (2.8), and calculations can be found in Appendix A.6:

$$\hat{\mathbf{y}}_{\text{orig}} = \begin{pmatrix} \hat{y}_6^{(\text{orig})} \\ \hat{y}_7^{(\text{orig})} \\ \hat{y}_8^{(\text{orig})} \end{pmatrix} = \begin{pmatrix} 0.6720 \\ -0.2415 \\ 1.0499 \end{pmatrix}, \quad \hat{\mathbf{y}}_{\text{new}} = \begin{pmatrix} \hat{y}_6^{(\text{new})} \\ \hat{y}_7^{(\text{new})} \\ \hat{y}_8^{(\text{new})} \end{pmatrix} = \begin{pmatrix} 1.1941 \\ 1.6172 \\ 0.8946 \end{pmatrix}.$$

We can now calculate RMSE:

$$\begin{aligned}\text{RMSE}_{\text{orig}} &= \sqrt{\frac{1}{3} \left[(2.4041 - 0.6720)^2 + (4.0878 - (-0.2415))^2 + (5.4950 - 1.0499)^2 \right]} \\ &= 3.7194, \\ \text{RMSE}_{\text{new}} &= \sqrt{\frac{1}{3} \left[(2.4041 - 1.1928)^2 + (4.0878 - 1.6173)^2 + (5.4950 - 0.8971)^2 \right]} \\ &= 3.0953.\end{aligned}$$

From which we can see that our network has improved the RMSE through backpropagation in just one epoch.

2.3 Linear models

To put neural networks, MLPs in particular, into perspective, we will compare them to the classic linear estimators OLS and LASSO. We will now briefly lay out the underlying theory behind the linear models.

2.3.1 OLS estimator

Ordinary Least Squares (OLS) is a simple yet elegant model for estimating linear relationships. As it is irrelevant in this context, this section will not focus on the properties of the OLS estimator but rather on the use case, so it can serve as a gateway to understanding why neural networks might be of interest in a financial context. OLS assumes that the conditional mean of a scalar response is a *linear* function of a set of predictors:

$$y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \varepsilon_i \quad (2.9)$$

where $y_i \in \mathbb{R}$, $\mathbf{x}_i \in \mathbb{R}^P$, and ε_i is the error term. OLS is the only model of the three described in this paper that has a closed-form solution. To solve the model, we minimize the squared residuals:

$$\text{RSS}(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2$$

and the OLS estimator is derived in Appendix A.7 as:

$$\hat{\boldsymbol{\beta}}^{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (2.10)$$

The OLS estimator performs well when the true model is as close to linear as possible, and the number of predictors P is small relative to the sample size N .

So, when might OLS struggle in financial forecasting? Modern factor models, as covered in the literature review, can include hundreds of features (technical indicators, fundamentals, sentiment scores, etc.), so P can approach or exceed N , ultimately making the design matrix $(\mathbf{X}^\top \mathbf{X})$ uninvertible as the matrix no longer holds full rank. Furthermore, threshold effects, volatility regimes, and interactions amongst variables rarely conform to the strict linear form assumed in OLS.

2.3.2 LASSO estimator

The Least Absolute Shrinkage and Selection Operator (LASSO) regression as proposed by Tibshirani (1996) augments the classical least-squares criterion with an ℓ_1 penalty on the coefficient vector. With the vast amount of potential predictor candidates, it is likely that the models are over-dimensional. Having a way of reducing dimensions can greatly help improve the linear model that struggles in higher-dimensional settings characterized by a low signal-to-noise ratio. Harvey, Liu, and Zhu (2015) writes, "We argue that most claimed research findings in financial economics are likely false," which, *ceteris paribus*, must imply substantial amounts of noise in large factor models. Where OLS insists on fitting every predictor in the matrix, LASSO encourages sparsity by shrinking its estimated coefficients, often to zero, and thereby performing automatic variable selection. Its optimization problem is defined as

$$\hat{\boldsymbol{\beta}}^{\text{LASSO}}(\lambda) \in \underset{\boldsymbol{\beta} \in \mathbb{R}^P}{\operatorname{argmin}} \left(\frac{\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2}{2N} + \lambda \|\boldsymbol{\beta}\|_1 \right), \quad \|\boldsymbol{\beta}\|_1 := \sum_{j=1}^P |\beta_j|, \quad (2.11)$$

where $\lambda \geq 0$ tunes the strength of the regularizing term. We have scaled the MSE by $\frac{1}{2}$, as is commonly done, as this simplifies the derivative, but this only affects the level of λ . Setting $\lambda = 0$ reduces LASSO to the OLS regression; increasing λ progressively contracts the feasible region as visualized in Figure 2.2.

In a forecasting context, this mechanism is invaluable as modern factor datasets can contain hundreds of indicators. When the number of candidate regressors approaches or overtakes the available sample size, the OLS normal equations become ill-conditioned or singular. The ℓ_1 penalty counteracts that proliferation by favouring a sparse subset of genuinely informative variables, thereby reducing estimation variance, mitigating multicollinearity, and yielding a model that favors parsimony and is, most likely, more interpretable.

Unlike OLS, LASSO does not possess a closed-form expression akin to equation (2.10). The absolute-value term $\|\boldsymbol{\beta}\|_1 = \sum_{j=1}^P |\beta_j|$ is not differentiable at zero, so we cannot write a simple matrix inverse. Instead, implementations leverage numerical methods to solve the problem, exploiting convexity. In a forecasting setting, as ours

in Section 2.5, we choose the regularization parameter λ via K -fold cross-validation (CV), using `scikit-Learn` implementation `LassoCV`.⁶ This method first partitions the training sample into K roughly equal *folds*. For each candidate λ , in a predefined grid, we then train the model on $K - 1$ folds using a coordinate descent algorithm that updates one coefficient at a time via soft-thresholding. Then the out-of-sample error is computed on the remaining fold. Cycling through all K folds ensures each subset serves once as the validation set, for each candidate λ . It then averages the K validation errors to assign a CV score to each λ . Finally, we select the λ that minimizes this average out-of-sample error and refit the model on the entire training data using that optimal value.

2.4 What is good forecasting

By now, we should have a strong understanding of what neural networks are and how they function. We have also briefly introduced LASSO and OLS, so we have some well-known models that can help us put neural networks in relief. But for this comparison to be meaningful, we first need to agree on a common language to understand what good forecasting is. By doing this, we can have meaningful discussions on what model performs best.

The literature has many guesses as to what good forecasting is. Just like we minimize a loss (or objective) function when fitting the model, we use these loss functions to evaluate the models. This happens on the out-of-sample test data. As per Masters (1993b), MSE, as we define it in section 2.1.3 equation 2.1, is the most often encountered metric in the literature. You will often see the root of MSE, as this rescales the loss to have the same unit as the predicted variable:

$$\mathcal{L}_{\text{RMSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f_{\theta}(\mathbf{x}_i))^2}$$

It is a great measurement for the easy intuition behind it. The further away from the true value the prediction is, the worse we consider the forecast. RMSE also puts more emphasis on large errors than on small ones, which in some situations can be desirable. We also report MAE as defined in 2.1.3 equation 2.2. MAE is a close relative to RMSE, but will put less emphasis on large errors. MAE can be of great interest in finance, where you might not care too much about outliers, as these will often be hard or borderline impossible to forecast. MAE will, compared to RMSE, put more emphasis on the more plausible outcomes.

When looking at the loss scores, we should take care in interpreting the differences

⁶Pedregosa et al. 2011.

between models. MAE corresponds to the actual average absolute error of the model, so when the target variable is excess returns in decimals, as ours will be, a decrease of 0.01 is equivalent to a, on average, 1%-point improvement in the predictions. As RMSE is the root of the average of the squared errors, we cannot make the same claims, and RMSE is best used to rank the models. Due to the nature of these models, we do not find any tools to measure statistical significance appropriate.

Neither RMSE nor MAE is a do-it-all metric. The first and arguably most important disadvantage, as framed by Masters (1993b), is that they are purely mathematical constructs. They can have little to no meaning with respect to the task performed by the model. They are only concerned with the *forecasting accuracy* of the model.

2.4.1 Forecasting for trading purposes

When you apply these models in *algorithmic investment strategies (AIS)*, classical forecast error functions such as MSE and MAE are not necessarily suited for assessing a model’s forecasting ability. As per Michańków, Sakowski, and Ślepaczuk (2023), when using the models to inform investment strategies, the overall objective is to make good investment decisions, not necessarily accurate forecasting. If you can systematically forecast the direction of the return correctly, you will achieve good portfolio performance even if your model’s forecasting accuracy is poor. Imagine your model forecasts a 1% return next month, so you buy the stock, but it ends up returning 15%. Your forecast was off by 14%-points, but the outcome was desirable because you invested due to the directional signal. Now imagine the realized return was actually -1% . This outcome is undesirable because you lost money, but your forecast was, in percentage points, much more accurate at an error of just 2%-points. To mitigate this issue, Michańków, Sakowski, and Ślepaczuk (ibid.) propose an alternative loss function called *Mean Absolute Directional Loss (MADL)*. The function evaluates the loss based solely on the forecasting ability of directional investment signals:

$$\mathcal{L}_{MADL} = \frac{1}{N} \sum_{i=1}^N (-1) \cdot \text{sign}(y_i \cdot \hat{y}_i) \cdot |y_i|$$

where the $\text{sign}(X)$ function returns -1 if $X < 0$, 0 if $X = 0$, and 1 if $X > 0$. $|y_i|$ is the absolute value of the true return. Hence, this function evaluates whether the prediction will result in profit or loss, and the amount of this profit or loss. As this is a loss function, profit is counted with a negative sign and vice versa. The interpretation is like other loss functions; the lower the loss, the better the model.

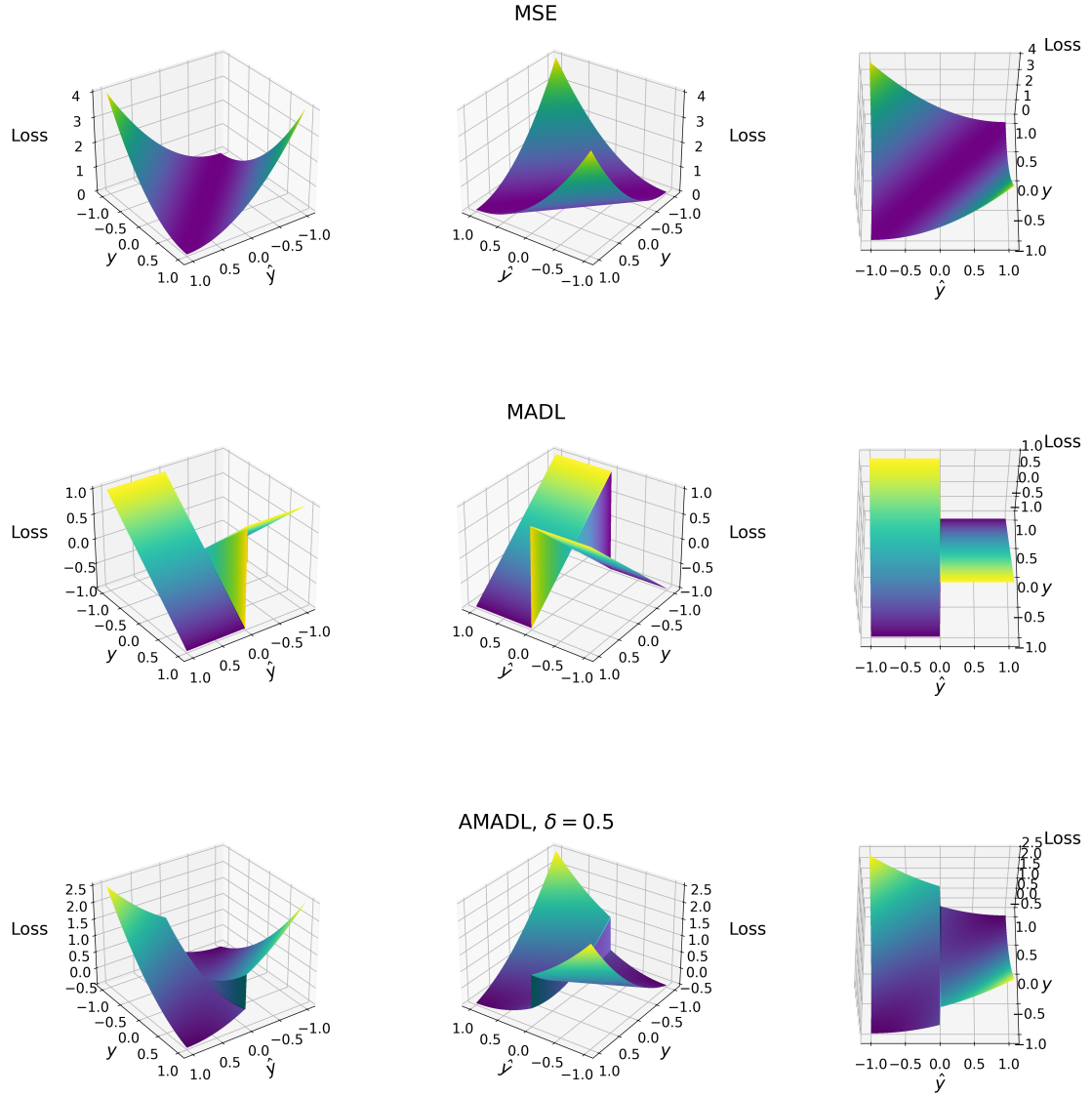
While Michańków, Sakowski, and Ślepaczuk (2023) address a very important issue, which is the mismatch between how the models are evaluated and what their use case is, we, however, find the proposed solution to be a deficient answer to the issue. The proposed implementation imposes a strong assumption on the forecasting evaluation, which is that forecasting accuracy should have absolutely no impact on the performance of the model. The current implementation only penalizes/rewards buy/sell signals. But if the aim of the model is to inform investment decisions, the *magnitude* of the signal cannot be completely neglected. This corresponds to ignoring the opportunity cost of investing in the asset. MADL is negative (corresponding to good performance) as long as an investment decision based on the forecast gives positive returns. But how much you invest in a given asset must depend on the magnitude of the signal; if a buy signal is weak, meaning the expected return is positive but low (e.g. 0.1%), a portfolio manager should be less willing to invest in that than in an alternative where the signal is strong, corresponding to a high expected positive return (e.g. 10%). By this logic, if the model is also able to provide information on forecast accuracy, such a model should be preferred, as a more accurate forecast can inform the asset allocation of the portfolio.

We adapt MADL by proposing a novel loss function, the *Augmented Mean Absolute Directional Loss (AMADL)*:

$$\mathcal{L}_{AMADL} = \frac{1}{N} \sum_{i=1}^N (\delta - 1) \cdot \text{sign}(y_i \cdot \hat{y}_i) \cdot |y_i| + \delta (y_i - \hat{y}_i)^2,$$

where forecast errors are penalized quadratically, akin to the penalization in MSE. Here, $\delta \in [0,1]$ tunes the relative importance of forecasting ability of investment signal direction and forecast accuracy. For $\delta = 1$ AMADL collapses to pure MSE, and for $\delta = 0$ AMADL is synonymous with MADL. In Figure 2.5, the loss surfaces for MADL, MSE and AMADL ($\delta = 0.5$) can be seen. By adding in the quadratic error, the otherwise flat surface of the MADL loss takes on some of the convex properties of the MSE. You will see a general inflection point travelling along the 45° line of AMADL where $y_i = \hat{y}_i$, which is the preference for forecasting accuracy from the quadratic error. In $\hat{y}_i = 0$, you see a wall reflecting the preference for correct signal forecasting as in MADL, that flips around $y_i = \hat{y}_i = 0$. The new AMADL surface possesses the exact properties we desire to inform investment decisions. If $y_i = 1$ and you forecasted $\hat{y}_i = 0.01$, the loss is much lower than if you had forecasted $\hat{y}_i = -0.01$. But the two-dimensional iso-return curve for $y_i = 1$ continuously has a negative first-order derivative but positive second-order as $\hat{y}_i$ moves towards y_i , until the 45° line where $y_i = \hat{y}_i$. In other words, we are happy that we predicted the

Figure 2.5: Loss surfaces of the MSE, MADL, and AMADL functions



NOTE: Loss surfaces for MSE, MADL, and AMADL ($\delta = 0.5$) loss functions plotted for $y_i, \hat{y}_i \in [-1, 1]$.

correct sign, but if the model had been able to predict $\hat{y}_i$ closer to y_i , that would have been better. Had it been more certain that the stock would return 100%, it would have made sense to invest more.

2.4.2 Portfolio performance metrics

To evaluate the portfolios formed by an AIS, multiple metrics exist, depending on what one deems desirable. We will be considering the Sharpe ratio, maximum drawdown, maximum 1-month loss, and turnover.

The Sharpe ratio is perhaps one of the most well-known measures first presented in Sharpe (1966). This ratio measures the excess return to volatility in a portfolio.

$$Sharpe = \frac{r_{\text{portfolio}}^{\text{excess}}}{\sigma_{\text{portfolio}}}$$

When a portfolio has a high Sharpe ratio, the investor is being well compensated for the risk they are exposed to, compared to portfolios with lower Sharpe ratios. Thereby, not saying that the portfolio with the highest Sharpe ratio delivers the highest returns, but rather compensates the investor the most for the risk they are exposed to. Hence, it is desirable to have a high Sharpe ratio. The Sharpe ratio uses volatility as a proxy for risk; however, we acknowledge that this approach has limitations, as volatility does not capture all dimensions of risk. Nonetheless, it remains the most commonly used measure in the literature.

We also report maximum drawdown, meaning the peak-to-trough decline in cumulative returns over the sample period:

$$Max\ DD = \max_{0 \leq t_1 \leq t_2 \leq T} \frac{R_{t_1} - R_{t_2}}{R_{t_1}},$$

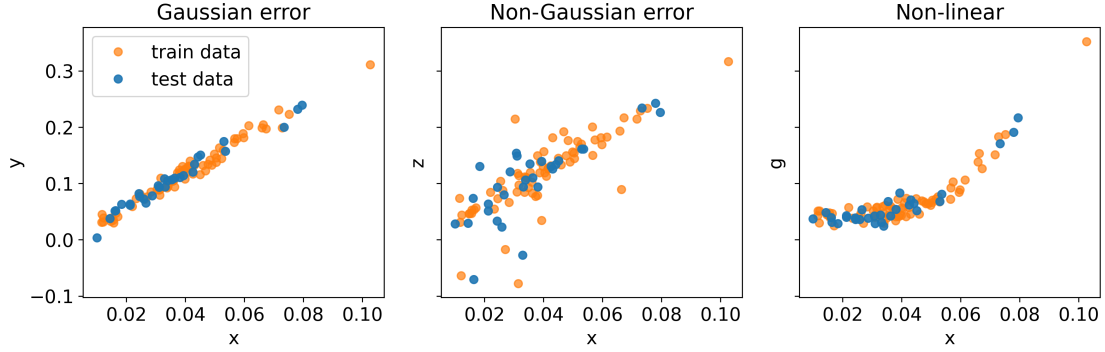
where R_t is the cumulative return of the portfolio at time $t = 0, 1, \dots, T$. A higher maximum drawdown reflects a larger loss from peak value, which is generally undesirable from a risk management perspective. We also report the maximum one-month loss, defined as the largest single-period loss in the portfolio return series:

$$\text{Max 1-M Loss} = \max_{1 \leq t \leq T} -r_t,$$

where r_t denotes the portfolio return in month t . This metric captures the worst monthly performance experienced over the evaluation period and provides insight into the short-term downside risk of the strategy.

Lastly, we report turnover for our portfolios. The turnover in our context is the

Figure 2.6: Scatter plot of simulated data for equations presented in equations (2.12)-(2.14)



NOTE: Data randomly drawn from (2.12)-(2.14) and randomly split into test and train sets with 70% pertaining to the training set.

average monthly change in stockholdings. We define the monthly turnover as:

$$\text{Turnover} = \frac{1}{T} \sum_{t=1}^T \left(\sum_i \left| \mathcal{W}_{i,t} - \frac{\mathcal{W}_{i,t-1} \cdot (1 + r_{i,t})}{1 + \sum_j \mathcal{W}_{j,t-1} r_{j,t}} \right| \right),$$

where $\mathcal{W}_{i,t}$ is the weight of stock i at time t in the portfolio. This measure, in part, captures how much monthly rebalancing the portfolio requires and is thus related to the trading costs a given portfolio will incur. It is therefore desirable to have a low turnover in a portfolio, all else equal.

2.5 Model simulations

To illustrate how the methods specified in Sections 2.1, 2.3.1, and 2.3.2 differ, we simulate three different scenarios. The data generating processes are:

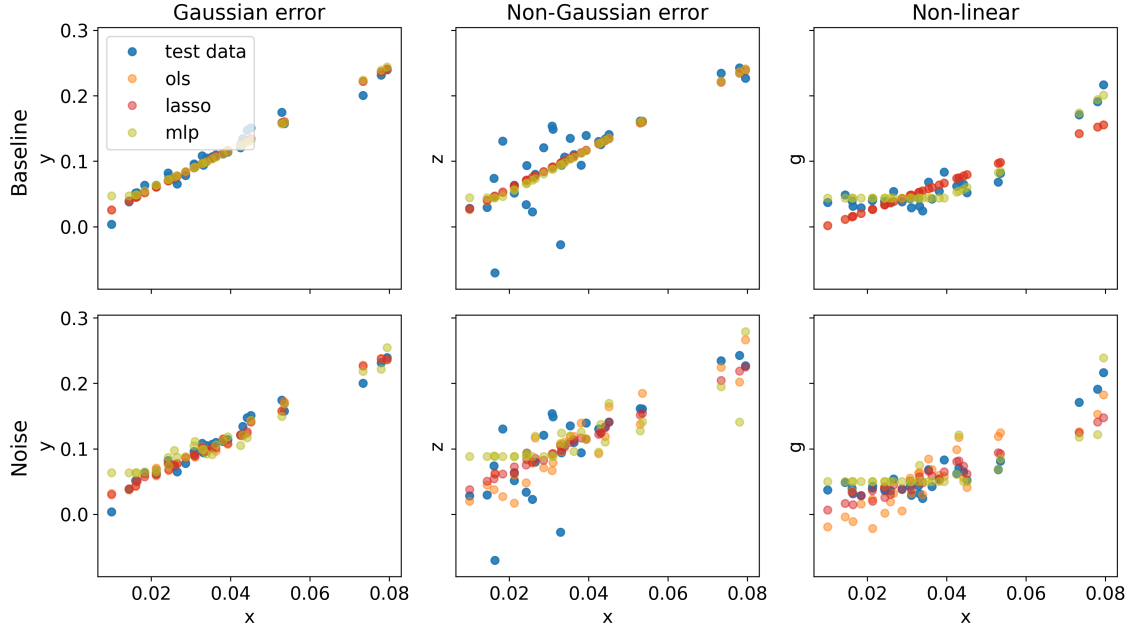
$$y_i = \Psi x_i + \varepsilon_i, \quad \varepsilon_i \sim N(0, 0.01), \quad (2.12)$$

$$z_i = \Psi x_i + \varepsilon_i, \quad \varepsilon_i \sim t_1(0, 0.01), \quad (2.13)$$

$$o_i = \Psi^3 x_i^2 + |x_i - \Omega| + \varepsilon_i, \quad \varepsilon_i \sim N(0, 0.01), \quad (2.14)$$

where y_i is a linear model with normal errors, z_i is a linear model with errors drawn from a Student-t distribution with one degree of freedom, and o_i is a non-linear model. For each model, we simulate 100 data points with $\Psi = 3$ and $\Omega = 0.05$ to get the data shown in Figure 2.6. Of the 100 simulated observations generated under each data-generating process, 70% (shown in orange) serve as the training sample, and the remaining 30% (shown in blue) comprise the out-of-sample test set.

Figure 2.7: Scatter plot of predictions for different models



NOTE: Data randomly drawn from (2.12)-(2.14). Noise specification adds an additional 19 noise variables randomly drawn from (2.15). OLS and LASSO are trained on all 70 training datapoints whilst the MLP FFNN is trained on 63 datapoints and validated up against 7 datapoints from the training set.

To evaluate and compare predictive performance, we estimate each model on the training data and compute forecast accuracy on the OOS data. The LASSO is implemented via `scikit-learn`'s `LassoCV` (Pedregosa et al. 2011) using a five-fold cross-validation routine over a logarithmically spaced penalty grid of 10,000 points on $[10^{-10}, 10]$. The *baseline* FFNN is a fully connected 1–2–1 MLP with ReLU activations, implemented in PyTorch (Paszke et al. 2019). We train the network for up to 1,000 epochs, with early stopping, using the ADAM optimizer on the full batch, and a learning rate of $\eta = 5 \cdot 10^{-3}$ and 10% of the training sample reserved for validation. Out-of-sample predictions of the simulations are found in the top three plots of Figure 2.7 labeled Baseline. Error metrics for the simulation are found in Table 2.2.

In the baseline linear simulation with Gaussian errors, all three methods yield virtually identical out-of-sample performance. Under the linear simulation with Student's *t* errors, differences emerge. Because LASSO can shrink coefficients exactly to zero, it estimates the intercept correctly at zero, whereas OLS cannot, resulting in a slightly larger Ψ though for the specific sample, this difference is negligible for predictive accuracy as seen in Table 2.2. The 1–2–1 MLP, by contrast, produces a

Table 2.2: Predictive performance in simulation

			OLS	LASSO	MLP	MLP w. ℓ_1
Baseline	RMSE	Gaussian error	0.00940	0.00940	0.01211	-
		Non-Gaussian error	0.04152	0.04158	0.04192	-
		Non-linear	0.02250	0.02250	0.01251	-
	MAE	Gaussian error	0.00727	0.00727	0.00825	-
		Non-Gaussian error	0.02597	0.02599	0.02702	-
		Non-linear	0.01830	0.01830	0.00953	-
Noise	RMSE	Gaussian error	0.01133	0.01056	0.01781	0.01301
		Non-Gaussian error	0.05132	0.04401	0.05598	0.04993
		Non-linear	0.03237	0.02306	0.02409	0.01437
	MAE	Gaussian error	0.00879	0.00819	0.01360	0.00896
		Non-Gaussian error	0.03714	0.02808	0.04375	0.03543
		Non-linear	0.02732	0.01674	0.01737	0.01212

NOTE: Out-of-sample predictive performance of linear and non-linear models on the simulated data, evaluated across metrics

flat prediction for $x < 0.025$ after which its slope mirrors those of OLS and LASSO. In the nonlinear case, OLS and LASSO again coincide, while the MLP remains flat up to $x \approx 0.04$ before adopting an upward slope, better matching the extreme values, both seen in Figure 2.7 and Table 2.2. The MLP is able to forecast the non-linear process with almost half the RMSE and MAE of OLS and LASSO.

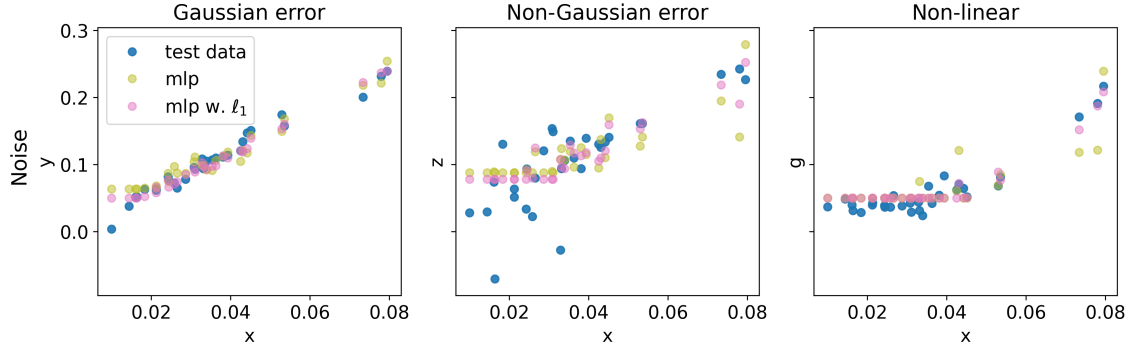
In our baseline experiments, no “noise” regressors are included, hence the near-identical performance of OLS and LASSO. In realistic large-scale factor models, however, one typically includes many candidate predictors, most of which are economically related and therefore correlated. To mimic this setting, we augment the true predictor x_i with 19 additional noise variables, indexed $s = 1, 2, \dots, 19$ defined by:

$$x_{i,s}^{\text{noise}} = \frac{1}{2}x_i + \nu_s, \quad \nu_s \sim \mathcal{N}(0, \sigma_x), \quad (2.15)$$

yielding a total of 20 covariates. Results are again shown in Figure 2.7 and error metrics in Table 2.2, this time labeled Noise.

In the linear Gaussian case, all three estimators again perform similarly. Under the linear Student’s t case, however, the LASSO correctly drives most noise-variable coefficients to zero, recovering a fit close to the baseline; the 20–2–1 MLP, by contrast, fails to fit properly, and the OLS also struggles to fit compared to LASSO. In the non-linear case, the MLP, as in the baseline, exhibits a “kink” at $x \approx 0.04$ but with diminished fit quality on the right-hand tail, while the LASSO again recovers a fit close to the linear baseline, thus, still not capturing the true data generating process, while fits OLS poorly.

Figure 2.8: Scatter plot of predictions MLP and MLP with ℓ_1 regularisation



NOTE: Data randomly drawn from (2.12)-(2.14). Noise specification adds an additional 19 noise variables randomly drawn from (2.15). The MLPs is trained on 63 datapoints and validated up against 7 datapoints from the training set. For the MLP with ℓ_1 penalty we set $\lambda_{\ell_1} = 3 \cdot 10^{-2}$

Just as LASSO's ℓ_1 penalty induces sparsity by shrinking irrelevant coefficients exactly to zero, one can impose an equivalent ℓ_1 regularization on the FFNN's weights. We therefore re-estimate the 20–2–1 MLP with an ℓ_1 penalty of $\lambda_{\ell_1} = 3 \cdot 10^{-2}$ on all network weights. Results of this are presented in Figure 2.8, and the last column of Table 2.2.

We find that this regularization helps the 20-2-1 MLP recover some of the performance in the linear Student's t case as well as in the non-linear case where much of the right-hand tail performance is recovered.

Our simulations confirm that OLS suffices when the true relationship is linear and uncontaminated by noise, matching the performance of more complex estimators. Once noise and correlated predictors enter a linear setting, the LASSO's sparsity-inducing penalty gives it an edge. In contrast, when the data-generating process is nonlinear or its functional form is unknown, a simple MLP captures the underlying pattern far better than any linear method. Finally, by imposing an ℓ_1 penalty on the network weights, the regularized MLP combines robustness to noise with a flexible nonlinear approximation, making it the estimator of choice in nonlinear, noise-prone applications.

3 Data

To conduct our empirical analysis, we construct a panel dataset of all firms listed on the NASDAQ Copenhagen Exchange from January 2002 through December 2024. Our principal data source is London Stock Exchange Group (2025), which provides daily trading series and annual accounting figures. Where London Stock Exchange Group (ibid.)'s coverage is incomplete, primarily for early-2000s observations, firms

that changed fiscal-year conventions, or missing line items such as depreciation and amortization, we manually retrieved the full annual reports from the Danish Business Authority (CVR) for a total of approximately 50 annual reports (Erhvervsstyrelsen (Danish Business Authority) 2025). In addition, we obtain macroeconomic interest-rate series from Statistics Denmark’s MPK3 database (Statistics Denmark 2025). The raw data can be found through <https://github.com/johanoelgaard/MastersThesis>.

In the following sections, we will go through each data source and how they are tied together to form the dataset underpinning our empirical analysis in Section 4. We generally attempt to get as close as possible to Gu, Kelly, and Xiu (2020), differing, however, as our focus is Denmark, and thus we look solely at companies listed on the NASDAQ Copenhagen Exchange, not companies on US exchanges.

3.1 Firm-level trading and accounting data

Sample definition

We begin with the universe of all common-share firms that have traded on the Copenhagen exchange since January 2001. As we include upwards of 12 months of lagged variables, 2001 is solely used for feature engineering. We include any firm with at least 13 months of trading on the exchange. Delistings are assumed to liquidate at the last observed daily price.

Daily price and volume series

From London Stock Exchange Group (2025), we, for each firm, download the daily adjusted closing price, $P_{i,t}^{\text{adj}}$, dividend payments, $D_{i,t}^{\text{adj}}$, trading volume, bid and ask quotes, and stock turnover, as well as the corresponding market-index series. Gaps in any price or quote series are filled by propagating the last valid observation to the next valid observation, implicitly assuming zero return over missing days.

Annual accounting variables

We extract the following from each firm’s official annual report: earnings before interest and taxes, gross profit, operating income, net income, R&D expenditure, cash and equivalents, current assets, total assets, current liabilities, long-term debt, total liabilities, shareholders’ equity, and cash flow. Because annual reports are typically released with a lag of 4–6 months, we impose a six-month publication delay: the financial figures for fiscal year end, t , are resampled to monthly frequency to cover months $t + 6$ through $t + 17$. Missing values in London Stock Exchange Group (ibid.) are supplemented by manual retrieval from the CVR registry when available. If not available, the observation is dropped from the dataset.

3.2 Macroeconomic variables

We enrich our firm-level panel with five monthly macroeconomic series, following the specification in Gu, Kelly, and Xiu (2020) and the variable definitions in Welch and Goyal (2008). Proxies for the long and short interest rates series are obtained from Statistics Denmark’s MPK3 database (Statistics Denmark 2025).

- **Short-term rate** (discount_t): Danmarks Nationalbank’s official discount rate.
- **Term spread** (tms_t): We construct the term spread as

$$\text{tms}_t = r_t^{\text{long}} - r_t^{\text{short}},$$

Where r_t^{long} is the yield on Danish 10-year government bond and r_t^{short} is the discount rate.

- **Earnings–price ratio** (ep_t): Sum of index constituents’ trailing-twelve-month earnings divided by the index level at month t .
- **Dividend–price ratio** (dp_t): Sum of cash dividends paid by index constituents over the preceding twelve months divided by the index level at month t .
- **Index variance** (svar_t): Variance of daily returns on the broad market index within month t .

All macro series are added to the stock-month observations as interaction terms with stock-month observations following Gu, Kelly, and Xiu (2020), the only exception to this is the case of boolean stock variables.⁷

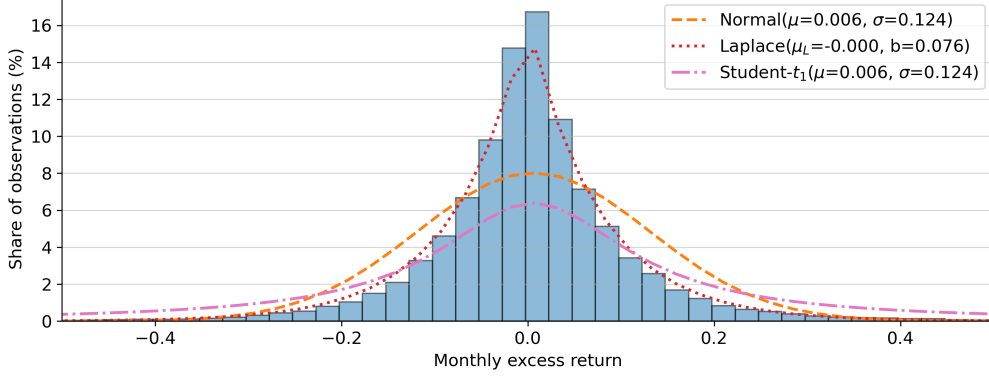
3.3 Feature engineering and preprocessing

Predictor construction

Following Gu, Kelly, and Xiu (ibid.) and definitions in Green, Hand, and Zhang (2017), we compute 55 firm-level characteristics. These include valuation ratios, profitability metrics, and trading-activity variables. Table B.1 lists each predictor’s name, definition, and descriptive statistics. The 53 non-boolean variables we compute are interacted with our five macroeconomic variables, resulting in a total of 372 features, including industry columns ($53 \cdot (5 + 1) + 2 + 52$ NACE industries).

⁷divi, and divo

Figure 3.1: Distribution of monthly excess returns



NOTE: Monthly excess return calculated following equation (3.1) for the entire dataset. The plot is constructed using 100 bins summing to 100%.

Target variable

Our dependent variable is the monthly excess return, including dividends:

$$r_{i,t+1} = \frac{P_{i,t+1}^{\text{adj}} + D_{i,t+1}^{\text{adj}}}{P_{i,t}^{\text{adj}}} - 1 - r_{t+1}^{\text{risk-free}}, \quad (3.1)$$

calculated from the last trading day of month t to that of month $t + 1$, where we for $r_{t+1}^{\text{risk-free}}$ use the effective monthly short rate. Figure 3.1 shows its cross-sectional density, which exhibits slight leptokurtosis (mean 0.006, standard deviation $\sigma = 0.164$).

Lagging and sample truncation

Because the 12-month momentum predictor requires returns from $t - 12$ to $t - 1$, we drop the first 12 observations for each firm. Further, as the return is predicted for the month ahead, we remove the last observation for each firm. Consequently, our final sample removes 13 months of data and includes only firm-months for which all predictors and the target are well-defined.

Data-split protocol

Consistent with the expanding-window scheme in Gu, Kelly, and Xiu (2020), we retrain our models on an annual basis, always withholding the most recent year for validation and evaluating OOS performance on the subsequent year. We generalize dataset notation as $\mathcal{D}_{\text{train},Y}$, $\mathcal{D}_{\text{val},Y}$, and $\mathcal{D}_{\text{OOS},Y}$, where Y is the window iteration. Concretely, our a neural network for the first expanding window is fitted on all firm-month observations from January 2002 through December 2019 denoted $\mathcal{D}_{\text{train},1}$, tuned on the 2020 validation data denoted $\mathcal{D}_{\text{val},1}$, and then assessed on 2021 data

denoted $\mathcal{D}_{OOS,1}$. Thereafter, we roll the window forward by one year, retraining on 2002–2020, validating on 2021, and testing on 2022, and so on until the end of the sample. For the OLS and LASSO benchmarks, which do not require a separate validation set for hyperparameter tuning, we instead estimate on the combined $\mathcal{D}_{train,Y} + \mathcal{D}_{val,Y}$, i.e., 2002–2020 for the first iteration and evaluate solely on the test year, i.e., 2021.

Standardization and outliers

Following Green, Hand, and Zhang (2017), we first winsorize all characteristics at the 1st and 99th percentiles of their cross-sectional distributions. Specifically, for each predictor $x_{p,i}$, observations below the 1st percentile or above the 99th percentile, calculated on the training dataset for each expanding window, are truncated to the corresponding percentile value. This approach retains as much characteristic information as possible while limiting the influence of extreme outliers.

Subsequently, we standardize each winsorized predictor. Concretely, we fit a single scaler on the entire training sample, such that for predictor p we compute

$$\tilde{x}_{p,i} = \frac{x_{p,i} - \bar{x}_{p,train,Y}}{\sigma_{p,train,Y}},$$

where

$$\bar{x}_{p,train,Y} = \frac{1}{N_{train,Y}} \sum_{i \in \mathcal{D}_{train,Y}} x_{p,i}, \quad \sigma_{p,train,Y} = \sqrt{\frac{1}{N_{train,Y}} \sum_{i \in \mathcal{D}_{train,Y}} (x_{p,i} - \bar{x}_{p,train,Y})^2},$$

and $\mathcal{D}_{train,Y}$ is the set of all firm-month observations in the training data for each expanding window Y . For the neural network $\tilde{x}_{p,train,Y}$ and $\sigma_{p,train,Y}$ are computed solely on the training data to avoid any forward-looking information leakage from the validation data and out-of-sample test data. In the case of the LASSO and OLS models, there is no validation data, and hence, solely the out-of-sample test data is excluded when computing $\bar{x}_p$ and σ_p . This standardization ensures that all predictors enter the neural network on a common scale, facilitating training convergence and mitigating the influence of extreme values.

4 Empirical study on Danish equities

4.1 Model specification

In this section, we will cover the exact setup of the models run in Section 4. We generally follow the theory of Gu, Kelly, and Xiu (2020) with some deviations where relevant. In this paper, the overarching goal is to develop a model to predict asset

risk premia. Let us first define the overarching problem we are trying to solve. We assume an asset’s excess return takes the form of an additive error prediction model:

$$r_{i,t+1} = \mathbb{E}_t(r_{i,t+1}) + \varepsilon_{i,t+1},$$

where

$$\mathbb{E}_t(r_{i,t+1}) = g^*(\mathbf{x}_{i,t}).$$

Here stocks are index by $i = 1, \dots, I$ and months by $t = 1, \dots, T$. Across all models, our goal is to find the best function $g^*(\cdot)$ based on a set of predictors represented as a P -dimensional vector $\mathbf{x}_{i,t}$ and $r_{i,t+1}$ is as defined in equation (3.1).

It is important to recognize that this formulation inherently imposes certain restrictions. Specifically, by assuming that $g^*(\cdot)$ is independent of both i and t , our model leverages pooled information across stocks and across time, thus becoming a generalized predictive model for asset return. This assumption implies that the functional form and relationship between predictors and expected excess returns remain constant both across different stocks and over the entire period. Although this assumption is restrictive, it facilitates broad applicability and practical feasibility in estimation.

OLS

We run our OLS model by calculating the closed form solution described in Appendix A.7 using the expanding window scheme described as Section 3.3 and, unlike Gu, Kelly, and Xiu (2020), we include all available predictors in the baseline OLS, thus estimating β in:

$$r_{i,t+1} = \beta \mathbf{x}_{i,t} + \varepsilon_{i,t+1},$$

following the closed form solution given by equation (2.10).

LASSO

To introduce variable selection and shrinkage, we apply the LASSO through the same expanding window as the OLS. We employ a 5-fold cross-validation LASSO on all the data using the `LassoCV` implementation by `scikit-learn`,⁸ evaluating potential penalties over a geometric gridspace in $[1 \cdot 10^{-7}, 10^2]$.

Neural networks

Our baseline predictive model is a fully connected, feed-forward neural network implemented in PyTorch (Paszke et al. 2019), with ReLU activation at each hidden

⁸Pedregosa et al. 2011.

layer (Gu, Kelly, and Xiu 2020). We initialize all weights using the Kaiming initialization (He et al. 2015) and optimize with the ADAM algorithm (Kingma and Ba 2017). Training proceeds in mini-batches of size 4096 for up to 250 epochs, employing early stopping with a patience of 25 epochs.

For our baseline model, we use the MSE augmented with an elastic-net penalty, to discourage overly large weights (see Section 2.1.4). Concretely, for batch B_τ , the loss is

$$\mathcal{L}_{batch} = \frac{1}{|B_\tau|} \sum_{i \in B_\tau} (r_{i,t+1} - \hat{r}_{i,t+1})^2 + \lambda_{\ell_1} \|\theta\|_1 + \lambda_{\ell_2} \|\theta\|_2^2,$$

where $r_{i,t+1}$ and $\hat{r}_{i,t+1}$ are the true and predicted targets, respectively. Although MSE remains the standard choice for regression tasks (Masters 1993b), we later explore alternative loss functions.

Further, we employ dropout on both input and hidden layers, and form an ensemble of five independently seeded MLPs to mitigate the risk of poor local minima. Pseudocode for model construction and training appears in Algorithms 1 and 2, respectively.

4.2 Hyperparameter tuning

Machine learning’s primary defense against overfitting is tuning of its hyperparameters. However, the literature on hyperparameterization of neural networks within finance, and in general, is sparse. There are many hyperparameters to tune, making tuning a tedious task; we will in this section go through the process of selecting the size of the network and regularization parameters, as well as shed light on the impact of choosing a specific activation and loss functions. All hyperparameter tuning are done by training the model on the $\mathcal{D}_{train,1}$ and validated on $\mathcal{D}_{val,1}$ to avoid any information leakage.

Depth and width

We start by exploring how the depth and width of a neural network affect its performance. It is essential to strike a balance: choosing too few neurons or creating a network that is too shallow may lead to underfitting, while selecting too many neurons or making the network too deep can increase training time and lead to overfitting, both of which impair out-of-sample performance. The engineering goal is therefore to allocate *just enough* neurons to capture the underlying structure without encouraging the model to capture noise. There are no clear guidelines in the literature. Orimoloye et al. (2020) finds that both deep and shallow network architectures have their merits, depending on the setting.

Gu, Kelly, and Xiu (2020) evaluate multiple neural networks of depth ranging

Algorithm 1 Build MLP model

Require:

input_dimension: integer,
depth: integer,
width: interger or array,
dropout: float,
output_dimension: integer,
activation_function: function

```
if isInteger(width) then
    widths  $\leftarrow$  [width]  $\times$  depth
else
    widths  $\leftarrow$  width
end if
layers  $\leftarrow$  []
input_features  $\leftarrow$  input_dimension
for  $w$  in widths do
    append nn.Dropout(dropout) to layers
    append nn.Linear(input_features,  $w$ ) to layers
    append activation_function to layers
    input_features  $\leftarrow$   $w$ 
end for
append nn.Linear(input_features, output_dimension) to layers
return nn.Sequential(layers)
```

NOTE: Psudocode for building the structure of our MLP FFNN using the PyTorch library (Paszke et al. 2019). For the full PyTorch implementation, see: <https://github.com/johanoelgaard/MastersThesis>

Algorithm 2 Train MLP with ℓ_1/ℓ_2 and early stopping

Require:

$\mathcal{D}_{train,Y}$: torch.tensor,
 $\mathcal{D}_{val,Y}$: torch.tensor,
model: nn.Module, ▷ from Algorithm 1
criterion: base loss function,
 λ_{ℓ_1} : float,
 λ_{ℓ_2} : float,
optimizer: torch.optim.Optimizer.Adam,
epochs: integer,
patience: integer,
batch_size: integer

```
train_batch ← torch.utils.data.DataLoader( $\mathcal{D}_{train,Y}$ , \
    batch_size = batch_size, shuffle = True)
val_batch ← torch.utils.data.DataLoader( $\mathcal{D}_{val,Y}$ , \
    batch_size = batch_size, shuffle = False)
best_loss ←  $+\infty$ ; counter ← 0
for  $e$  to epochs do
    model.train();  $\mathcal{L}_{train,Y} \leftarrow 0$  ▷ Training phase
    for each batch in train_batch do
         $\hat{y} \leftarrow \text{model}(\mathbf{X})$ 
         $\mathcal{L}_{batch} \leftarrow \text{criterion}(\hat{y}, \mathbf{y}) + \lambda_{\ell_1} \|\theta\|_1 + \lambda_{\ell_2} \|\theta\|_2^2$ 
         $\mathcal{L}_{batch}.\text{backward}$  ▷ Calculating gradient with torch.autograd
        optimizer.step() ▷ Updating  $\theta$  with optimizer
         $\mathcal{L}_{train,Y} += \mathcal{L}_{batch}$ 
    end for
    model.eval(),  $\mathcal{L}_{val,Y} \leftarrow 0$  ▷ Validation phase
    for each batch in val_batch do
         $\hat{y} \leftarrow \text{model}(\mathbf{X})$ 
         $\mathcal{L}_{val,Y} += \text{criterion}(\hat{y}, \mathbf{y})$ 
    end for
    if  $\mathcal{L}_{val,Y} < \text{best\_loss}$  then ▷ Early stopping logic
        best_loss ←  $\mathcal{L}_{val,Y}$ 
        best_model ← model; counter ← 0
    else
        counter += 1
        if counter  $\geq$  patience then
            break
        end if
    end if
end for
return best_model
```

NOTE: Psudocode for training one MLP model using the PyTorch library (Paszke et al. 2019). For the full PyTorch implementation, see: <https://github.com/johanoelgaard/MastersThesis>

from 1 through 5, employing a geometric *pyramid* rule for their layer width, with the shallowest network having [32] neurons in its hidden single layer; the second having two hidden layers with [32, 16] neurons, and so on with the deepest network having [32, 16, 8, 4, 2] neurons. This technique was first proposed by Masters (1993a) as a way to encourage the model to progressively distil information as the network deepens and reduce the number of neurons in the network. However, this is only a rule of thumb, and alternative layouts are possible; e.g. Bianchi, Büchner, and Tamoni (2020) retain a constant width across layers when forecasting bond returns.

Using the data from the first expanding window, we grid-search over `width` $\in$ 8,16,32,64,128 and `depth` $\in$ 1, ..., 7, and we include the geometric pyramids of Gu, Kelly, and Xiu (2020). For every configuration, we train an ensemble of five models and report the mean validation loss in Figure 4.1.

For constant-width networks, performance improves up to about four hidden layers; beyond that, additional depth yields no benefit and even harms performance at depth seven. Likewise, increasing width helps until `width` = 64, after which validation loss rises.

From Figure 4.2 for `width` = 128, we see that models start to overfit after only a few epochs. Furthermore, narrow networks seem to have trouble converging properly. We therefore restrict our subsequent constant-width experiments to `width` = 64 and `depth` = 2.

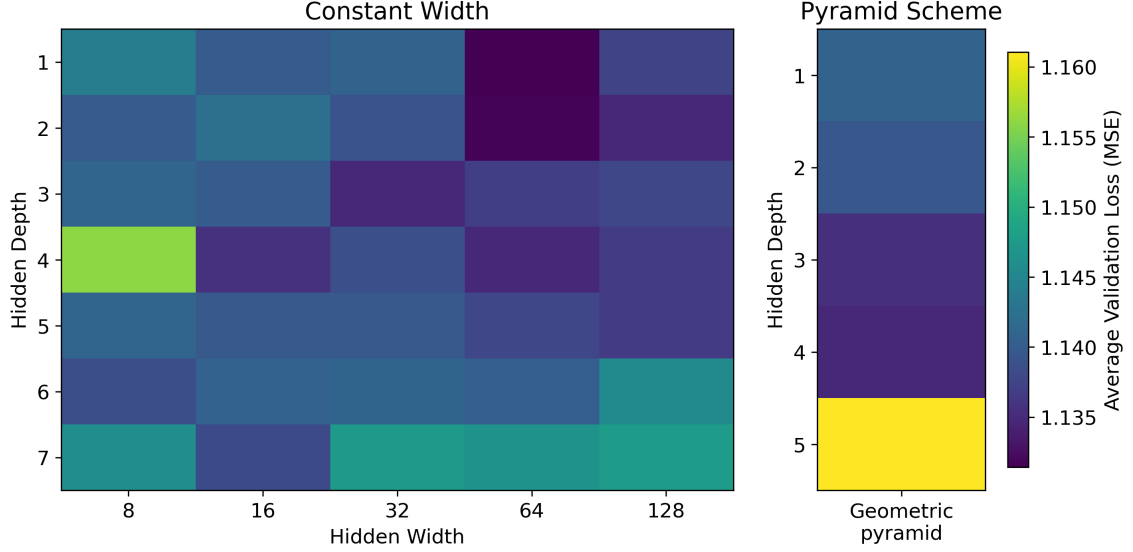
The pyramid architectures display a similar pattern: validation loss decreases with the third and, marginally, the fourth layer, but deteriorates when a fifth layer is added. Closer inspection of individual training runs (Appendix Figure B.1) shows that a subset of models fail to converge, suggesting instability at depth five. Sticking with the principle of choosing the smallest model that provides *adequate* performance, we adopt the pyramid of depth three and width [32,16,8].

Generally, we find that shallower networks perform the best on the data we are working with. There seems to be a trade-off between width and depth; the shallower the network, the more neurons are required in each layer. E.g., we find that the best performing network with depth 4 has 16 neurons, the one with depth 3 has 32 neurons, and the one with depth 2 has 64 neurons. All deep networks struggle, thus indicating a low signal-to-noise ratio similar to what Gu, Kelly, and Xiu (ibid.) finds.

Regularization

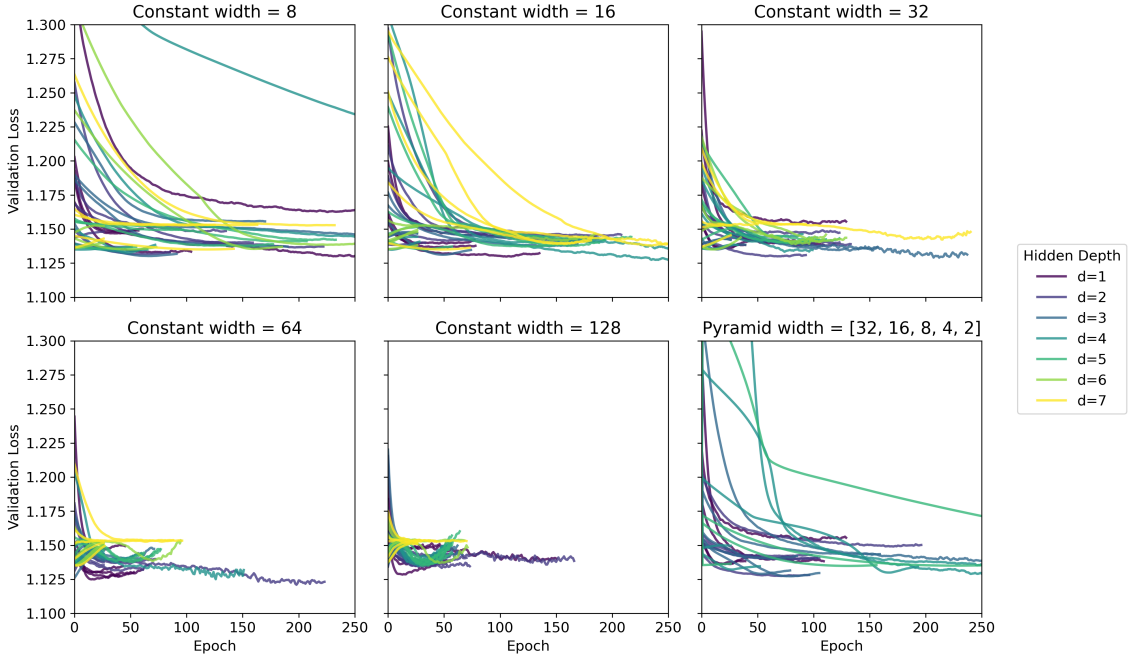
Next, we examine the effect of varying regularization strength, as another way for neural networks to avoid overfitting the model to the training data. We will use the model structures found in the previous section and sweep over the 3-dimensional hyperparameterspace of an elastic net with $\lambda_{\ell_1} \in \{0, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$ and

Figure 4.1: Average validation loss for constant-width networks and geometric pyramid networks



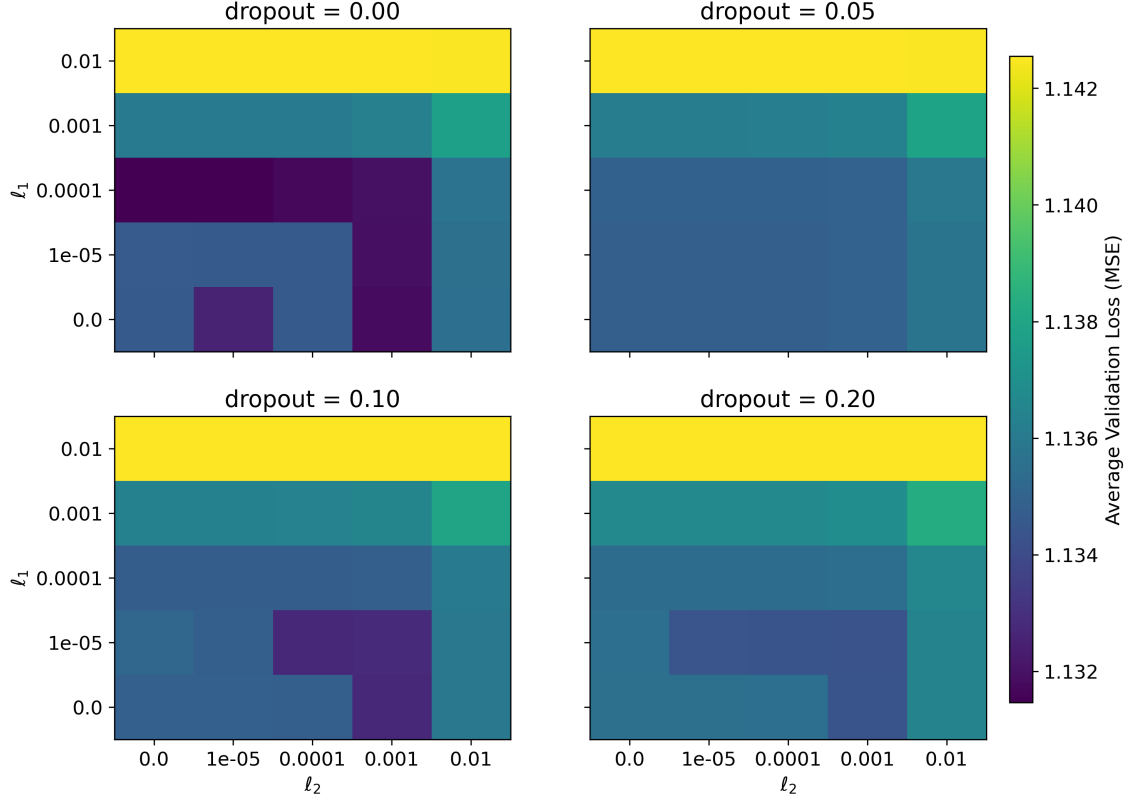
NOTE: Simple average validation loss evaluated across an ensemble of five models trained on data up until 2020 and validated on 2020 data (first year in our expanding window), with $\lambda_{\ell_1} = 10^{-4}$, $\lambda_{\ell_2} = 10^{-4}$, dropout of 0.0, and learning rate of $5 \cdot 10^{-5}$. The geometric pyramid scheme follows Gu, Kelly, and Xiu (2020), with widths $[32]$, $[32, 16]$, $[32, 16, 8]$, $[32, 16, 8, 4]$, and $[32, 16, 8, 4, 2]$. For individual runs, see Figure B.1.

Figure 4.2: Validation loss over time for each run over varying widths



NOTE: Validation loss for each of the five models trained on data up until 2020 and validated on 2020 data (first year in our expanding window) with $\lambda_{\ell_1} = 10^{-4}$, $\lambda_{\ell_2} = 10^{-4}$, dropout of 0.0, and learning rate of $5 \cdot 10^{-5}$. The geometric pyramid scheme follows Gu, Kelly, and Xiu (2020), with $\text{width} \in \{[32], [32, 16], [32, 16, 8], [32, 16, 8, 4], [32, 16, 8, 4, 2]\}$.

Figure 4.3: Average validation loss in a constant-width network with varying regularization



NOTE: Simple average validation loss evaluated across an ensemble of five models trained on data up until 2020 and validated on 2020 data (first year in our expanding window), with depth 2, constant width of 64, and learning rate of $5 \cdot 10^{-5}$.

$\lambda_{\ell_2} \in \{0, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$, complemented with **dropout** $\in \{0, 0.05, 0.1, 0.2\}$.

Figure 4.3 reports the resulting heatmaps for the cross-section of the three parameters for a fixed constant-width network of 64×2 , and Figure B.2 shows it for the pyramid structure with **depth** = 3 and **width** = [32,16,8].

Figure 4.3 and Appendix Figure B.2 show that dropout does *generally* not provide large improvements in either model. We find that in our network of constant width, shown in 4.3, dropout leads to a decline in performance, while the pyramidal structure, shown in B.2, sees little to no improvements. Thus, we will not be using dropout in the final models.

In terms of norm penalties, we find that an ℓ_1 regularization of $\lambda_{\ell_1} = 10^{-4}$ and an ℓ_2 regularization of $\lambda_{\ell_2} = 10^{-3}$ provide an improvement of the validation loss in the network of constant width while the pyramid structure seem more robust across dropout rates with minimal difference for $\lambda_{\ell_1} \in [10^{-5}, 10^{-4}]$ and $\lambda_{\ell_2} \in [10^{-4}, 10^{-3}]$. For simplicity, we therefore move forward with identical regularization parameters

for both models with `dropout` = 0.0, $\lambda_{\ell_1} = 10^{-4}$, and $\lambda_{\ell_2} = 10^{-3}$.

Activations

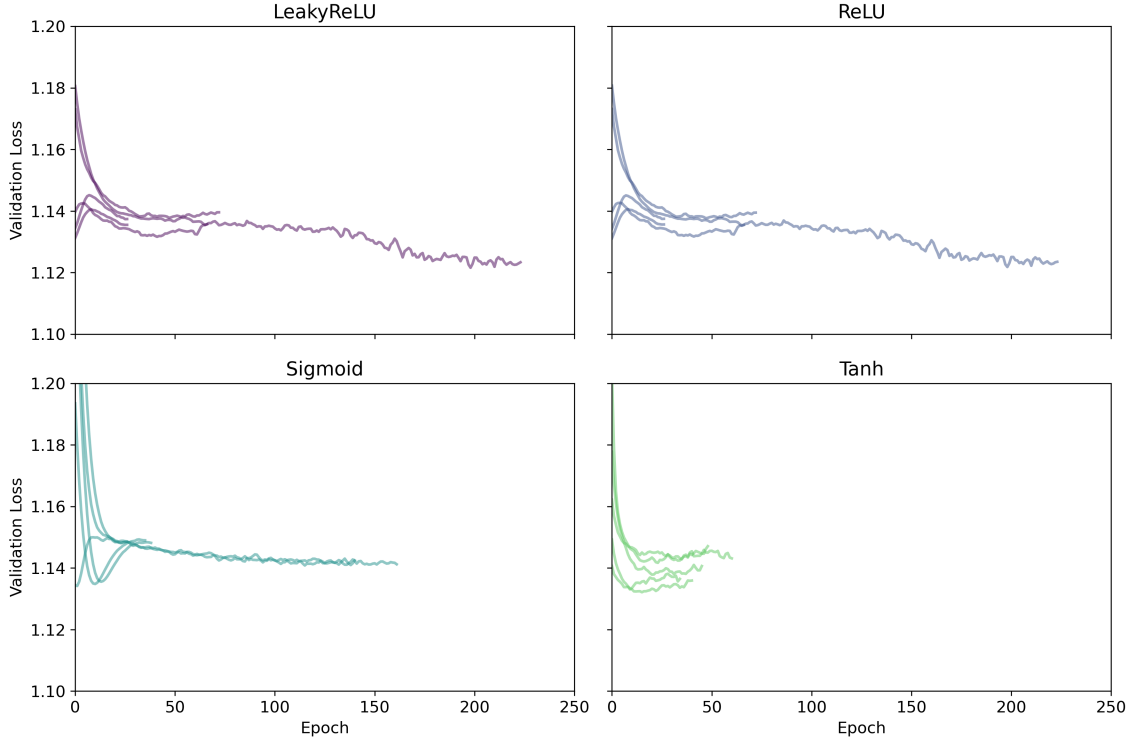
Our baseline models use ReLU activations, as is suggested for DL by Peña and Tsay (2021) and Goodfellow, Bengio, and Courville (2016). ReLU is also used by Gu, Kelly, and Xiu (2020). Nevertheless, alternatives, even though sparingly used in the modern literature, are commonly presented in textbooks. Both Peña and Tsay (2021) and Goodfellow, Bengio, and Courville (2016), among others, introduce sigmoid and hyperbolic tangent activations. Here, we will treat activation choice as a type of hyperparameter and test if the choice of activation function in the currently specified network structure can lead to validation loss improvements.

Figure 4.4 shows validation loss in training for our constant-width network using sigmoid, tanh, ReLU, and Leaky ReLU activations. The convergence paths of Leaky ReLU and ReLU are very hard to tell apart, and sigmoid has very similar convergence paths. Tanh tends to have a hard time converging. The five models across all activation functions use the same five initializations. We see essentially all models, except tanh, struggle with the same initializations, although it looks like the rectifiers take longer to converge.

Beyond the convergence properties of the models, we are, of course, interested in the predictive performance of the models using the different activation functions. Table 4.1 reports RMSE, MAE, and AMADL for the ensemble models trained with the different activation functions. Generally, all the models achieve very similar results. While sigmoid seemed to have great convergence, it lags behind in terms of predictive performance, being the worst for both MSE and MAE, and second-to-last on AMADL. The rectifiers are the same. They were expected to be similar, although it is somewhat surprising that they are the same. Tanh performs slightly worse than the rectifiers on MAE, but lags behind on RMSE and AMADL. From a predictive performance perspective, the rectifiers seem to be the most attractive alternative. Although the rectifiers appear to have somewhat slower convergence, they compensate for this in computational efficiency.

Given this, we see no reason to deviate from ReLU as the chosen activation function for the networks. With this said, the networks seem quite robust to the choice of activations, and no substantial benefits seem to arise for the individual functions. Keep in mind that we did the hyperparameter tuning on a model using ReLU, and it is not unlikely that there are interdependencies. Thus, tuning the network fully for each activation function might yield different results. Residual histograms for each ensemble can be found in Appendix B, Figure B.3.

Figure 4.4: Validation loss for a constant-width network with varying activation functions



NOTE: Validation loss for five models trained on data up until 2020 and validated on 2020 data (first year in our expanding window). Models have different activation functions but all have depth 2, constant width of 64, learning rate of $5 \cdot 10^{-5}$ and regularization $\lambda_{\ell_1} = 10^{-4}$, $\lambda_{\ell_2} = 10^{-3}$ and $\text{dropout} = 0.0$.

Loss functions

Similar to the previous section, the choice of loss functions is an interesting topic. We cannot quite view it as a hyperparameter-like choice as we did for activations, as the loss function is not part of optimizing the network; it is the sheer objective of the network. As highlighted by Masters (1993b), by a large margin, the most common loss function for FFNNs is MSE. For general forecasting purposes, MSE has great intuition behind it, as it does encourage the network to forecast more accurately, meaning the predicted values should be as close as possible to the true values. With this said, it does so by minimizing the squared deviations. It is less clear that this is necessarily the ideal objective in the case of heavy-tailed error distributions. As also noted by Gu, Kelly, and Xiu (2020), it is quite well shown in the literature that returns have significantly heavier tails than a normal distribution. MSE might be overly concerned by these large errors. Whether or not this is desirable is not trivial.

Here, we will test our constant-width specification trained using MSE, MAE, and Huber to see how this affects the validation loss performance evaluation. In Table 4.2,

Table 4.1: Predictive performance for sigmoid, tanh, ReLU, and Leaky ReLU models

	ReLU	LeakyReLU	Tanh	Sigmoid
RMSE	0.13388	0.13388	0.13434	0.13433
MAE	0.08596	0.08596	0.08605	0.08654
AMADL	0.00814	0.00814	0.00836	0.00828

NOTE: Model performance evaluated across metrics on the validation set. The MLPs are the average of five networks initiated from different seeds. For AMADL, we use $\delta = 0.5$. Models have different activation functions but all have depth 2, constant width of 64, learning rate of $5 \cdot 10^{-5}$ and regularization $\lambda_{\ell_1} = 10^{-4}$, $\lambda_{\ell_2} = 10^{-3}$ and **dropout** = 0.0.

we see RMSE, MAE, and AMADL on the validation set for each model trained using the different loss functions. We have not optimized the Huber hyperparameter, but residual histograms can be found in Appendix B, Figure B.5, where we set $\psi = 0.5$ to catch extreme values. Furthermore, we do not dive into the convergence properties in this section, as the validation loss for each model ensemble is not comparable. The validation loss charts can be found in Appendix B, Figure B.4.

Interestingly, the validation RMSE for the MSE ensemble is the lowest of the bunch, and the validation MAE is lowest for the MAE ensemble. This might seem trivial, as we trained with the same objective as we evaluate, but since we train and evaluate on different data sets, this is not a given. Furthermore, the Huber ensemble sits right in between on both the RMSE and MAE metrics. Given the structure of the Huber loss function, this makes sense, as it will treat smaller errors as MSE does, and larger errors like MAE. The MSE ensemble also leads in AMADL, followed by Huber and then MAE. Since the quadratic loss is part of the AMADL loss metric, the RMSE numbers would indicate this to be somewhat likely, depending on the ability to forecast the sign correctly.

Like for the model ensembles of varying activation functions, the models all perform very similarly. To put this into perspective, the MAE ensemble has an average absolute forecast error expressed in excess returns, which is $\approx 0.007\%$ -points better than the MSE ensemble.

Hence, we find that for a network of constant width the optimal size to be **width** = 64 and **depth** = 2 while we for a pyramid scheme find the optimum to be **depth** = 3 with **width** = [32,16,8]. Furthermore, we find no need for dropout, but include norm penalization of $\lambda_{\ell_1} = 10^{-4}$, and $\lambda_{\ell_2} = 10^{-3}$. Lastly, the networks seem quite robust to the choice of both activation and loss functions. Therefore, our two final models are a constant-width 372-64-64-1 MLP, (from here on called MLP) and a pyramidal 372-32-16-8-1 MLP (from here on called MLP-Pyr) using ReLU activations trained on an MSE loss function.

Table 4.2: Predictive performance for MSE, MAE, and Huber models

	MSE	MAE	Huber
RMSE	0.13388	0.13543	0.13514
MAE	0.08596	0.08589	0.08591
AMADL	0.00814	0.00872	0.00867

NOTE: Model performance evaluated across metrics on the validation set. The MLPs are the average of five networks initiated from different seeds. Huber is trained with $\psi = 0.5$. For AMADL, we use $\delta = 0.5$. Models have different objective function but all have depth 2, constant width of 64, learning rate of $5 \cdot 10^{-5}$ and regularization $\lambda_{\ell_1} = 10^{-4}$, $\lambda_{\ell_2} = 10^{-3}$ and `dropout` = 0.0.

4.3 Out-of-sample forecasting

We now move on from tuning the networks to instead use the specified models to forecast out-of-sample excess returns using dataset $\mathcal{D}_{OOS,Y}$. To compare the OOS performance of forecasts, we use the metrics described in Section 2.4. Furthermore, as is good practice, we also include a naïve forecast. Here, we define the naïve forecast for a given expanding window to be:⁹

$$\hat{r}_{i,t+1}^{\text{naïve}} = \mathbb{E}(r_{i,t+1}) = \frac{1}{N_{\text{train},Y} + N_{\text{val},Y}} \sum_{i,t \in \mathcal{D}_{\text{train},Y} + \mathcal{D}_{\text{val},Y}} r_{i,t+1}$$

We do this as ranking the forecasts among themselves does not tell us if the forecasts are useful. To know if we can actually gain from using the forecasts, we benchmark against naïvely guessing that the excess return tomorrow is the average of past excess returns. Table 4.3 reports the respective RMSE, MAE, and AMADL ($\delta = 0.5$) for both the naïve forecast and the forecasts of all models for individual expanding windows, as per section 3.3, as well as the aggregate performance across the 4 years. The MLP and MLP-Pyr predictions are based on a five-model ensemble.

Generally, we find that the linear OLS and LASSO forecasts are the poorest performing. OLS achieves the highest loss scores for all periods and for all loss metrics in total, with exceptions of AMADL in year 2022 and 2023. It does not come as a big surprise that the penalized but linear LASSO regression is an improvement over the classical OLS. We have quite a lot of parameters, and expectedly a low signal-to-noise ratio. Furthermore, as some degree of multicollinearity is to be expected, small changes in parameter values can yield significant changes in estimations. As the 5-fold CV provides a somewhat optimized regularization for LASSO, it will have a

⁹Gu, Kelly, and Xiu (2020) argues that stock returns are so close to a random walk, that the appropriate benchmark is $\mathbb{E}(r_{i,t+1}) = 0$ for their R^2 statistics. We do not use this benchmark, because RMSE on a naïve forecast of $\mathbb{E}(r_{i,t+1}) = 0$ on our data is higher than the RMSE of a mean naïve forecast.

Table 4.3: Out of sample predictive performance of OLS, LASSO, and MLP

		Naïve	OLS	LASSO	MLP	MLP-Pyr
RMSE	2021	0.10729	0.11147	0.10598	0.10619	0.10609
	2022	0.12309	0.12599	0.12419	0.12432	0.12443
	2023	0.10608	0.11413	0.10856	0.10608	0.10552
	2024	0.10975	0.11445	0.11169	0.10949	0.10948
	Total	0.11177	0.11664	0.11281	0.11179	0.11166
MAE	2021	0.06921	0.07593	0.06953	0.06903	0.06882
	2022	0.08553	0.08749	0.08727	0.08722	0.08747
	2023	0.06752	0.07352	0.06862	0.06718	0.06636
	2024	0.06864	0.07467	0.07103	0.06856	0.06843
	Total	0.07277	0.07794	0.07414	0.07304	0.07282
AMADL	2021	-0.00444	0.00084	-0.00484	-0.00481	-0.00452
	2022	0.01058	0.01002	0.01000	0.01100	0.00980
	2023	0.00543	0.00605	0.00521	0.00606	0.00269
	2024	0.00389	0.00620	0.00617	0.00372	0.00424
	Total	0.00380	0.00574	0.00405	0.00393	0.00299

NOTE: Model performance evaluated across metrics. The MLP model is the average of five networks initiated from different seeds. For AMADL, we use $\delta = 0.5$. The year reflects the performance.

tendency to select less correlated variables, which might be part of the explanation. Furthermore, the shrinkage introduced by the regularization will aid in raising the signal-to-noise ratio. Nevertheless, LASSO still comes in second to last of all the forecasts for the totals of each metric. This could indicate that there might be some non-linearities in the data generating process, whether for each input feature or non-linear interactions among the features.

The most interesting insight from Table 4.3 is the fact that the naïve forecast performs on par with both of our MLPs. The naïve forecast achieves the lowest total MAE, and a slightly lower RMSE than MLP, with the MLP-Pyr achieving the lowest RMSE of all models. The naïve forecast also beats MLP on AMADL. This is interesting. With similar RMSE scores, this implies that the sign predictions of MLP are worse than always guessing that excess returns are positive. The MLP-Pyr does outperform the naïve forecast on AMADL. Our overall takeaway is that, based on these metrics, none of the tested methods are great at forecasting excess returns.

Overall, all models have low predictive power, as they barely beat the naïve forecast. Nevertheless, MLP-Pyr appears to be the overall winner, with the linear models emerging as the losers. This indicates that using FFNN does have an edge over its linear counterparts, albeit a limited one.

4.3.1 Which predictors matter?

Although we do not find our models to be effective at predicting point estimates compared to a naïve approach, we still observe improvement in the neural networks compared to the linear models. To investigate the relative importance of individual covariates for the performance of each model, we follow Gu, Kelly, and Xiu (2020). We calculate the delta in MSE^{10} by zeroing a predictor in each ensemble model for each period in our expanding window and then averaging the effect across models and periods.

Figure 4.5 reports the overall ranking of each firm-level characteristic across models. For each method, the variable-importance weights are normalized to sum to unity, so that darker shades correspond to greater explanatory power in that specific model. However, cross-sectional comparisons of colour intensity should be interpreted with caution. We rank the importance of each characteristic and order them such that the average highest ranking is on top and the average lowest ranking is at the bottom.

As seen in Figure 4.5, we find that while the models that allow for sparsity largely agree on the top 15 predictive variables, they differ somewhat in their specific rankings, while OLS disagrees with all but a select few.

The top predictor is the size of the firm, represented by the logarithmic market capitalization (`mvel1`). Beyond that, we can broadly classify the top variables into four categories. The first being liquidity variables with the DKK-denominated trading volume (`dkkvol`), its standard deviation (`std.dkkvol`), the monthly average of daily bid-ask spread (`baspread`), and the Hou and Moskowitz (2005) price delay (`pricedelay`).

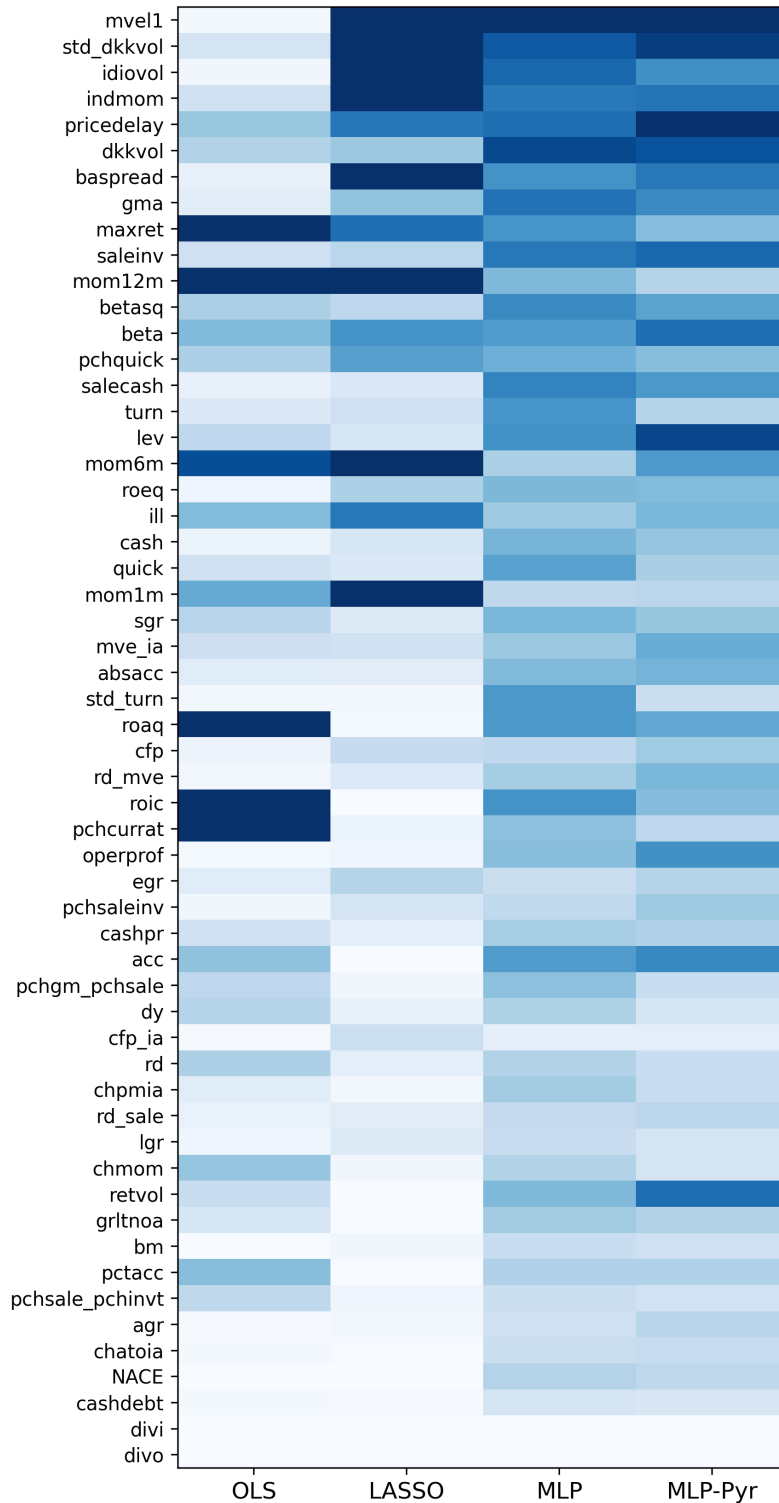
The second category comprises risk measures, notably the idiosyncratic volatility (`idiovol`), market beta, and its squared value (`beta` and `betasq`).

The third group reflects momentum characteristics, including industry momentum (`indmom`), recent maximum single-day return (`maxret`), and twelve-month momentum (`mom12m`), making it to the top 15 across the board. 1- and 6-month momentum (`mom1m` and `mom6m` respectively) do not play as significant a role in the non-linear models, while OLS and LASSO find them relevant.

Finally, some fundamental indicators also appear among the top predictors with gross profitability (`gma`) and sales-to-inventory (`saleinv`) both being in the top 15. This category, however, is where the model-specific differences among the categories are most pronounced. For instance, the OLS model assigns high importance to return on invested capital and return on assets (`roic` and `roaq`). In contrast, these are

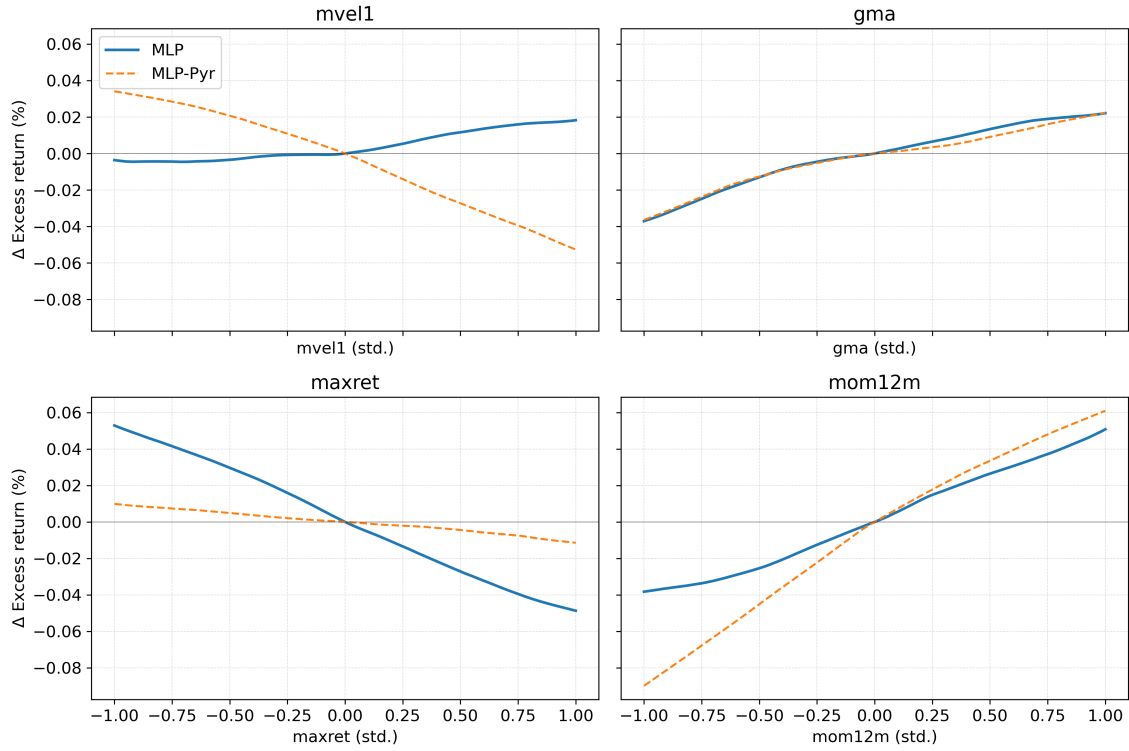
¹⁰Gu, Kelly, and Xiu (2020) uses the delta in R^2

Figure 4.5: Relative effect on MSE of zeroing variable across models



NOTE: Rankings of stock-level characteristics and the industry dummy (NACE) in terms of overall model contribution. Characteristics are ordered based on the average of their contribution across models. Colour gradients within each column indicate the most influential (dark blue) to the least influential (white) variables.

Figure 4.6: Marginal effect between expected return and select characteristics



NOTE: Expected change in monthly excess returns (vertical axis) to the individual characteristics (holding all other covariates fixed at their median values).

excluded entirely by the LASSO model, likely due to multicollinearity with other variables. This observation is consistent with the neural network models, which also assign relatively low importance to these signals, suggesting that other predictors may already subsume their informational content.

Marginal effects

As previously argued, the primary advantage of neural networks over linear models is their ability to capture complex, nonlinear interactions between predictors. However, this also leads to a certain opacity when interpreting models, as there is a high degree of interdependency. We attempt to shed some light on this in Figure 4.6. By illustrating the marginal effects in a stylized *all-else-equal* scenario where all predictors not considered are set to 0, representing their standardized mean values.

Figure 4.6 plots the marginal relationship between each firm characteristic, logarithm of market capitalization (**mvel1**), gross profitability (**gma**), maximum single-day return over the past month (**maxret**), and twelve-month momentum (**mom12m**), and the model's forecasted monthly excess return, holding all other covariates fixed at their median values. For a complete set of curves across additional covariates, see Appendix Figure B.6.

The MLP-Pyr specification exhibits the classic size effect: as `mvel1` decreases (i.e., as firms become smaller), expected excess returns rise, all else equal. In contrast, the constant-width MLP shows little to no negative slope over most of the size distribution and even hints at a slight positive tilt for larger market-capitalization stocks. Both architectures, however, agree on a positive, roughly monotonic association between gross profitability and forecasted excess return, indicating that higher annual gross profitability consistently corresponds to higher expected returns over the coming year.

Turning to return-based signals, both models produce upward-sloping curves for twelve-month momentum (`mom12m`), reinforcing the continuation effect; stocks that performed strongly over the past year tend to receive higher predicted excess returns in the subsequent month. Looking in the shorter term on just the maximum single-day return in the past month, both models' marginal effect curves for `maxret` slope downward, implying a short-term reversal mechanism. I.e., an extreme single-day gain within the past month is associated with lower expected excess returns over the next month, consistent with an overreaction-and-mean-reversion narrative.

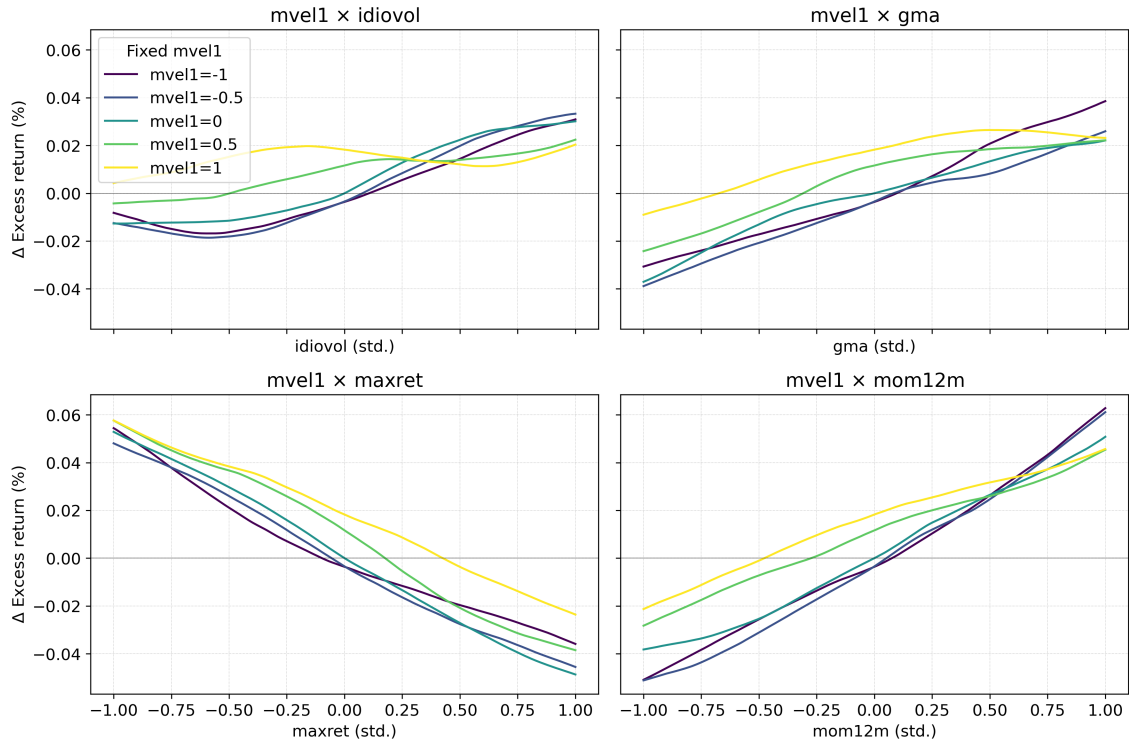
Hence, MLP-Pyr exhibits the classic size effect, while both the MLP-Pyr and constant-width MLP architectures capture (i) the positive influence of gross profitability, (ii) the continuation effect of long-run momentum, and (iii) a short-term reversal effect tied to extreme one-day moves.

Interaction effects

Figure 4.7 reports how the expected return varies while allowing for interactions between multiple variables, specifically for the MLP model of constant width. For the MLP-Pyr specification, see Appendix Figure B.7. In Figure 4.7 we allow for size (`mvel1`) to interact with the idiosyncratic volatility (`idiovol`), gross profitability (`gma`), maximum single-day return over the past month (`maxret`), and 12-month momentum (`mom12m`), with `mvel1` taking one of five fixed values, ($\text{mvel1} \in \{-1, -0.5, 0, 0.5, 1\}$). Each panel shows a family of five curves, with the darkest representing a firm with log market cap one standard deviation below the mean, and the lightest representing firms with log market cap one standard deviation above the mean.

In the first panel, examining the interaction between size and idiosyncratic volatility, we observe that idiosyncratic volatility has a minimal effect on larger-cap stocks, consistent with the theoretical CAPM view that idiosyncratic risk should be fully diversifiable. In contrast, smaller-cap stocks exhibit a positive correlation between excess return and idiosyncratic volatility. The second panel interacts size and gross profitability, showing that there is a positive relationship between excess

Figure 4.7: Marginal effect between expected return and select characteristic interactions (MLP)



NOTE: Expected change in monthly excess returns (vertical axis) to the individual interactions effects for $mvel1$ with $idiovol$, gma , $mom12m$ and $maxret$ in our MLP model (holding all other covariates fixed at their median values). For interactions in our MLP-Pyr model, see Figure B.7

return and gross profitability, as also seen in Figure 4.6. However, while the positive effects of high gross profitability are more pronounced among small-cap companies, the negative effects are also more pronounced. This relationship could be explained by large-cap companies being more resilient compared to smaller-cap companies.

In the bottom panels of Figure 4.7 we interact size with the momentum variables $maxret$ and $mom12m$. Here, we see the same general picture as in Figure 4.6, with size not drastically impacting $maxret$. For 12-month momentum, we observe that small-cap firms carry their 12-month momentum to a greater degree than large-cap companies.

It is important to note that these interactions are model-specific and *not* transferable to our MLP-Pyr specification. As seen in Appendix Figure B.7, the MLP-Pyr specification seems to have fewer non-linear interactions between the described variables and size. In addition, it should also be mentioned that the level of both marginal and interaction effects are very small when measured in excess returns in this ceteris paribus setting.

Table 4.4: Performance of the machine learning portfolios

	OLS				LASSO			
	Pred, %	Avg, %	SD	SR	Pred, %	Avg, %	SD	SR
Low (L)	-3.460	0.281	0.079	0.123	-1.602	0.505	0.070	0.249
2	-1.335	-0.219	0.063	-0.120	-0.356	-0.075	0.065	-0.040
3	-0.018	0.568	0.054	0.364	0.376	-0.399	0.052	-0.267
4	1.348	0.723	0.064	0.394	1.134	1.069	0.060	0.619
High (H)	3.636	0.177	0.062	0.099	2.485	0.471	0.067	0.244
H-L	7.096	-0.104	0.065	-0.055	4.088	-0.034	0.071	-0.017

	MLP				MLP-Pyr			
	Pred, %	Avg, %	SD	SR	Pred, %	Avg, %	SD	SR
Low (L)	-0.085	0.254	0.066	0.133	-0.252	-0.273	0.065	-0.145
2	0.390	0.195	0.054	0.126	0.385	0.913	0.056	0.566
3	0.632	0.545	0.073	0.260	0.677	1.387	0.063	0.758
4	0.896	-0.163	0.059	-0.096	0.906	-0.555	0.061	-0.316
High (H)	1.257	0.674	0.072	0.323	1.248	0.325	0.071	0.159
H-L	1.342	0.420	0.063	0.232	1.500	0.599	0.066	0.317

NOTE: Performance of prediction-sorted equally weighted portfolios over the 4-year out-of-sample testing period. All stocks are sorted into quintiles based on their predicted returns for the next month. 'Pred,' 'Avg,' 'SD,' and 'SR' provide the predicted return, average realized return, standard deviations, and annualized Sharpe ratios. All portfolios are equal-weighted.

4.4 Machine learning portfolios

Up to this point, our analysis has focused exclusively on point forecasts of excess returns, where model performance has yielded limited improvements relative to a naïve benchmark predicting the mean return. However, the mean alone provides no actionable guidance for portfolio construction; it can only tell whether an investor should go either long or short in the market portfolio. Furthermore, a model's forecasting accuracy does not necessarily translate into profitable investment strategies, as point accuracy is not a necessary requirement for selecting the right stock. To address this, we construct a set of portfolios explicitly designed to exploit the model-generated return forecasts. At the end of each month, we rebalance the portfolio based on the one-month-ahead out-of-sample predictions.

Given that our models are specifically trained on Danish equity data, we restrict the investment universe to constituents of the OMXC25 index. We do this to mitigate potential concerns related to illiquidity, which is likely pronounced among smaller-cap Danish stocks and could hinder the realization of excess returns in a practical setting.

Table 4.5: Performance of the OMXC25 index, $\text{OMXC25} - r^{\text{risk free}}$

	OMXC25				
	Avg, %	Std	SR	Max DD, %	Max 1M loss, %
Long index	0.066	0.048	0.048	27.189	12.976

NOTE: Performance of OMXC25. Report metrics are average return (Avg), standard deviation (SD), annualized Sharpe ratio (SR) maximum drawdowns (Max DD) and the most extreme negative monthly return (Max 1M Loss).

The portfolio is structured as a zero-net-investment strategy: each month, we take long positions in the five stocks with the highest predicted returns (quintile 5) and short positions in the five with the lowest predicted returns (quintile 1).¹¹

In the event that a stock exits the OMXC25 index, either due to regular semiannual rebalancing or exceptional circumstances,¹² we allow the portfolio to retain the position until the end of the current month, after which the index update is reflected in the rebalancing.

Table 4.4 reports, for each model, the average out-of-sample predicted excess return (Pred), average realized return (Avg), portfolio volatility (SD), and the resulting Sharpe ratio (SR). The OLS model yields a High-minus-Low (H-L) portfolio with a negative average realized monthly return of -0.1% . LASSO improves slightly on OLS, consistent with its stronger in-sample predictive statistics. Nonetheless, they are both handily outperformed by our two neural networks. Both the baseline MLP and the MLP-Pyr produce H-L portfolios with positive average monthly returns of 0.4% and 0.6% , respectively. The Sharpe ratios of the portfolios follow a similar pattern. OLS and LASSO perform the worst, while the MLP-Pyr appears to have an edge over the MLP model, with Sharpe ratios of 0.317 and 0.232 , respectively. In comparison, Gu, Kelly, and Xiu (2020) achieve an out-of-sample Sharpe ratio of 1.35 in their best value-weighted 10-1 H-L portfolio. This picture is in unison with the relative ranking of predictive performance following our AMADL measure, as reported in Table 4.3, where MLP-Pyr scored the lowest loss, followed by MLP, LASSO, and, lastly, OLS, even though we have shrunk our stock universe significantly.

However, both neural networks struggle to sort stocks correctly across all quintiles. For example, each network predicts an average monthly return of about 0.9% for the fourth quintile, yet the realized average returns in that bucket are negative.

¹¹Gu, Kelly, and Xiu (2020) construct similar portfolios using deciles, however, their stock universe is S&P 500. As we only trade on 25 stocks, we use quintiles instead.

¹²E.g., when Chr. Hansen A/S was removed from the index following its merger with Novozymes, Nasdaq, Inc. (2024).

Table 4.6: Drawdowns, max 1-month loss, and turnover for machine learning portfolios

OLS			
	Max DD, %	Max 1M loss, %	Turnover, %
Low	40.467	23.583	38.230
High	37.000	14.979	33.062
H-L	26.582	17.271	64.149
LASSO			
	Max DD, %	Max 1M loss, %	Turnover, %
Low	26.786	12.972	32.229
High	39.285	17.042	32.031
H-L	34.338	14.868	56.937
MLP			
	Max DD, %	Max 1M loss, %	Turnover, %
Low	27.904	15.006	35.547
High	43.960	15.783	36.505
H-L	39.949	12.760	57.935
MLP-Pyr			
	Max DD, %	Max 1M loss, %	Turnover, %
Low	32.795	12.972	33.451
High	48.183	15.783	37.919
H-L	34.160	12.129	64.934

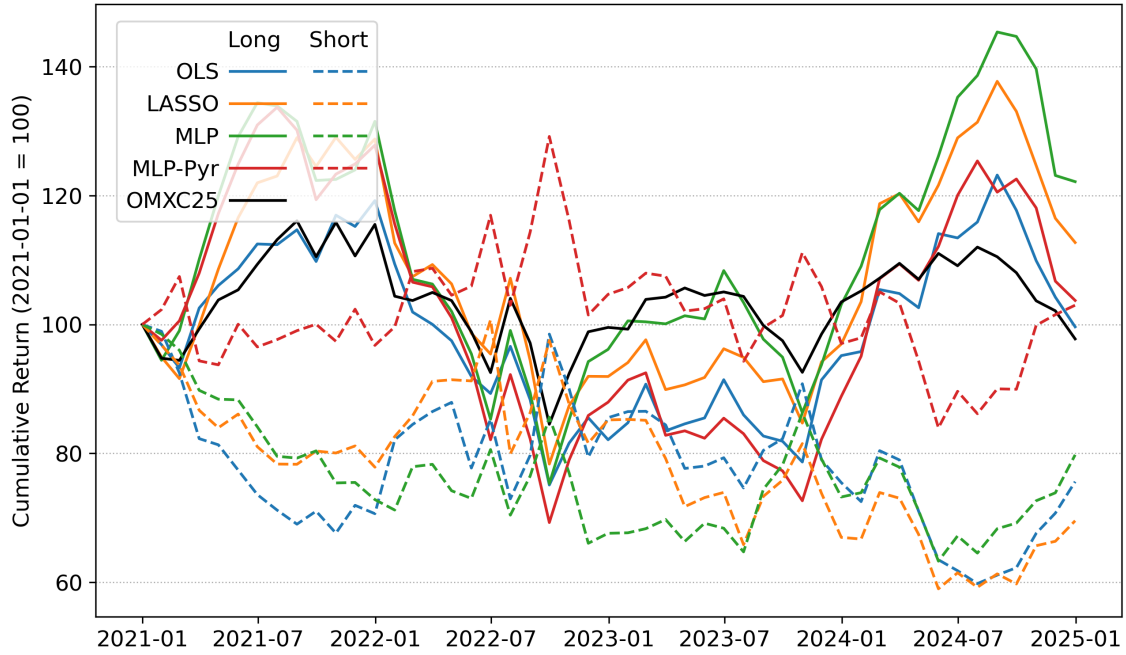
NOTE: Portfolio performance for the equally weighted prediction sorted, low, high, and high-low portfolios. Report metrics are maximum drawdowns (Max DD), the most extreme negative monthly return (Max 1M Loss), and the average monthly percentage change in holdings (Turnover).

Furthermore, while the baseline MLP generates higher returns on the long side of the H-L portfolio, the MLP-Pyr's edge lies in its ability to predict and profit from the short side, resulting in its overall edge when comparing the H-L portfolios.

Table 4.5 reports performance metrics for the OMXC25 index. Over the past four years, the index's average monthly return was just 0.0066%, which translates into an annualized Sharpe ratio of 0.048, less than one-sixth that of the MLP-Pyr and one-quarter that of the baseline MLP.

Table 4.6 reports maximum drawdown (Max DD), maximum one-month loss (Max 1M loss), and average monthly turnover (Turnover). The MLP-Pyr's H-L portfolio exhibits the smallest single-month loss (-12.1%) over the sample period, beating both the OMXC25 (-13.0%) and the baseline MLP (-12.8%). Its peak

Figure 4.8: Cumulative excess return of machine learning portfolios in OMXC25



NOTE: The figure shows the cumulative returns of machine learning portfolios trading with stocks in the OMXC25 $- r^{\text{risk-free}}$. The solid and dashed lines represent long (top quintile) and short (bottom quintile) positions, respectively. All portfolios are equal weighted.

drawdown of 32.6% is the second-lowest among algorithmic strategies, only OLS and the index drawdowns are lower. On the other hand, MLP-Pyr has the highest turnover, which, in the real world, under non-negligible trading costs, would erode some, if not all, of the portfolio excess returns.¹³

Figure 4.8 plots cumulative excess returns separately for the long and short sides of each H-L net-zero portfolio. All models generate long-side returns above the OMXC25 by the end of 2024, but only the MLP-Pyr's short book achieves a net profit over time, suggesting that reliable short signals are more challenging to identify for the other specifications.

In summary, while our neural networks do not universally outperform naïve estimates in predictive performance, they do permit the construction of net-zero H-L portfolios that would have outperformed the OMXC25 index over the past four years when trading costs are not taken into account. Nonetheless, we remain cautious; none of the models consistently have the worst performers allocated to the lowest quintile and the strongest to the highest quintile, which limits the robustness of these trading strategies.

¹³Saxo Bank A/S (2025) currently quotes an eight bps trading cost, hence a portfolio with 50% would incur commissions of 4 bps. Furthermore, one should consider the bid-ask spread, the impact of trading, and the borrowing costs associated with the short position.

5 Discussion

Data

Much of the literature in this space is using US data. There are readily available data spanning more than 70 years. We, however, attempt to predict asset risk premia on Danish equities. In writing this paper, we had to synthesize the dataset ourselves, at considerable time expense and with great difficulty. Most of the used data comes from LSEG Workspace, and here, the available history of data is limited. We also found the data to be of varying quality. Specifically, we have had to remove series where, for instance, the available stock price did not reflect the true price, which seemed to be due to a faulty dividend correction. For some of the missing data, we have gathered data independently where possible. At the end, we are only able to replicate a subset of just 55 firm-level predictors of the 94 characteristics presented by Green, Hand, and Zhang (2017). This inherently limits the information that conditions our forecasts. Moreover, the Danish stock universe is greatly limited compared to that of the US. In our final data set, a little over 400 individual stocks are present, compared to the nearly 30,000 stocks analyzed by Gu, Kelly, and Xiu (2020).

Beyond investing in improving the Danish data available, it could also be a possibility to train models on US equities and do OOS testing on Danish equities. There is no guarantee that the underlying data generating process is similar, but it would be interesting to test this. We should be clear that this only solves the issue of limited training data, e.g., over time and stocks, not the limited number of firm-level predictors. However, it is not unlikely to think that such an approach could be beneficial.

AMADL

We introduce AMADL, a novel loss function to evaluate and rank forecasting methods. We find it attractive to utilize a metric that more closely aligns the loss function for evaluating the forecasts with the purpose of the forecast. If your sole objective is to accurately measure risk premia, the likes of RMSE and MAE are great tools. They ensure, albeit in different ways, that the error of the model is as small as possible. But oftentimes, the underlying purpose of measuring asset risk premia is to take advantage of such a forecast for AIS. AMADL proposes a solution for this by combining MADL and RMSE. In this paper, we arbitrarily pick $\delta = 0.5$. We do not present further theory as to how δ should be chosen. δ tunes the relative importance of forecasting the correct sign vs. forecast accuracy. There is no trivial answer to an appropriate δ . It goes without saying that if a model achieved perfect accuracy, it

would likewise forecast the sign correctly. As soon as a forecast deviates from perfect accuracy, which probably goes for all forecasts, forecasting the correct sign becomes relevant.

AMADL is an attempt to bridge the gap between what we ask of our forecasts and how we evaluate them. Limiting the scope of this paper prevents us from exhaustively testing AMADL to further explore the properties of this loss function, which would be an excellent future idea. Nevertheless, we do actually see the ranking of forecasts produced by AMADL over the full forecast period on all stocks translate into the ranking in our trading backtesting. We should not put too much emphasis on this result, as the backtesting is performed on a small subset consisting of OMXC25 stocks, not to mention that the strategies are formed on quintiles, further reducing the overlap of data. We can speculate that this result would generalize if we backtested on the entire forecasting population, but we do not test this formally.

We would like to highlight a potential issue that arises when we use the well-known quadratic error of MSE to translate into the importance of portfolio weighting in AIS. While this forms a desirable loss surface that holds the properties we seek, it is not trivial how this should translate into portfolio strategies. Since the forecast error is not known *ex ante*, this cannot inform portfolio strategy. Some sort of weighting must be done, but it cannot translate directly into the portfolio performance, exactly as the quadratic loss penalizes AMADL. Thus, how AMADL ranks forecasts can never be guaranteed to translate into the portfolio returns. The better the accuracy, the less the issue. While this is not a paper on optimal portfolio strategy, when we try to bridge the gap between the use case of the forecasts and how we evaluate them, we cannot avoid pointing out this weakness.

Lastly, we use AMADL in the context of evaluation, but equally interesting would be to train a network with this objective. We argue that forecasting the correct sign is important, but when we train the model on MSE, there is no concern for forecasting the correct sign embedded in the loss function. There is a chance that explicitly training the model with this objective might produce a model with better forecasting abilities for AIS. White (1988) conclude similarly, "Finally, it is extremely important to point out that while the method of least squares (equivalently, back-propagation) is adequate for testing the efficient markets hypothesis, it is not necessarily the method that one should use if interest attaches to building a network for market trading purposes." Furthermore, you could consider whether the sign of the forecast should be handled by a different type of network. We, of course, need to limit the scope of this paper, and thus focus solely on feedforward neural nets, but you could consider assigning the task of forecasting the sign to a classification network, or use

a tree algorithm.

Evaluation of forecasts

In our paper, we solely rely on RMSE and MAE as classical metrics for evaluating the forecasts, in addition to our own metric AMADL. Gu, Kelly, and Xiu (2020) rely heavily on R^2 . However, we do not find the metric particularly appealing, as it can give a false sense of absolute evaluation of the forecasts. We do include the naïve benchmark in Table 4.3, to give the reader an idea of an absolute anchor for the forecasts, but the metrics are best used to rank these models. In other words, if guessing naïvely compares similarly to our forecasts, these forecasts are not, in a classical sense, very good. Gu, Kelly, and Xiu (ibid.) also argues that R^2 should not be mean-corrected, corresponding to a naïve forecast of zero. We do not find a naïve forecast of zero superior to a mean forecast in our data.

Beyond R^2 , Gu, Kelly, and Xiu (ibid.) apply a modified Diebold-Mariano test by averaging squared-error differences across assets to form a single loss-differential. However, we unfortunately do not find the Diebold-Mariano test to be suitable for our forecasts. In their original paper Diebold and Mariano (1995) they note, "The distribution theory required for joint relaxation of (2b) and (2c), for example, is complicated by the presence of fourth-order cumulants in the distribution of the sample autocovariances[...]", with (2b) and (2c) being the assumptions of Gaussian and serially uncorrelated forecast errors. Not only can we not make such claims, but we, simultaneously, cannot argue for (2d), the assumption of contemporaneously uncorrelated errors. Furthermore, the original DM framework was derived for a two-forecast comparison, not a large-panel setting. It is not clear to us if this could be appropriate. Nevertheless, we do not find any alternatives to speak to statistical significance in pairwise or panel comparisons of forecasts. Thus, we avoid incorporating such measures.

Hyperparameter tuning

In Section 4.2, we tune the hyperparameters of the model and explore the importance of activation and loss choice. We should mention that we are fully aware that all hyperparameter choices are expected to be highly interdependent. For simplicity and computational limitations, we do not suboptimize all of these choices. You might hypothetically be able to improve the OOS network performance by doing so.

We rely on grid search to tune most hyperparameters of our neural network. While straightforward, our grid search is not exhaustive due to the grid size and may miss optimal configurations, especially in high-dimensional parameter spaces. A more sophisticated alternative is Bayesian hyperparameter optimization of tools

like Optuna. Unlike grid search, Bayesian methods model the relationship between hyperparameters and performance, allowing for more intelligent exploration of the search space. This may lead to better-performing models with fewer evaluations, making them particularly appealing when training is computationally costly. Future work could benefit from adopting such approaches to improve both efficiency and performance.

Comparing to OLS and LASSO

We use OLS and LASSO to put neural networks in perspective. While many alternative, and more sophisticated, models could be interesting to include, like Gu, Kelly, and Xiu (2020). We take our starting point in OLS, as the most well-known regression method. We add the LASSO norm penalty to the linear model to show the effect of regularization, and lastly, the neural networks allow us to deviate from linearity. This is exemplified in our simulations in Section 2.5. While we think decomposing some of the advantages of neural networks into both regularization and non-linearity over the classical least squares, we would argue that the simple OLS implementation is not appropriate here. We can almost speculate prior to conducting this analysis that OLS will have a hard time. We do not report statistical significance for the OLS, as we have no similar alternatives for FFNNs, and thus, this will not add to our understanding of FFNNs. Gu, Kelly, and Xiu (ibid.) do give linear models a fair fighting chance, by also doing principal component analysis and partial least squares, both of which offer some of the dimension reduction that is achieved implicitly by adding a norm penalty as we do on the LASSO regression.

6 Concluding remarks

This thesis has sought first to demystify the inner workings of neural networks by unpacking the underlying mechanisms and illustrating their capabilities through simulation. Building on this foundation, we have explored the practical trade-offs involved in specifying and evaluating such neural networks in a financial context, highlighting the choices we made throughout the process. Finally, we have applied these insights to estimate a large-scale factor model for Danish equities, assessed to what extent neural networks meaningfully can predict point excess returns, and applied the forecasts in a trading context.

In Section 4.1, we assemble a formalized generalized network from first principles. We go to great lengths to explain the inner workings of such a machine learning method, including linear layers, non-linear activations, loss function, and regularizers. In Section 2.2, we showcase the calculations of a simple backpropagation algorithm to bring about a common understanding of how these networks function. The

main argument for utilizing this kind of estimation is the flexibility of the network. In Section 2.5 we showcase how, in the presence of non-linearities and noise, a regularized network can perform very well compared to its classical counterparts. A low signal-to-noise ratio and complex variable interactions are likely to arise in the stock market, making neural networks a tempting method of estimation. Thus, the network is revealed not as a “black box” but as an adaptive collection of linear sub-models tied together by activation functions.

We continue by rethinking how to evaluate forecasts in this context in Section 2.4. The well-known error-based loss functions have great merits when the underlying purpose of the forecasts is to accurately measure asset risk premia. We, however, argue that the underlying purpose of measuring asset risk premia is often to inform AIS, in which case there is a gap between the criterion of evaluation and the purpose. We are not the first ones to take an interest in this area, but we adopt the loss function of Michańków, Sakowski, and Ślepaczuk (2023) to propose a novel loss function called AMADL, which aims to evaluate and rank forecasts based on their ability to inform AIS. By doing so, we do lose some of that neat interpretability of the original MADL function, but potentially improve the usefulness in informing AIS.

Furthermore, we showcase our model specification and tuning choices. We deep dive into width vs depth, activation functions, and loss functions. We do see a tendency towards a width-to-depth trade-off, where networks do seem to improve by adding neurons to a shallow network, but as networks increase in width, the appetite for depth seems to decrease. Furthermore, we find the network to be quite robust to the choice of activation function. Given the size of our network, no activation functions are computationally infeasible, and all achieve very similar validation losses. Similarly, the choice of loss function seems to have a negligible impact on validation loss. We find that if you train a network with the objective of MSE or MAE, it will also perform best in its category, in terms of RMSE and MAE, respectively, with Huber-loss landing somewhere in between.

In Section 4 we conduct a large-scale empirical analysis of a panel of roughly 400 currently or previously listed Danish firms with more than 55 firm-level characteristics. Overall, neither our linear specifications nor our neural network architectures deliver substantial improvements in point forecasts relative to a naïve benchmark; only a single neural network variant (MLP-Pyr) shows slight improvements. When we examine variable importance across the models, we observe that most of the top predictors, for both the neural networks and the LASSO model, are broad market factors related to size, liquidity, systematic risk, and momentum. Only a

few accounting-based predictors appear among the fifteen most influential features, underscoring the limited incremental value of fundamental signals in our setting when a 6-month delay is added.

Finally, we evaluate our models in a trading context within the OMXC25 stock universe by back-testing a High-Low net-zero portfolio based on predicted excess returns. We find that though our neural networks do not universally outperform a naïve point estimate, we do manage to outperform the OMXC25 with both neural network specifications. Our best neural network specification, MLP-Pyr, achieves a monthly return of 0.6% and a Sharpe ratio of 0.317, well below the best 10-1 portfolio of Gu, Kelly, and Xiu (2020), earning a Sharpe ratio of 1.35. However, this result should be interpreted with caution. Even in the MLP-Pyr network, the third quintile of stocks (rather than the top quintile) posts the highest realized return, indicating that the model’s ranking stability remains fragile. As such, while we demonstrate that a carefully regularized neural network can extract modest excess return in a small market, the practical robustness of these gains is constrained by the model’s imperfect ability to sort stocks reliably.

Bibliography

- Banz, Rolf W. (Mar. 1981). “The relationship between return and market value of common stocks”. In: *Journal of Financial Economics* 9.1, pp. 3–18. DOI: [10.1016/0304-405X\(81\)90018-0](https://doi.org/10.1016/0304-405X(81)90018-0).
- Basu, S. (1977). “Investment Performance of Common Stocks in Relation to Their Price-Earnings Ratios: A Test of the Efficient Market Hypothesis”. In: *The Journal of Finance* 32.3, pp. 663–682. DOI: [10.2307/2326304](https://doi.org/10.2307/2326304).
- Bianchi, Daniele, Matthias Büchner, and Andrea Tamoni (Feb. 2020). *Bond Risk Premia with Machine Learning*. DOI: [10.2139/ssrn.3232721](https://doi.org/10.2139/ssrn.3232721).
- Black, Fischer (1972). “Capital Market Equilibrium with Restricted Borrowing”. In: *The Journal of Business* 45.3, pp. 444–455.
- Black, Fischer and Myron Scholes (1973). “The Pricing of Options and Corporate Liabilities”. In: *Journal of Political Economy* 81.3, pp. 637–654.
- Carhart, Mark M. (1997). “On Persistence in Mutual Fund Performance”. In: *The Journal of Finance* 52.1, pp. 57–82. DOI: [10.2307/2329556](https://doi.org/10.2307/2329556).
- Cochrane, John H. (2011). “Presidential Address: Discount Rates”. In: *The Journal of Finance* 66.4, pp. 1047–1108. DOI: [10.1111/j.1540-6261.2011.01671.x](https://doi.org/10.1111/j.1540-6261.2011.01671.x).
- Cybenko, G. (Dec. 1989). “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314. DOI: [10.1007/BF02551274](https://doi.org/10.1007/BF02551274).
- Diebold, Francis X. and Roberto S. Mariano (1995). “Comparing Predictive Accuracy”. In: *Journal of Business & Economic Statistics* 13.3, pp. 253–263. DOI: [10.2307/1392185](https://doi.org/10.2307/1392185).
- Erhvervsstyrelsen (Danish Business Authority) (Apr. 2025). *Central Business Register (CVR) Database*. URL: <https://datacvr.virk.dk/>.
- Fama, Eugene F. (1970). “Efficient Capital Markets: A Review of Theory and Empirical Work”. In: *The Journal of Finance* 25.2, pp. 383–417. DOI: [10.2307/2325486](https://doi.org/10.2307/2325486).
- Fama, Eugene F. and Kenneth R French (2004). “The Capital Asset Pricing Model: Theory and Evidence”. In: *Journal of Economic Perspectives* 18.3, pp. 25–46.
- (1992). “The Cross-Section of Expected Stock Returns”. In: *The Journal of Finance* 47.2, pp. 427–465. DOI: [10.2307/2329112](https://doi.org/10.2307/2329112).
- (Feb. 1993). “Common risk factors in the returns on stocks and bonds”. In: *Journal of Financial Economics* 33.1, pp. 3–56. DOI: [10.1016/0304-405X\(93\)90023-5](https://doi.org/10.1016/0304-405X(93)90023-5).
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. MIT Press.

- Graham, Benjamin and David L. Dodd (1934). *Security Analysis*. New York: McGraw-Hill Book Company.
- Green, Jeremiah, John R. M. Hand, and X. Frank Zhang (Dec. 2017). “The Characteristics that Provide Independent Information about Average U.S. Monthly Stock Returns”. In: *The Review of Financial Studies* 30.12, pp. 4389–4436. DOI: [10.1093/rfs/hhx019](https://doi.org/10.1093/rfs/hhx019).
- Gu, Shihao, Bryan Kelly, and Dacheng Xiu (May 2020). “Empirical Asset Pricing via Machine Learning”. In: *The Review of Financial Studies* 33.5. Ed. by Andrew Karolyi, pp. 2223–2273. DOI: [10.1093/rfs/hhaa009](https://doi.org/10.1093/rfs/hhaa009).
- Harvey, Campbell R., Yan Liu, and Caroline Zhu (Feb. 2015). ... and the Cross-Section of Expected Returns. DOI: [10.2139/ssrn.2249314](https://doi.org/10.2139/ssrn.2249314).
- Hastie, Trevor, Robert Tibshirani, and Jerome Friedman (2009). *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY: Springer New York. ISBN: 978-0-387-84858-7.
- He, Kaiming et al. (Feb. 2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. DOI: [10.48550/arXiv.1502.01852](https://doi.org/10.48550/arXiv.1502.01852).
- Hebb, Donald O. (1949). *Organization of Behavior: A Neuropsychological Theory*. JOHN WILEY & SONS Inc. ISBN: 978-1-135-63190-1.
- Hoerl, Arthur E. and Robert W. Kennard (1970). “Ridge Regression: Biased Estimation for Nonorthogonal Problems”. In: *Technometrics* 12.1, pp. 55–67. DOI: [10.2307/1267351](https://doi.org/10.2307/1267351).
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White (Jan. 1989). “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5, pp. 359–366. DOI: [10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8).
- Hou, Kewei and Tobias J. Moskowitz (2005). “Market Frictions, Price Delay, and the Cross-Section of Expected Returns”. In: *The Review of Financial Studies* 18.3, pp. 981–1020.
- Hutchinson, James M., Andrew W. Lo, and Tomaso Poggio (1994). “A Nonparametric Approach to Pricing and Hedging Derivative Securities Via Learning Networks”. In: *The Journal of Finance* 49.3, pp. 851–889. DOI: [10.1111/j.1540-6261.1994.tb00081.x](https://doi.org/10.1111/j.1540-6261.1994.tb00081.x).
- Jegadeesh, Narasimhan and Sheridan Titman (1993). “Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency”. In: *The Journal of Finance* 48.1, pp. 65–91. DOI: [10.2307/2328882](https://doi.org/10.2307/2328882).
- Kingma, Diederik P. and Jimmy Ba (Jan. 2017). *Adam: A Method for Stochastic Optimization*. DOI: [10.48550/arXiv.1412.6980](https://doi.org/10.48550/arXiv.1412.6980).

- Lintner, John (1965). “The Valuation of Risk Assets and the Selection of Risky Investments in Stock Portfolios and Capital Budgets”. In: *The Review of Economics and Statistics* 47.1, pp. 13–37. DOI: [10.2307/1924119](https://doi.org/10.2307/1924119).
- London Stock Exchange Group (May 2025). *LSEG Workspace*. URL: <https://www.lseg.com/en/data-analytics/products/workspace/download-workspace>.
- Markowitz, Harry M. (1959). *Portfolio Selection: Efficient Diversification of Investments*. Yale University Press. ISBN: 978-0-300-01372-6.
- Masters, Timothy (Jan. 1993a). “10 - Designing Feedforward Network Architectures”. In: *Practical Neural Network Recipes in C++*. Morgan Kaufmann, pp. 173–185. ISBN: 978-0-08-051433-8.
- (Jan. 1993b). “19 - Evaluating Performance of Neural Networks”. In: *Practical Neural Network Recipes in C++*. Morgan Kaufmann, pp. 343–360. ISBN: 978-0-08-051433-8.
- McCulloch, Warren S. and Walter Pitts (Dec. 1943). “A logical calculus of the ideas immanent in nervous activity”. In: *The bulletin of mathematical biophysics* 5.4, pp. 115–133. DOI: [10.1007/BF02478259](https://doi.org/10.1007/BF02478259).
- Michańków, Jakub, Paweł Sakowski, and Robert Ślepaczuk (Sept. 2023). *Mean Absolute Directional Loss as a New Loss Function for Machine Learning Problems in Algorithmic Investment Strategies*. DOI: [10.48550/arXiv.2309.10546](https://doi.org/10.48550/arXiv.2309.10546).
- Nasdaq, Inc. (Jan. 2024). *Novozymes A/S and Chr. Hansen A/S - exchange notice regarding merger*. URL: <https://www.nasdaq.com/press-release/novozymes-a-s-and-chr.-hansen-a-s-exchange-notice-regarding-merger-2024-01-29>.
- Orimoloye, Larry Olanrewaju et al. (Jan. 2020). “Comparing the effectiveness of deep feedforward neural networks and shallow architectures for predicting stock price indices”. In: *Expert Systems with Applications* 139, p. 112828. DOI: [10.1016/j.eswa.2019.112828](https://doi.org/10.1016/j.eswa.2019.112828).
- Paszke, Adam et al. (Dec. 2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. DOI: [10.48550/arXiv.1912.01703](https://doi.org/10.48550/arXiv.1912.01703).
- Pedregosa, Fabian et al. (2011). “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12.85, pp. 2825–2830.
- Peña, Daniel and Ruey S. Tsay (2021). *Statistical learning for big dependent data*. First edition. Wiley series in probability and statistics. Hoboken, NJ: Wiley. ISBN: 978-1-119-41741-5.
- Rosenblatt, F. (1958). “The perceptron: A probabilistic model for information storage and organization in the brain.” In: *Psychological Review* 65.6, pp. 386–408. DOI: [10.1037/h0042519](https://doi.org/10.1037/h0042519).

- Ross, Stephen A (Dec. 1976). “The arbitrage theory of capital asset pricing”. In: *Journal of Economic Theory* 13.3, pp. 341–360. DOI: [10.1016/0022-0531\(76\)90046-6](https://doi.org/10.1016/0022-0531(76)90046-6).
- Saxo Bank A/S (June 2025). *Stock commissions*. URL: <https://www.home.saxo.com/rates-and-conditions/stocks/commissions>.
- Sharpe, William F. (1964). “Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk”. In: *The Journal of Finance* 19.3, pp. 425–442. DOI: [10.2307/2977928](https://doi.org/10.2307/2977928).
- (1966). “Mutual Fund Performance”. In: *The Journal of Business* 39.1, pp. 119–138.
- Statistics Denmark (Apr. 2025). *MPK3: StatBank Denmark Database*. URL: <https://www.statistikbanken.dk/statbank5a/default.asp?w=1470>.
- Sutskever, Ilya et al. (May 2013). “On the importance of initialization and momentum in deep learning”. In: *Proceedings of the 30th International Conference on Machine Learning*. PMLR, pp. 1139–1147.
- Terven, Juan et al. (Apr. 2025). “Loss Functions and Metrics in Deep Learning”. In: *Artificial Intelligence Review* 58.7, p. 195. DOI: [10.1007/s10462-025-11198-7](https://doi.org/10.1007/s10462-025-11198-7).
- Tibshirani, Robert (1996). “Regression Shrinkage and Selection via the Lasso”. In: *Journal of the Royal Statistical Society. Series B (Methodological)* 58.1, pp. 267–288.
- Welch, Ivo and Amit Goyal (July 2008). “A Comprehensive Look at The Empirical Performance of Equity Premium Prediction”. In: *The Review of Financial Studies* 21.4, pp. 1455–1508. DOI: [10.1093/rfs/hhm014](https://doi.org/10.1093/rfs/hhm014).
- White (1988). “Economic prediction using neural networks: the case of IBM daily stock returns”. In: *IEEE International Conference on Neural Networks*. IEEE, 451–458 vol.2. DOI: [10.1109/ICNN.1988.23959](https://doi.org/10.1109/ICNN.1988.23959).
- White, Halbert (Dec. 1989). “Some Asymptotic Results for Learning in Single Hidden-Layer Feedforward Network Models”. In: *Journal of the American Statistical Association* 84.408, pp. 1003–1013.
- Zou, Hui and Trevor Hastie (Apr. 2005). “Regularization and Variable Selection Via the Elastic Net”. In: *Journal of the Royal Statistical Society Series B: Statistical Methodology* 67.2, pp. 301–320. DOI: [10.1111/j.1467-9868.2005.00503.x](https://doi.org/10.1111/j.1467-9868.2005.00503.x).

A Appendix

A.1 Pseudocode for gradient descent

Algorithm 3 Batch Gradient Descent

Require: Dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, f_θ , Loss $\mathcal{L}(\cdot)$, initial θ_0 , learning rate η , iterations T
initialize : $\theta_{t-1} \leftarrow \theta_0$
for $t = 1$ to T **do**
 $g_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1})$
 $\theta_t \leftarrow \theta_{t-1} - \eta g_t$
end for
return θ_t

Algorithm 4 Mini-Batch Gradient Descent

Require: Dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, f_θ , Sample loss $\xi(\cdot)$, initial θ_0 , batch size b ($b = 1$ is pure SGD), learning rate η , epochs E
initialize : $T \leftarrow \lceil \frac{N}{b} \rceil$
for $e = 1$ to E **do**
 Shuffle dataset
 Split the data into batches $B_1, \dots, B_T$ of size b (last may be smaller)
 for $t = 1$ to T **do**
 $g_t \leftarrow \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_\theta \xi(y_i, f_\theta(\mathbf{x}_i))$
 $\theta_t \leftarrow \theta_{t-1} - \eta g_t$
 end for
end for
return θ_t

Algorithm 5 Mini-Batch Gradient Descent with Momentum

Require: Dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, f_θ , Sample loss $\xi(\cdot)$, initial θ_0 , batch size b , learning rate η , epochs E , momentum β

initialize : $T \leftarrow \lceil \frac{N}{b} \rceil$, $v_{t-1} \leftarrow 0$

for $e = 1$ to E **do**

 Shuffle dataset

 Split the data into batches $B_1, \dots, B_T$ of size b (last may be smaller)

for $t = 1$ to T **do**

$g_t \leftarrow \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_\theta \xi(y_i, f_\theta(\mathbf{x}_i))$

$v_t \leftarrow \beta v_{t-1} + (1 - \beta) g_t$

$\theta_t \leftarrow \theta_{t-1} - \eta v$

end for

end for

return θ_t

Algorithm 6 ADAM (Mini-batch)

Require: Dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, f_θ , Sample loss $\xi(\cdot)$, initial θ_0 , batch size b , learning rate η , epochs E , decay rates $upsilon_1, upsilon_2, \epsilon$ (term for numerical stability)

initialize : $T \leftarrow \lceil \frac{N}{b} \rceil$, $v_0 \leftarrow 0$, $m_0 \leftarrow 0$

for $e = 1$ to E **do**

 Shuffle dataset

 Split the data into batches $B_1, \dots, B_T$ of size b (last may be smaller)

for $t = 1$ to T **do**

$g_t \leftarrow \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_\theta \xi(y_i, f_\theta(\mathbf{x}_i))$

$m_t \leftarrow upsilon_1 m_{t-1} + (1 - upsilon_1) g_t$

$v_t \leftarrow upsilon_2 v_{t-1} + (1 - upsilon_2) g_t^2$

$\hat{m}_t \leftarrow m_t / (1 - upsilon_1^t)$

$\hat{v}_t \leftarrow v_t / (1 - upsilon_2^t)$

$\theta_t \leftarrow \theta_{t-1} - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$

end for

end for

return θ_t

NOTE: ADAM mini-batch implementation. The gradients is therefore averaged for the mini-batch. If $b = 1$ then this collapses to an SDG implementation of ADAM

A.2 Pseudocode for early stopping

Algorithm 7 Early Stopping

Require: n number of iterations between evaluations, ι patience, θ_0 initial parameters.

$\theta \leftarrow \theta_0$

$i \leftarrow 0$

$j \leftarrow 0$

$v \leftarrow \infty$

$\theta^* \leftarrow \theta$

$i^* \leftarrow i$

while $j < \iota$ **do**

 Update θ by running the training algorithm for n steps.

$i \leftarrow i + n$

$v' \leftarrow \mathcal{L}_{val}(\theta)$

if $v' < v$ **then**

$j \leftarrow 0$

$\theta^* \leftarrow \theta$

$i^* \leftarrow i$

$v \leftarrow v'$

else

$j \leftarrow j + 1$

end if

end while

return best parameters θ^* , best number of training steps i^*

NOTE: Early stopping pseudocode following Goodfellow, Bengio, and Courville (2016). The algorithm returns the best model parameters θ^* that was obtained before patience ρ was reached. Like Goodfellow, Bengio, and Courville (2016), we here do not keep the time-subscripts. Like we propose in section 2.2, variables are simply overwritten.

A.3 Deriving gradient descent

We have MSE:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

Let us first define:

$$f(k_i(\theta)) \equiv \frac{1}{2} (k_i(\theta))^2, \quad k_i(\theta) \equiv y_i - \hat{y}_i = y_i - \boldsymbol{\beta}^\top \mathbf{h}_i,$$

$$\theta \equiv (\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3, \boldsymbol{\beta}) \in \Theta = (\mathbb{R}^3)^4.$$

To find the gradient with respect to $\boldsymbol{\beta}$, we can first derive:

$$\frac{\partial f(k_i(\theta))}{\partial k_i(\theta)} = k_i(\theta), \quad \nabla_{\boldsymbol{\beta}} k_i(\theta) = -\mathbf{h}_i,$$

which we can use to find:

$$\begin{aligned} \nabla_{\boldsymbol{\beta}} \mathcal{L}_{MSE} &= \frac{1}{N} \sum_{i=1}^N \frac{\partial f(k_i(\theta))}{\partial k_i(\theta)} \cdot \nabla_{\boldsymbol{\beta}} k_i(\theta) \\ &= \frac{1}{N} \sum_{i=1}^N k_i(\theta) \cdot (-\mathbf{h}_i) \\ &= \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \cdot \mathbf{h}_i. \end{aligned} \tag{A.1}$$

We can now move on to the gradients for $\mathbf{w}_m$ for $m = 1, 2, 3$:

$$\nabla_{\mathbf{w}_m} \mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N \frac{\partial f(k_i(\theta))}{\partial k_i(\theta)} \cdot \frac{\partial k_i(\theta)}{\partial \mathbf{w}_m}$$

For $\nabla_{\mathbf{w}_m} \mathcal{L}_{MSE}$, we need:

$$\nabla_{\mathbf{w}_m} k_i(\theta) = -\beta_m \cdot \mathbf{x}_i \cdot g'(u_{m,i}),$$

where we from equation 2.3 have:

$$g'(u_{mi}) = \begin{cases} 1, & u_{m,i} \geq 0, \\ \alpha, & u_{m,i} < 0 \end{cases}, \quad \alpha = 0.1.$$

As such, we now have:

$$\begin{aligned} \nabla_{\mathbf{w}_m} \mathcal{L}_{MSE} &= \frac{1}{N} \sum_{i=1}^N k_i(\theta) \cdot (-\beta_m) \cdot \mathbf{x}_i \cdot g'(u_{m,i}) \\ &= \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \cdot \beta_m \cdot \mathbf{x}_i \cdot g'(u_{m,i}) \end{aligned} \tag{A.2}$$

A.4 Forward pass calculations

For observation two, we have:

$$\mathbf{x}_2^\top = [0, 0, 3], \quad y_2 = 1.6127,$$

and the forward pass is:

$$\begin{aligned} u_{1,2} &= \mathbf{w}_1^\top \mathbf{x}_2 = (-0.1502)(0) + (-0.5532)(0) + (0.9932)(3) = 2.9796, \\ u_{2,2} &= \mathbf{w}_2^\top \mathbf{x}_2 = (-0.1255)(0) + (-0.3480)(0) + (-0.2861)(3) = -0.8583, \\ u_{3,2} &= \mathbf{w}_3^\top \mathbf{x}_2 = (0.4325)(0) + (0.2969)(0) + (0.3353)(3) = 1.0059, \\ h_{1,2} &= g(u_{1,2}) = u_{1,2} = 2.9796, \\ h_{2,2} &= g(u_{2,2}) = 0.1u_{2,2} = 0.1(-0.8583) = -0.0858, \\ h_{3,2} &= g(u_{3,2}) = u_{3,2} = 1.0059, \\ q_2 &= \boldsymbol{\beta}^\top \mathbf{h}_2 = (0.2487)(2.9796) + (1.2365)(-0.0858) + (-0.2346)(1.0059) = 0.3989, \\ \hat{y}_2 &= g_v(q_2) = q_2 = 0.3989. \end{aligned}$$

For observation three, we have:

$$\mathbf{x}_3^\top = [-3, 1, -2], \quad y_3 = -9.9532,$$

and the forward pass is:

$$\begin{aligned} u_{1,3} &= \mathbf{w}_1^\top \mathbf{x}_3 = (-0.1502)(-3) + (-0.5532)(1) + (0.9932)(-2) = -2.0890, \\ u_{2,3} &= \mathbf{w}_2^\top \mathbf{x}_3 = (-0.1255)(-3) + (-0.3480)(1) + (-0.2861)(-2) = 0.6007, \\ u_{3,3} &= \mathbf{w}_3^\top \mathbf{x}_3 = (0.4325)(-3) + (0.2969)(1) + (0.3353)(-2) = -1.6712, \\ h_{1,3} &= g(u_{1,3}) = 0.1 u_{1,3} = -0.2089, \\ h_{2,3} &= g(u_{2,3}) = u_{2,3} = 0.6007, \\ h_{3,3} &= g(u_{3,3}) = 0.1 u_{3,3} = -0.1671, \\ q_3 &= \boldsymbol{\beta}^\top \mathbf{h}_3 = (0.2487)(-0.2089) + (1.2365)(0.6007) + (-0.2346)(-0.1671) = 0.7300, \\ \hat{y}_3 &= g_v(q_3) = q_3 = 0.7300. \end{aligned}$$

For observation four, we have:

$$\mathbf{x}_4^\top = [-3, 0, 3], \quad y_4 = -4.5859,$$

and the forward pass is:

$$\begin{aligned}
u_{1,4} &= \mathbf{w}_1^\top \mathbf{x}_4 = (-0.1502)(-3) + (-0.5532)(0) + (0.9932)(3) = 3.4302, \\
u_{2,4} &= \mathbf{w}_2^\top \mathbf{x}_4 = (-0.1255)(-3) + (-0.3480)(0) + (-0.2861)(3) = -0.4818, \\
u_{3,4} &= \mathbf{w}_3^\top \mathbf{x}_4 = (0.4325)(-3) + (0.2969)(0) + (0.3353)(3) = -0.2916, \\
h_{1,4} &= g(u_{1,4}) = u_{1,4} = 3.4302, \\
h_{2,4} &= g(u_{2,4}) = 0.1u_{2,4} = -0.0482, \\
h_{3,4} &= g(u_{3,4}) = 0.1u_{3,4} = -0.0292, \\
q_4 &= \boldsymbol{\beta}^\top \mathbf{h}_4 = (0.2487)(3.4302) + (1.2365)(-0.0482) + (-0.2346)(-0.0292) = 0.8004, \\
\hat{y}_4 &= g_v(q_4) = q_4 = 0.8004
\end{aligned}$$

For observation five, we have:

$$\mathbf{x}_5^\top = [2, 2, 2], \quad y_5 = -0.9631,$$

and the forward pass is:

$$\begin{aligned}
u_{1,5} &= \mathbf{w}_1^\top \mathbf{x}_5 = (-0.1502)(2) + (-0.5532)(2) + (0.9932)(2) = 0.5796, \\
u_{2,5} &= \mathbf{w}_2^\top \mathbf{x}_5 = (-0.1255)(2) + (-0.3480)(2) + (-0.2861)(2) = -1.5192, \\
u_{3,5} &= \mathbf{w}_3^\top \mathbf{x}_5 = (0.4325)(2) + (0.2969)(2) + (0.3353)(2) = 2.1294, \\
h_{1,5} &= g(u_{1,5}) = u_{1,5} = 0.5796, \\
h_{2,5} &= g(u_{2,5}) = 0.1u_{2,5} = -0.1519, \\
h_{3,5} &= g(u_{3,5}) = u_{3,5} = 2.1294, \\
q_5 &= \boldsymbol{\beta}^\top \mathbf{h}_5 = (0.2487)(0.5796) + (1.2365)(-0.1519) + (-0.2346)(2.1294) = -0.5433, \\
\hat{y}_5 &= g_v(q_5) = q_5 = -0.5433.
\end{aligned}$$

We can gather the estimates in:

$$\hat{\mathbf{y}}_{train} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \hat{y}_4 \\ \hat{y}_5 \end{bmatrix} = \begin{bmatrix} 0.0177 \\ 0.3989 \\ 0.7300 \\ 0.8004 \\ -0.5433 \end{bmatrix}. \quad (\text{A.3})$$

A.5 Gradient calculations

The gradient with respect to $\mathbf{w}_2$:

$$\begin{aligned}\nabla_{\mathbf{w}_2} \mathcal{L}_{MSE} &= \frac{1}{5} \sum_{i=1}^5 (\hat{y}_i - y_i) \beta_2 g'(u_{2,i}) \mathbf{x}_i \\ &= \frac{1}{5} \left[(0.0177 + 11.4934) \cdot 1.2365 \cdot 0.1 \begin{pmatrix} -3 \\ 2 \\ 1 \end{pmatrix} \right. \end{aligned} \quad (\text{A.4})$$

$$\begin{aligned} &+ (0.3989 - 1.6127) \cdot 1.2365 \cdot 0.1 \begin{pmatrix} 0 \\ 0 \\ 3 \end{pmatrix} \\ &+ (0.7300 + 9.9532) \cdot 1.2365 \cdot 1 \begin{pmatrix} -3 \\ 1 \\ -2 \end{pmatrix} \end{aligned} \quad (\text{A.5})$$

$$\begin{aligned} &+ (0.8004 + 4.5859) \cdot 1.2365 \cdot 0.1 \begin{pmatrix} -3 \\ 0 \\ 3 \end{pmatrix} \\ &+ (-0.5433 + 0.9631) \cdot 1.2365 \cdot 0.1 \begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix} \end{aligned}$$

$$= \frac{1}{5} \begin{pmatrix} -45.7935 \\ 16.1605 \\ -23.3445 \end{pmatrix} = \begin{pmatrix} -9.1587 \\ 3.2321 \\ -4.6689 \end{pmatrix}. \quad (\text{A.6})$$

The gradient with respect to $\mathbf{w}_3$

$$\begin{aligned}
\nabla_{\mathbf{w}_3} \mathcal{L}_{MSE} &= \frac{1}{5} \sum_{i=1}^5 (\hat{y}_i - y_i) \beta_3 g'(u_{3,i}) \mathbf{x}_i \\
&= \frac{1}{5} \left[(0.0177 + 11.4934) \cdot (-0.2346) \cdot 0.1 \begin{pmatrix} -3 \\ 2 \\ 1 \end{pmatrix} \right. \\
&\quad + (0.3989 - 1.6127) \cdot (-0.2346) \cdot 0.1 \begin{pmatrix} 0 \\ 0 \\ 3 \end{pmatrix} \\
&\quad + (0.7300 + 9.9532) \cdot (-0.2346) \cdot 1 \begin{pmatrix} -3 \\ 1 \\ -2 \end{pmatrix} \\
&\quad + (0.8004 + 4.5859) \cdot (-0.2346) \cdot 0.1 \begin{pmatrix} -3 \\ 0 \\ 3 \end{pmatrix} \\
&\quad \left. + (-0.5433 + 0.9631) \cdot (-0.2346) \cdot 1 \begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix} \right] \\
&= \frac{1}{5} \begin{pmatrix} 1.7441 \\ -0.9875 \\ 0.5096 \end{pmatrix} = \begin{pmatrix} 0.3488 \\ -0.1975 \\ 0.1019 \end{pmatrix}. \tag{A.7}
\end{aligned}$$

A.6 RMSE calculations

Forward pass with initialization weights (original model):

$$\begin{aligned}
u_{1,6}^{(\text{orig})} &= \mathbf{w}_1^\top \mathbf{x}_6 = (-0.1502)(2) + (-0.5532)(0) + (0.9932)(-3) = -3.2800, \\
u_{2,6}^{(\text{orig})} &= \mathbf{w}_2^\top \mathbf{x}_6 = (-0.1255)(2) + (-0.3480)(0) + (-0.2861)(-3) = 0.6073, \\
u_{3,6}^{(\text{orig})} &= \mathbf{w}_3^\top \mathbf{x}_6 = (0.4325)(2) + (0.2969)(0) + (0.3353)(-3) = -0.1409, \\
h_{1,6}^{(\text{orig})} &= g(u_{1,6}) = 0.1u_{1,6} = -0.3280, \\
h_{2,6}^{(\text{orig})} &= g(u_{2,6}) = u_{2,6} = 0.6073, \\
h_{3,6}^{(\text{orig})} &= g(u_{3,6}) = 0.1u_{3,6} = -0.0141, \\
q_6^{(\text{orig})} &= \boldsymbol{\beta}^\top \mathbf{h}_6 = 0.2487 \cdot (-0.3280) + 1.2365 \cdot (0.6073) - 0.2346 \cdot (-0.0141) \\
&= 0.6720, \\
\hat{y}_6^{(\text{orig})} &= q_6^{(\text{orig})} = 0.6720, \\
u_{1,7}^{(\text{orig})} &= \mathbf{w}_1^\top \mathbf{x}_7 = (-0.1502)(2) + (-0.5532)(0) + (0.9932)(0) = -0.3004, \\
u_{2,7}^{(\text{orig})} &= \mathbf{w}_2^\top \mathbf{x}_7 = (-0.1255)(2) + (-0.3480)(0) + (-0.2861)(0) = -0.2510, \\
u_{3,7}^{(\text{orig})} &= \mathbf{w}_3^\top \mathbf{x}_7 = (0.4325)(2) + (0.2969)(0) + (0.3353)(0) = 0.8650, \\
h_{1,7}^{(\text{orig})} &= g(u_{1,7}) = 0.1u_{1,7} = -0.0300, \\
h_{2,7}^{(\text{orig})} &= g(u_{2,7}) = 0.1u_{2,7} = -0.0251, \\
h_{3,7}^{(\text{orig})} &= g(u_{3,7}) = u_{3,7} = 0.8650, \\
q_7^{(\text{orig})} &= \boldsymbol{\beta}^\top \mathbf{h}_7 = 0.2487 \cdot (-0.0300) + 1.2365 \cdot (-0.0251) - 0.2346 \cdot (0.8650) \\
&= -0.2415, \\
\hat{y}_7^{(\text{orig})} &= -0.2415, \\
u_{1,8}^{(\text{orig})} &= \mathbf{w}_1^\top \mathbf{x}_8 = (-0.1502)(-1) + (-0.5532)(-2) + (0.9932)(3) = 4.2362, \\
u_{2,8}^{(\text{orig})} &= \mathbf{w}_2^\top \mathbf{x}_8 = (-0.1255)(-1) + (-0.3480)(-2) + (-0.2861)(3) = -0.0368, \\
u_{3,8}^{(\text{orig})} &= \mathbf{w}_3^\top \mathbf{x}_8 = (0.4325)(-1) + (0.2969)(-2) + (0.3353)(3) = -0.0204, \\
h_{1,8}^{(\text{orig})} &= g(u_{1,8}) = u_{1,8} = 4.2362, \\
h_{2,8}^{(\text{orig})} &= g(u_{2,8}) = 0.1u_{2,8} = -0.0037, \\
h_{3,8}^{(\text{orig})} &= g(u_{3,8}) = 0.1u_{3,8} = -0.0020, \\
q_8^{(\text{orig})} &= \boldsymbol{\beta}^\top \mathbf{h}_8 = 0.2487 \cdot 4.2362 + 1.2365 \cdot (-0.0037) - 0.2346 \cdot (-0.0020) \\
&= 1.0499, \\
\hat{y}_8^{(\text{orig})} &= 1.0499.
\end{aligned}$$

Forward pass with updated weights (new model):

$$\begin{aligned}
u_{1,6}^{(\text{new})} &= \mathbf{w}_1^{(\text{new})\top} \mathbf{x}_6 = (0.1137)(2) + (-0.6772)(0) + (0.8802)(-3) = -2.4132, \\
u_{2,6}^{(\text{new})} &= \mathbf{w}_2^{(\text{new})\top} \mathbf{x}_6 = (0.7904)(2) + (-0.6712)(0) + (0.1808)(-3) = 1.0384, \\
u_{3,6}^{(\text{new})} &= \mathbf{w}_3^{(\text{new})\top} \mathbf{x}_6 = (0.3976)(2) + (0.3167)(0) + (0.3251)(-3) = -0.1801, \\
h_{1,6}^{(\text{new})} &= g(u_{1,6}) = 0.1 u_{1,6}^{(\text{new})} = -0.2413, \\
h_{2,6}^{(\text{new})} &= g(u_{2,6}) = u_{2,6}^{(\text{new})} = 1.0384, \\
h_{3,6}^{(\text{new})} &= g(u_{3,6}) = 0.1 u_{3,6}^{(\text{new})} = -0.0180, \\
q_6^{(\text{new})} &= \boldsymbol{\beta}^{(\text{new})\top} \mathbf{h}_6 = (-0.0866)(-0.2413) + 1.1264(1.0384) - 0.1807(-0.0180) \\
&= 1.1928, \\
\hat{y}_6^{(\text{new})} &= 1.1928, \\
u_{1,7}^{(\text{new})} &= \mathbf{w}_1^{(\text{new})\top} \mathbf{x}_7 = (0.1137)(2) + (-0.6772)(0) + (0.8802)(0) = 0.2274, \\
u_{2,7}^{(\text{new})} &= \mathbf{w}_2^{(\text{new})\top} \mathbf{x}_7 = (0.7904)(2) + (-0.6712)(0) + (0.1808)(0) = 1.5808, \\
u_{3,7}^{(\text{new})} &= \mathbf{w}_3^{(\text{new})\top} \mathbf{x}_7 = (0.3976)(2) + (0.3167)(0) + (0.3251)(0) = 0.7952, \\
h_{1,7}^{(\text{new})} &= g(u_{1,7}) = u_{1,7}^{(\text{new})} = 0.2274, \\
h_{2,7}^{(\text{new})} &= g(u_{2,7}) = u_{2,7}^{(\text{new})} = 1.5808, \\
h_{3,7}^{(\text{new})} &= g(u_{3,7}) = u_{3,7}^{(\text{new})} = 0.7952, \\
q_7^{(\text{new})} &= \boldsymbol{\beta}^{(\text{new})\top} \mathbf{h}_7 = (-0.0866)(0.2274) + 1.1264(1.5808) - 0.1807(0.7952) \\
&= 1.6173, \\
\hat{y}_7^{(\text{new})} &= 1.6173, \\
u_{1,8}^{(\text{new})} &= \mathbf{w}_1^{(\text{new})\top} \mathbf{x}_8 = (0.1137)(-1) + (-0.6772)(-2) + (0.8802)(3) = 3.8813, \\
u_{2,8}^{(\text{new})} &= \mathbf{w}_2^{(\text{new})\top} \mathbf{x}_8 = (0.7904)(-1) + (-0.6712)(-2) + (0.1808)(3) = 1.0944, \\
u_{3,8}^{(\text{new})} &= \mathbf{w}_3^{(\text{new})\top} \mathbf{x}_8 = (0.3976)(-1) + (0.3167)(-2) + (0.3251)(3) = -0.0557, \\
h_{1,8}^{(\text{new})} &= g(u_{1,8}) = u_{1,8}^{(\text{new})} = 3.8813, \\
h_{2,8}^{(\text{new})} &= g(u_{2,8}) = u_{2,8}^{(\text{new})} = 1.0944, \\
h_{3,8}^{(\text{new})} &= g(u_{3,8}) = 0.1 u_{3,8}^{(\text{new})} = -0.0056, \\
q_8^{(\text{new})} &= \boldsymbol{\beta}^{(\text{new})\top} \mathbf{h}_8 = (-0.0866)(3.8813) + 1.1264(1.0944) - 0.1807(-0.0056) \\
&= 0.8971, \\
\hat{y}_8^{(\text{new})} &= 0.8971.
\end{aligned}$$

A.7 Deriving the OLS estimator

We aim to minimize the sum of residual squares:

$$\text{RSS}(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2$$

Thus, we can write the objective function as:

$$\min_{\boldsymbol{\beta}} Q(\boldsymbol{\beta}), \quad Q(\boldsymbol{\beta}) := (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

We can expand to:

$$Q(\boldsymbol{\beta}) = \mathbf{y}^\top \mathbf{y} - 2\mathbf{y}^\top \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\beta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\beta}$$

We can now take the gradient:

$$\nabla_{\boldsymbol{\beta}} Q(\boldsymbol{\beta}) = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\beta}$$

and find the first order conditions to identify the normal equations:

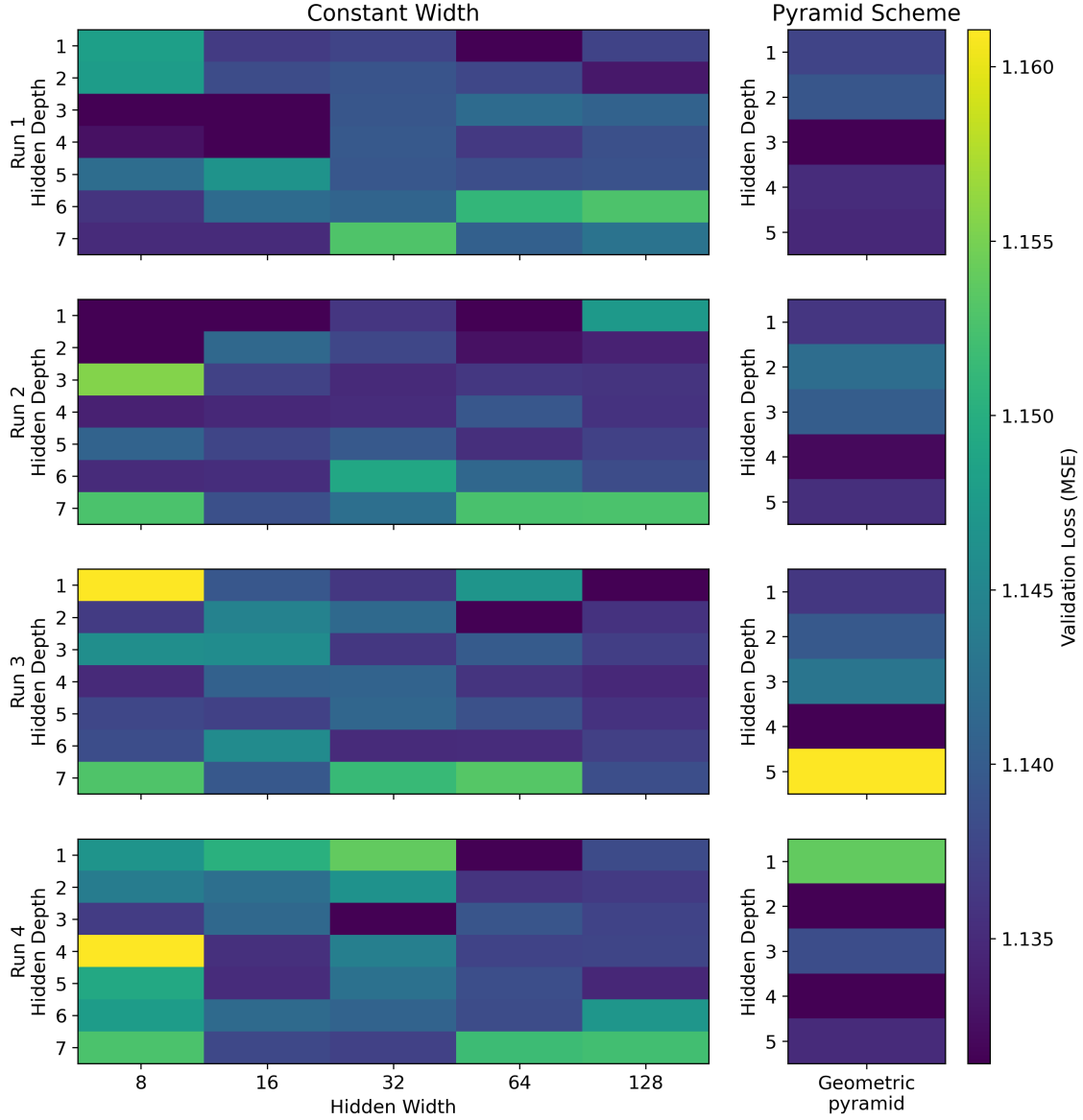
$$\mathbf{X}^\top \mathbf{X}\boldsymbol{\beta} = \mathbf{X}^\top \mathbf{y}$$

Closed form solution when $\mathbf{X}^\top \mathbf{X}$ is non-singular:

$$\hat{\boldsymbol{\beta}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

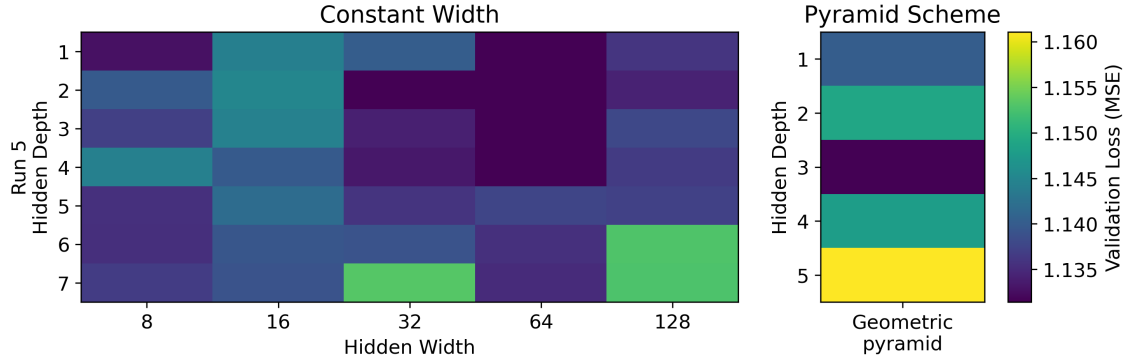
B Additional Tables and Figures

Figure B.1: Validation loss for constant width networks and geometric pyramid networks



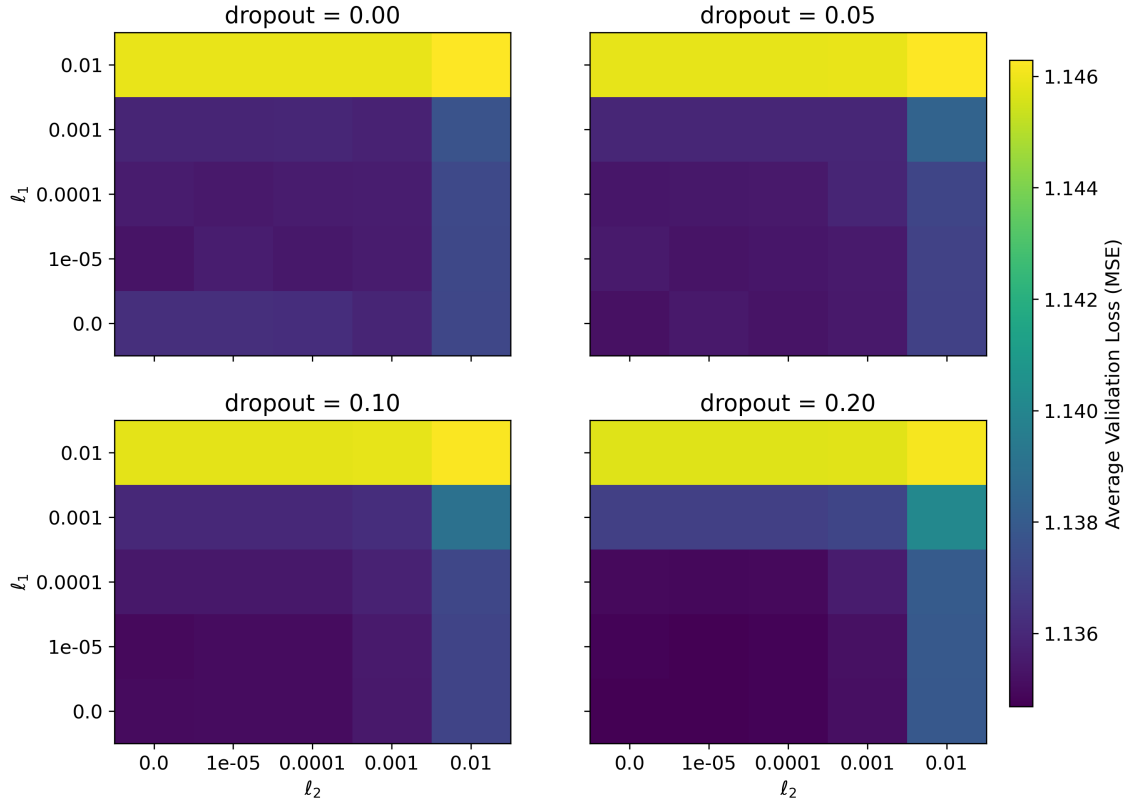
NOTE: Minimum validation loss for each of the five models trained on data up until 2020 and validated on 2020 data (first year in our expanding window) with $\lambda_{\ell_1} = \lambda_{\ell_2} = 10^{-4}$, dropout of 0.0, and learning rate of $5 \cdot 10^{-5}$. The geometric pyramid scheme follows Gu, Kelly, and Xiu (2020), with widths [32], [32, 16], [32, 16, 8], [32, 16, 8, 4], and [32, 16, 8, 4, 2]

Figure B.1: Validation loss for constant width networks and geometric pyramid networks (continued)



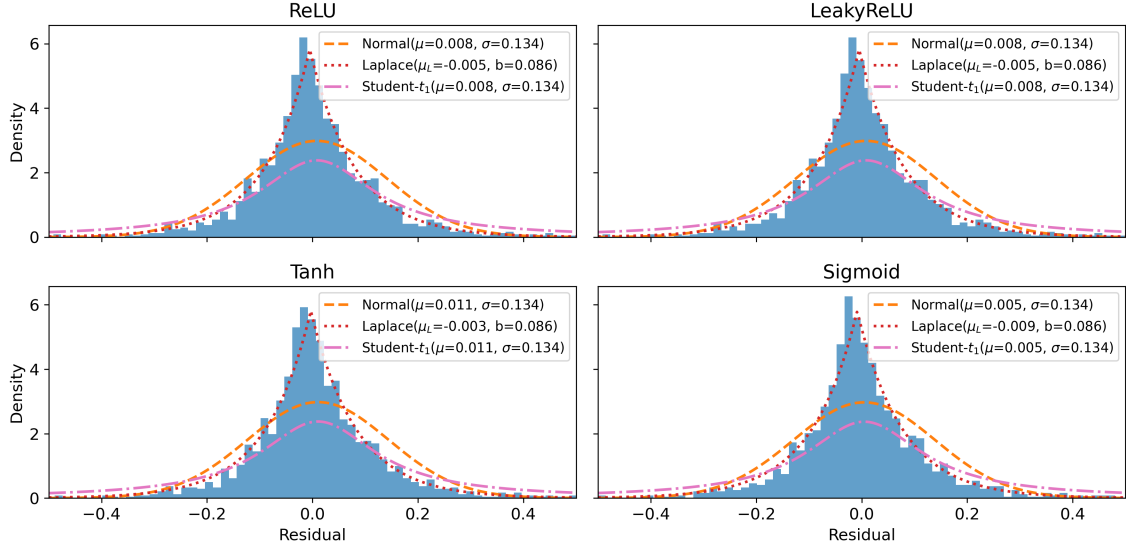
NOTE: Minimum validation loss for each of the five models trained on data up until 2020 and validated on 2020 data (first year in our expanding window) with $\lambda_{\ell_1} = \lambda_{\ell_2} = 10^{-4}$, dropout of 0.0, and learning rate of $5 \cdot 10^{-5}$. The geometric pyramid scheme follows Gu, Kelly, and Xiu (2020), with widths $[32]$, $[32, 16]$, $[32, 16, 8]$, $[32, 16, 8, 4]$, and $[32, 16, 8, 4, 2]$

Figure B.2: Average validation loss in a pyramid structured network with varying regularization



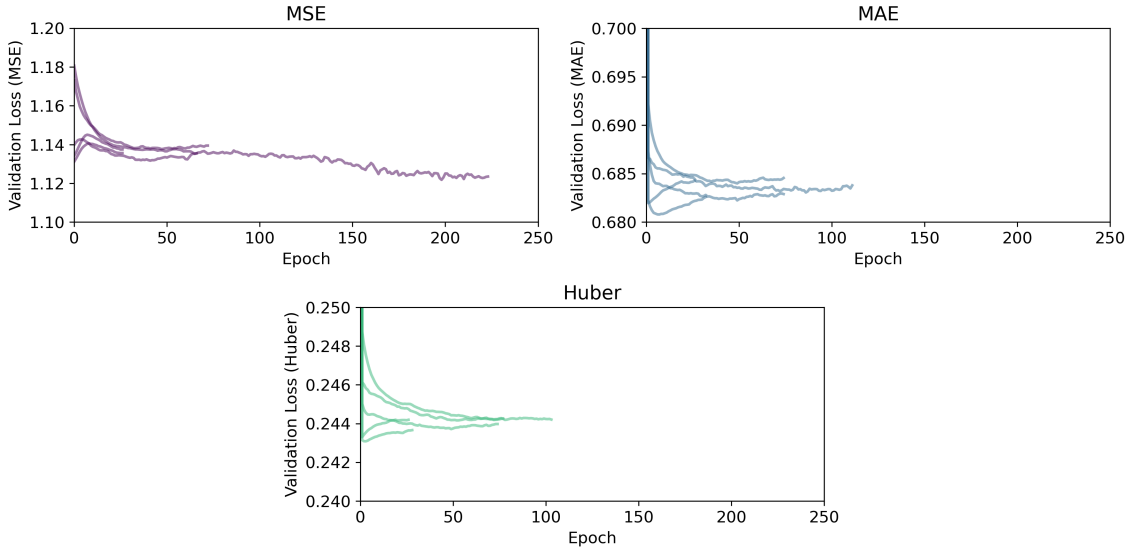
NOTE: Simple average validation loss evaluated across an ensemble of five models trained on data up until 2020 and validated on 2020 data (first year in our expanding window), with **depth** = 3, **width** = $[32, 16, 8]$, and learning rate of $5 \cdot 10^{-5}$.

Figure B.3: Residuals on the validation data for varying activation functions



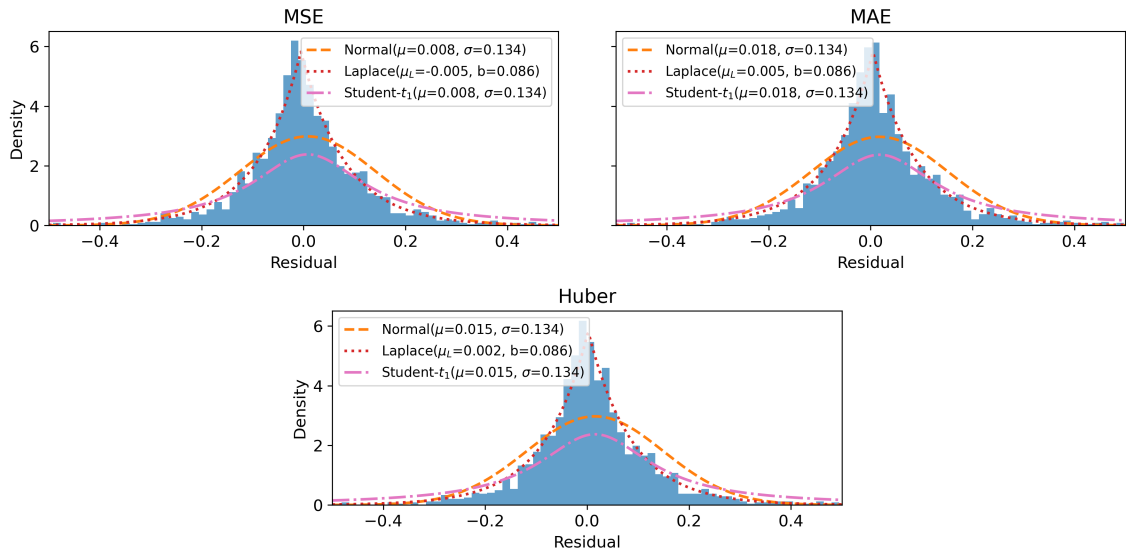
NOTE: Simple average validation loss evaluated across an ensemble of five models with varying activation functions. The models are trained on data up until 2020 and validated on 2020 data (first year in our expanding window), with other parameters following our preferred model of constant width.

Figure B.4: Validation loss for a constant-width network with varying loss functions



NOTE: Simple average validation loss evaluated across an ensemble of five models with varying activation functions. The models are trained on data up until 2020 and validated on 2020 data (first year in our expanding window), with other parameters following our preferred model of constant width.

Figure B.5: Residuals on the validation data for varying loss functions



NOTE: Simple average validation loss evaluated across an ensemble of five models with varying loss functions. The models are trained on data up until 2020 and validated on 2020 data (first year in our expanding window), with other parameters following our preferred model of constant width.

Table B.1: Variable description and statistics

No.	Acronym	Description	Mean	StdDev	Min	25%	Median	75%	Max
1	absacc	Absolute accruals	$6.62 \cdot 10^{-2}$	1.10	0.00	$2.33 \cdot 10^{-3}$	$3.15 \cdot 10^{-2}$	$5.67 \cdot 10^{-2}$	$6.62 \cdot 10^1$
2	acc	Working capital accruals	$-5.63 \cdot 10^{-2}$	1.11	$-6.62 \cdot 10^1$	$-5.46 \cdot 10^{-2}$	$-2.82 \cdot 10^{-2}$	$-1.64 \cdot 10^{-3}$	3.14
3	agr	Asset growth	1.09	$4.70 \cdot 10^1$	$-9.98 \cdot 10^{-1}$	$-3.95 \cdot 10^{-2}$	$3.35 \cdot 10^{-2}$	$1.28 \cdot 10^{-1}$	$4.26 \cdot 10^3$
4	baspread	Bid-ask spread	$7.96 \cdot 10^{-3}$	$1.69 \cdot 10^{-2}$	$-2.32 \cdot 10^{-1}$	$1.52 \cdot 10^{-3}$	$3.66 \cdot 10^{-3}$	$8.39 \cdot 10^{-3}$	$4.76 \cdot 10^{-1}$
5	beta	Beta	$5.35 \cdot 10^{-1}$	$5.86 \cdot 10^{-1}$	-6.72	$1.55 \cdot 10^{-1}$	$4.66 \cdot 10^{-1}$	$8.49 \cdot 10^{-1}$	$1.10 \cdot 10^1$
6	betasq	Beta squared	$6.30 \cdot 10^{-1}$	1.67	0.00	$4.26 \cdot 10^{-2}$	$2.36 \cdot 10^{-1}$	$7.41 \cdot 10^{-1}$	$1.21 \cdot 10^2$
7	bm	Book-to-market	9.34	$2.65 \cdot 10^2$	$-6.30 \cdot 10^1$	$1.56 \cdot 10^{-1}$	$5.39 \cdot 10^{-1}$	1.14	$3.10 \cdot 10^4$
8	cash	Cash holdings	$1.36 \cdot 10^{-1}$	$1.89 \cdot 10^{-1}$	0.00	$2.56 \cdot 10^{-2}$	$6.91 \cdot 10^{-2}$	$1.57 \cdot 10^{-1}$	1.00
9	cashdebt	Cash flow to debt	$3.19 \cdot 10^1$	$7.77 \cdot 10^2$	$-5.08 \cdot 10^3$	0.00	$1.77 \cdot 10^{-1}$	$5.64 \cdot 10^{-1}$	$3.91 \cdot 10^4$
10	cashpr	Cash productivity	$1.11 \cdot 10^{12}$	$3.69 \cdot 10^{13}$	$-2.64 \cdot 10^{14}$	$-2.64 \cdot 10^{10}$	$-8.61 \cdot 10^8$	$1.53 \cdot 10^{10}$	$2.20 \cdot 10^{15}$
11	cfp	Cash flow to price ratio	$4.45 \cdot 10^{-1}$	$4.26 \cdot 10^1$	$-5.74 \cdot 10^2$	$6.07 \cdot 10^{-3}$	$5.87 \cdot 10^{-2}$	$1.35 \cdot 10^{-1}$	$5.00 \cdot 10^3$
12	cfp_ia	Industry-adjusted cash flow to price ratio	$9.68 \cdot 10^{-2}$	$1.34 \cdot 10^1$	$-5.64 \cdot 10^2$	$-6.73 \cdot 10^{-2}$	0.00	$5.30 \cdot 10^{-2}$	$5.18 \cdot 10^2$

Continued on next page

NOTE: Variables are based on those selected by Gu, Kelly, and Xiu (2020) from Green, Hand, and Zhang (2017) and Welch and Goyal (2008) and available data from London Stock Exchange Group and Erhvervsstyrelsen (Danish Business Authority). Variables *discount* and *tms* are based on data from Statistics Denmark. In addition we include dummy variables for each 2 digit NACE industry in our models. Descriptive statistics are based on the raw, un-winsorized and un-standardized, data.

Table B.1 Variable description and statistics (continued)

No.	Acronym	Description	Mean	StdDev	Min	25%	Median	75%	Max
13	chatoia	Industry-adjusted change in asset turnover	$-8.91 \cdot 10^{-3}$	6.29	$-1.40 \cdot 10^2$	$-1.29 \cdot 10^{-1}$	0.00	$8.25 \cdot 10^{-2}$	$4.50 \cdot 10^2$
14	chmom	Change in 6-month momentum	$5.10 \cdot 10^{-2}$	$3.82 \cdot 10^{-1}$	$-9.73 \cdot 10^{-1}$	$-1.10 \cdot 10^{-1}$	$2.10 \cdot 10^{-2}$	$1.65 \cdot 10^{-1}$	$1.52 \cdot 10^1$
15	chpmia	Industry-adjusted change in profit margin	$-9.07 \cdot 10^{-2}$	6.56	$-2.68 \cdot 10^2$	$-3.26 \cdot 10^{-2}$	$2.97 \cdot 10^{-2}$	$1.92 \cdot 10^{-1}$	$3.90 \cdot 10^1$
16	discount	National Banken's discount rate	$1.44 \cdot 10^{-2}$	$1.44 \cdot 10^{-2}$	0.00	0.00	$1.00 \cdot 10^{-2}$	$2.60 \cdot 10^{-2}$	$4.50 \cdot 10^{-2}$
17	divi	Dividend initiation	$7.77 \cdot 10^{-3}$	$8.78 \cdot 10^{-2}$	0.00	0.00	0.00	0.00	1.00
18	divo	Dividend omission	$8.43 \cdot 10^{-3}$	$9.14 \cdot 10^{-2}$	0.00	0.00	0.00	0.00	1.00
19	dkkvol	DKK trading volume	$2.61 \cdot 10^7$	$1.06 \cdot 10^8$	$2.71 \cdot 10^1$	$6.28 \cdot 10^4$	$3.29 \cdot 10^5$	$4.52 \cdot 10^6$	$3.59 \cdot 10^9$
20	dp	Diff. in log of total annualized dividends accross companies and log of index price	2.90	1.63	$-3.79 \cdot 10^{-1}$	1.78	2.78	3.86	7.14

Continued on next page

NOTE: Variables are based on those selected by Gu, Kelly, and Xiu (2020) from Green, Hand, and Zhang (2017) and Welch and Goyal (2008) and available data from London Stock Exchange Group and Erhvervsstyrelsen (Danish Business Authority). Variables *discount* and *tms* are based on data from Statistics Denmark. In addition we include dummy variables for each 2 digit NACE industry in our models. Descriptive statistics are based on the raw, un-winsorized and un-standarized, data.

Table B.1 Variable description and statistics (continued)

No.	Acronym	Description	Mean	StdDev	Min	25%	Median	75%	Max
21	dy	Dividend-to-price ratio	$1.61 \cdot 10^{-7}$	$3.85 \cdot 10^{-6}$	0.00	0.00	$4.96 \cdot 10^{-12}$	$1.46 \cdot 10^{-9}$	$2.46 \cdot 10^{-4}$
22	egr	Growth in common shareholder equity	$2.24 \cdot 10^{-1}$	$2.04 \cdot 10^1$	$-5.44 \cdot 10^2$	$-3.94 \cdot 10^{-2}$	$4.45 \cdot 10^{-2}$	$1.36 \cdot 10^{-1}$	$1.22 \cdot 10^3$
23	ep	Diff. in log of total earnings accross companies and log of index price	$2.16 \cdot 10^1$	$4.35 \cdot 10^{-1}$	$2.06 \cdot 10^1$	$2.13 \cdot 10^1$	$2.15 \cdot 10^1$	$2.19 \cdot 10^1$	$2.24 \cdot 10^1$
24	gma	Gross profitability	$2.84 \cdot 10^{-1}$	$4.20 \cdot 10^{-1}$	$-2.78 \cdot 10^1$	$5.41 \cdot 10^{-2}$	$2.13 \cdot 10^{-1}$	$4.38 \cdot 10^{-1}$	8.78
25	grltnoa	Growth in long-term net operating assets	$5.96 \cdot 10^{-1}$	$1.68 \cdot 10^1$	-1.00	$-4.14 \cdot 10^{-2}$	$3.00 \cdot 10^{-2}$	$1.39 \cdot 10^{-1}$	$2.42 \cdot 10^3$
26	idiovol	Idiosyncratic return volatility	$5.09 \cdot 10^{-2}$	$5.95 \cdot 10^{-2}$	$9.73 \cdot 10^{-4}$	$2.79 \cdot 10^{-2}$	$3.93 \cdot 10^{-2}$	$5.85 \cdot 10^{-2}$	3.15
27	ill	Illiquidity	$7.89 \cdot 10^{-6}$	$2.91 \cdot 10^{-4}$	0.00	$3.22 \cdot 10^{-9}$	$6.24 \cdot 10^{-8}$	$4.76 \cdot 10^{-7}$	$5.58 \cdot 10^{-2}$
28	indmom	Industry momentum	$1.28 \cdot 10^{-1}$	$3.98 \cdot 10^{-1}$	-1.00	$-8.73 \cdot 10^{-2}$	$9.42 \cdot 10^{-2}$	$3.08 \cdot 10^{-1}$	7.92
29	lev	Leverage	$2.45 \cdot 10^{-1}$	$2.90 \cdot 10^{-1}$	0.00	$3.69 \cdot 10^{-2}$	$1.90 \cdot 10^{-1}$	$3.80 \cdot 10^{-1}$	$1.37 \cdot 10^1$

Continued on next page

NOTE: Variables are based on those selected by Gu, Kelly, and Xiu (2020) from Green, Hand, and Zhang (2017) and Welch and Goyal (2008) and available data from London Stock Exchange Group and Erhvervsstyrelsen (Danish Business Authority). Variables *discount* and *tms* are based on data from Statistics Denmark. In addition we include dummy variables for each 2 digit NACE industry in our models. Descriptive statistics are based on the raw, un-winsorized and un-standarized, data.

Table B.1 Variable description and statistics (continued)

No.	Acronym	Description	Mean	StdDev	Min	25%	Median	75%	Max
30	lgr	Growth in long-term debt	1.74	$2.50 \cdot 10^1$	-1.00	$-1.17 \cdot 10^{-1}$	0.00	$7.91 \cdot 10^{-2}$	$7.88 \cdot 10^2$
31	maxret	Maximum daily return	$6.68 \cdot 10^{-2}$	$8.54 \cdot 10^{-1}$	0.00	$2.45 \cdot 10^{-2}$	$3.99 \cdot 10^{-2}$	$6.74 \cdot 10^{-2}$	$1.47 \cdot 10^2$
32	mom12m	12-month momentum	$1.29 \cdot 10^{-1}$	$6.38 \cdot 10^{-1}$	-1.00	$-1.73 \cdot 10^{-1}$	$5.74 \cdot 10^{-2}$	$3.18 \cdot 10^{-1}$	$1.64 \cdot 10^1$
33	mom1m	1-month momentum	$1.02 \cdot 10^{-2}$	$1.57 \cdot 10^{-1}$	-1.00	$-4.32 \cdot 10^{-2}$	0.00	$5.04 \cdot 10^{-2}$	9.07
34	mom6m	6-month momentum	$5.33 \cdot 10^{-2}$	$3.83 \cdot 10^{-1}$	-1.00	$-1.08 \cdot 10^{-1}$	$2.26 \cdot 10^{-2}$	$1.69 \cdot 10^{-1}$	$1.52 \cdot 10^1$
35	mve_ia	Industry-adjusted mvel1	$2.13 \cdot 10^{-2}$	1.83	-7.10	-1.01	0.00	$9.80 \cdot 10^{-1}$	6.72
36	mvel1	Log of market cap	$2.09 \cdot 10^1$	2.52	9.75	$1.92 \cdot 10^1$	$2.06 \cdot 10^1$	$2.26 \cdot 10^1$	$3.12 \cdot 10^1$
37	operprof	Operating profitability	$2.86 \cdot 10^{-3}$	$5.42 \cdot 10^{-1}$	$-5.38 \cdot 10^1$	$2.94 \cdot 10^{-3}$	$3.00 \cdot 10^{-2}$	$8.06 \cdot 10^{-2}$	2.43
38	pchcurrat	% change in current ratio	$8.40 \cdot 10^{-1}$	$1.94 \cdot 10^1$	-1.00	$-6.44 \cdot 10^{-2}$	0.00	$1.21 \cdot 10^{-1}$	$1.39 \cdot 10^3$

Continued on next page

NOTE: Variables are based on those selected by Gu, Kelly, and Xiu (2020) from Green, Hand, and Zhang (2017) and Welch and Goyal (2008) and available data from London Stock Exchange Group and Erhvervsstyrelsen (Danish Business Authority). Variables *discount* and *tms* are based on data from Statistics Denmark. In addition we include dummy variables for each 2 digit NACE industry in our models. Descriptive statistics are based on the raw, un-winsorized and un-standardized, data.

Table B.1 Variable description and statistics (continued)

No.	Acronym	Description	Mean	StdDev	Min	25%	Median	75%	Max
39	pchgm_pchsale	% change in gross margin - % change in sales	$4.50 \cdot 10^{-3}$	4.88	$-7.96 \cdot 10^1$	$-2.05 \cdot 10^{-2}$	0.00	$1.76 \cdot 10^{-2}$	$1.94 \cdot 10^2$
40	pchquick	% change in quick ratio	1.88	$6.24 \cdot 10^1$	-8.67	$-1.22 \cdot 10^{-1}$	0.00	$1.18 \cdot 10^{-1}$	$3.41 \cdot 10^3$
41	pchsale_pchinvt	% change in sales - % change in inventory	$-2.22 \cdot 10^{-1}$	$2.19 \cdot 10^1$	$-1.39 \cdot 10^3$	$-8.00 \cdot 10^{-2}$	0.00	$7.95 \cdot 10^{-2}$	$1.03 \cdot 10^3$
42	pchsaleinv	% change sales-to-inventory	$2.57 \cdot 10^{-1}$	$1.05 \cdot 10^1$	$-2.90 \cdot 10^2$	$-7.03 \cdot 10^{-2}$	0.00	$7.93 \cdot 10^{-2}$	$7.40 \cdot 10^2$
43	pctacc	Percent accruals	$2.38 \cdot 10^6$	$1.43 \cdot 10^8$	$-5.96 \cdot 10^6$	-1.19	$-4.11 \cdot 10^{-1}$	$-9.74 \cdot 10^{-2}$	$8.78 \cdot 10^9$
44	pricedelay	Price delay	$5.55 \cdot 10^{-1}$	$3.03 \cdot 10^{-1}$	$1.38 \cdot 10^{-3}$	$2.83 \cdot 10^{-1}$	$5.46 \cdot 10^{-1}$	$8.47 \cdot 10^{-1}$	1.00
45	quick	Quick ratio	2.36	$1.30 \cdot 10^1$	$-2.65 \cdot 10^{-1}$	$1.22 \cdot 10^{-1}$	$7.92 \cdot 10^{-1}$	1.35	$5.69 \cdot 10^2$
46	rd	R&D increase	$3.35 \cdot 10^{-2}$	$6.90 \cdot 10^{-1}$	-7.78	0.00	0.00	0.00	$3.12 \cdot 10^1$
47	rd_mve	R&D to market capitalization	$1.10 \cdot 10^{-2}$	$1.84 \cdot 10^{-1}$	$-1.55 \cdot 10^{-2}$	0.00	0.00	0.00	$1.05 \cdot 10^1$
48	rd_sale	R&D to sales	$2.33 \cdot 10^{-1}$	3.60	$-1.58 \cdot 10^{-1}$	0.00	0.00	0.00	$1.41 \cdot 10^2$
49	retvol	Return volatility	$2.62 \cdot 10^{-2}$	$1.90 \cdot 10^{-1}$	0.00	$1.23 \cdot 10^{-2}$	$1.87 \cdot 10^{-2}$	$2.89 \cdot 10^{-2}$	$3.22 \cdot 10^1$
50	roaq	Return on assets	$-3.22 \cdot 10^{-2}$	1.22	$-6.73 \cdot 10^1$	$-3.11 \cdot 10^{-3}$	$1.92 \cdot 10^{-2}$	$5.80 \cdot 10^{-2}$	4.62

Continued on next page

NOTE: Variables are based on those selected by Gu, Kelly, and Xiu (2020) from Green, Hand, and Zhang (2017) and Welch and Goyal (2008) and available data from London Stock Exchange Group and Erhvervsstyrelsen (Danish Business Authority). Variables *discount* and *tms* are based on data from Statistics Denmark. In addition we include dummy variables for each 2 digit NACE industry in our models. Descriptive statistics are based on the raw, un-winsorized and un-standardized, data.

Table B.1 Variable description and statistics (continued)

No.	Acronym	Description	Mean	StdDev	Min	25%	Median	75%	Max
51	roeq	Return on equity	$2.26 \cdot 10^{-1}$	$1.56 \cdot 10^1$	$-5.21 \cdot 10^2$	$-4.17 \cdot 10^{-3}$	$7.95 \cdot 10^{-2}$	$1.51 \cdot 10^{-1}$	$9.11 \cdot 10^2$
52	roic	Return on invested capital	$-3.22 \cdot 10^{-2}$	1.22	$-6.73 \cdot 10^1$	$-3.11 \cdot 10^{-3}$	$1.92 \cdot 10^{-2}$	$5.80 \cdot 10^{-2}$	4.62
53	salecash	Sales to cash	$1.89 \cdot 10^{11}$	$6.70 \cdot 10^{11}$	$-7.42 \cdot 10^{11}$	$9.98 \cdot 10^8$	$9.75 \cdot 10^9$	$6.28 \cdot 10^{10}$	$2.83 \cdot 10^{13}$
54	saleinv	Sales to inventory	1.86	3.71	$-4.96 \cdot 10^1$	0.00	1.65	2.73	$1.46 \cdot 10^2$
55	sgr	Sales growth	$4.59 \cdot 10^{-1}$	$1.36 \cdot 10^1$	$-1.26 \cdot 10^2$	$-3.99 \cdot 10^{-2}$	$3.82 \cdot 10^{-2}$	$1.50 \cdot 10^{-1}$	$1.03 \cdot 10^3$
56	std_dkkvol	Volatility of liquidity (dkk trading volume)	$1.46 \cdot 10^7$	$9.20 \cdot 10^7$	0.00	$6.29 \cdot 10^4$	$3.38 \cdot 10^5$	$4.24 \cdot 10^6$	$1.24 \cdot 10^{10}$
57	std_turn	Volatility of liquidity (share turnover)	5.38	$1.73 \cdot 10^2$	0.00	$2.97 \cdot 10^{-3}$	$3.25 \cdot 10^{-2}$	$1.59 \cdot 10^{-1}$	$2.47 \cdot 10^4$
58	svar	Index variance	$1.32 \cdot 10^{-4}$	$8.39 \cdot 10^{-5}$	$3.80 \cdot 10^{-5}$	$7.36 \cdot 10^{-5}$	$1.19 \cdot 10^{-4}$	$1.57 \cdot 10^{-4}$	$4.23 \cdot 10^{-4}$
59	target	Monthly excess return	$6.23 \cdot 10^{-3}$	$1.24 \cdot 10^{-1}$	-1.00	$-4.47 \cdot 10^{-2}$	$-4.37 \cdot 10^{-4}$	$4.90 \cdot 10^{-2}$	1.50
60	tms	Term spread	2.38	1.70	$-6.90 \cdot 10^{-1}$	$6.90 \cdot 10^{-1}$	2.53	3.88	5.41

Continued on next page

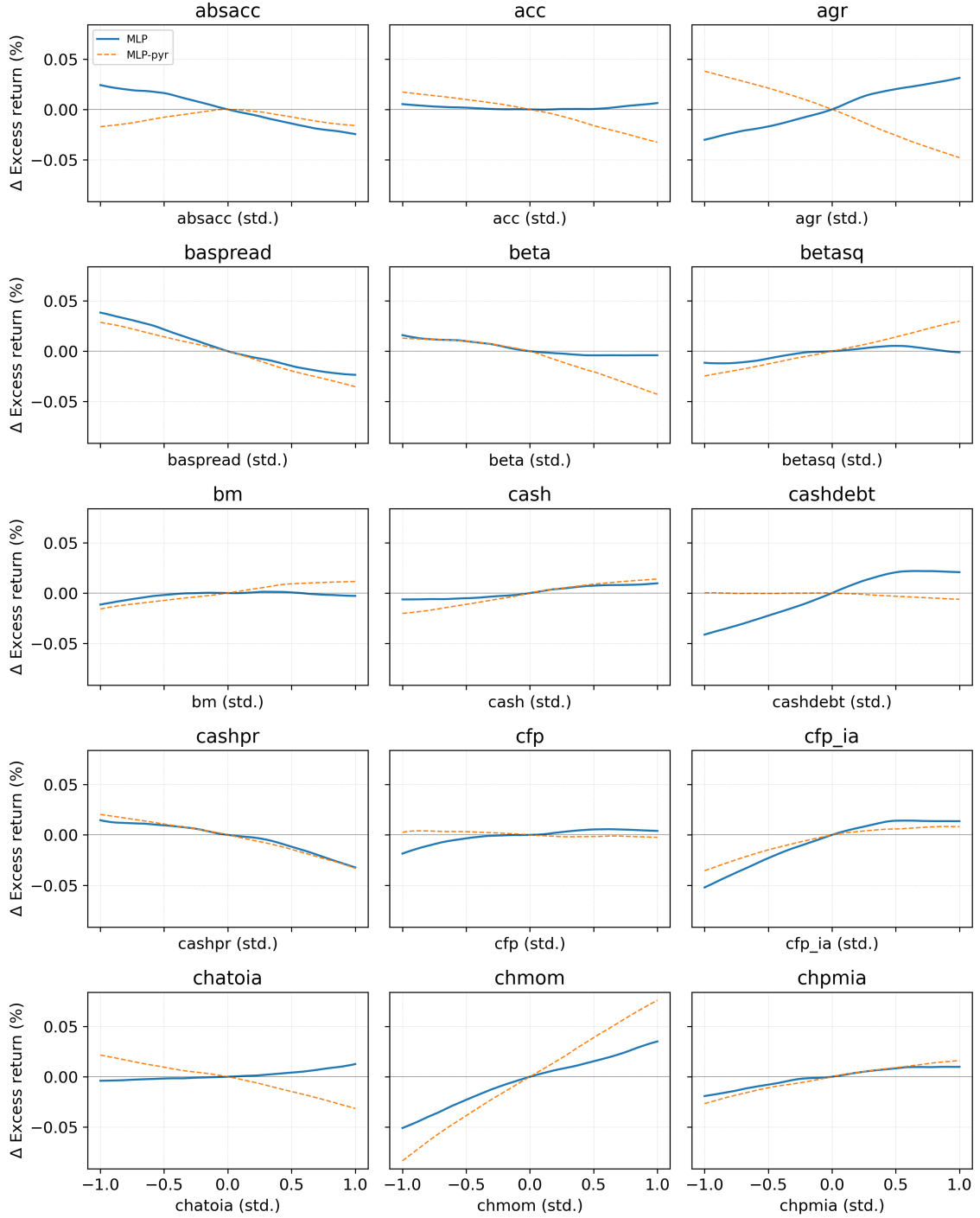
NOTE: Variables are based on those selected by Gu, Kelly, and Xiu (2020) from Green, Hand, and Zhang (2017) and Welch and Goyal (2008) and available data from London Stock Exchange Group and Erhvervsstyrelsen (Danish Business Authority). Variables *discount* and *tms* are based on data from Statistics Denmark. In addition we include dummy variables for each 2 digit NACE industry in our models. Descriptive statistics are based on the raw, un-winsorized and un-standardized, data.

Table B.1 Variable description and statistics (continued)

No.	Acronym	Description	Mean	StdDev	Min	25%	Median	75%	Max
61	turn	3-month rolling avg. of monthly trading volume scaled by number of shares	7.43	$1.82 \cdot 10^2$	$1.26 \cdot 10^{-8}$	$3.99 \cdot 10^{-3}$	$4.11 \cdot 10^{-2}$	$2.12 \cdot 10^{-1}$	$1.37 \cdot 10^4$

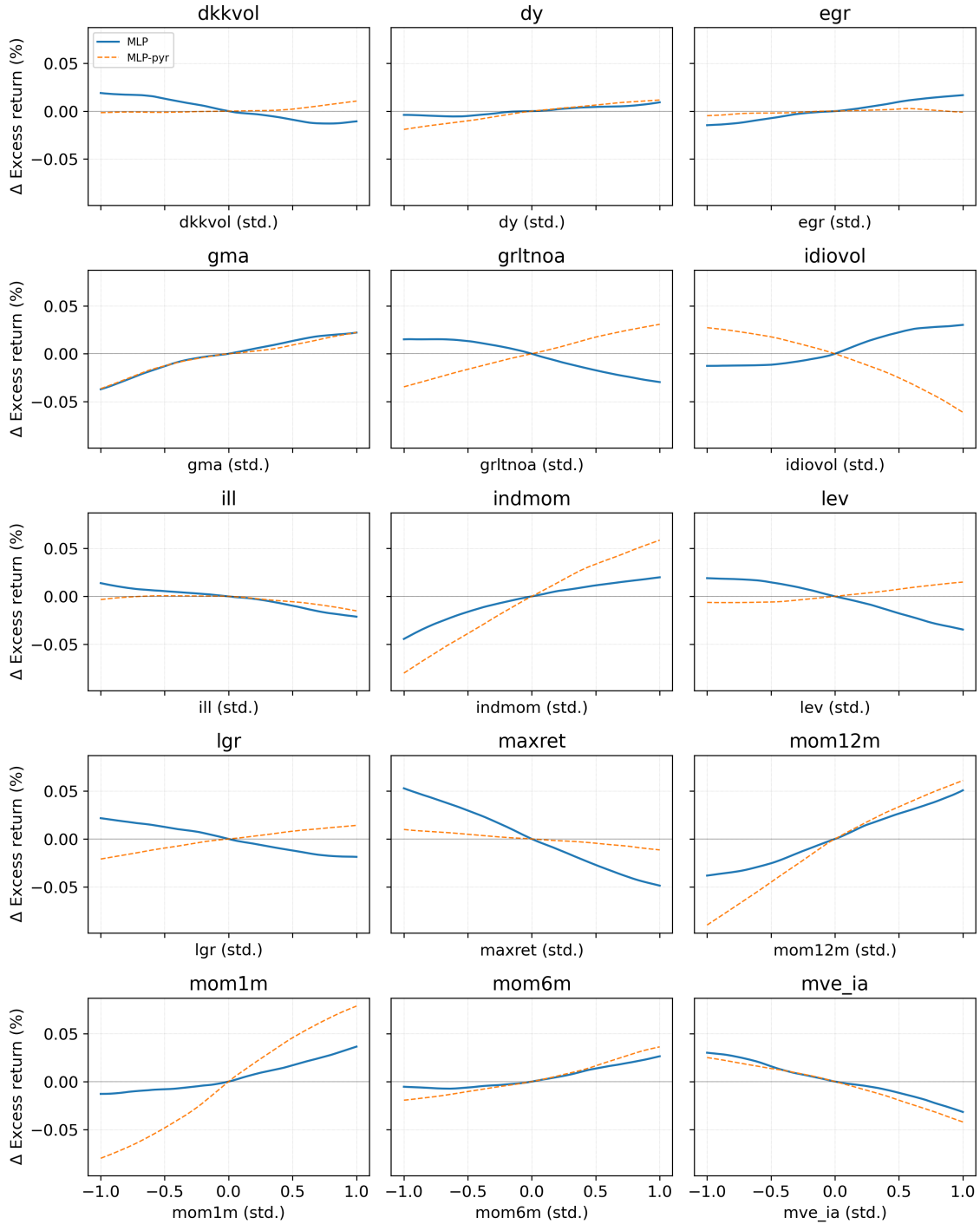
NOTE: Variables are based on those selected by Gu, Kelly, and Xiu (2020) from Green, Hand, and Zhang (2017) and Welch and Goyal (2008) and available data from London Stock Exchange Group and Erhvervsstyrelsen (Danish Business Authority). Variables *discount* and *tms* are based on data from Statistics Denmark. In addition we include dummy variables for each 2 digit NACE industry in our models. Descriptive statistics are based on the raw, un-winsorized and un-standardized, data.

Figure B.6: Marginal effect between expected return and numerical characteristics



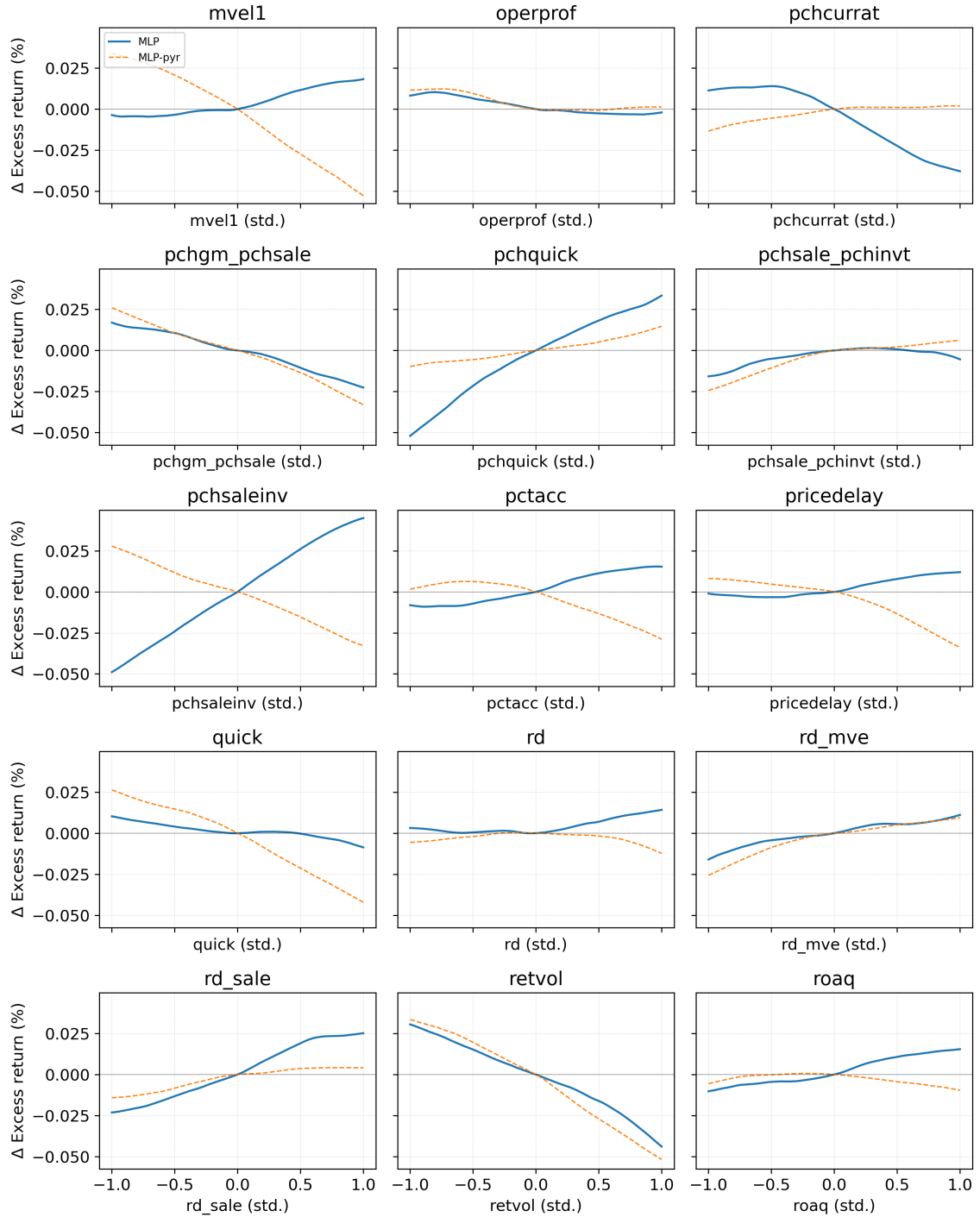
NOTE: Expected change in monthly excess returns (vertical axis) to the individual characteristics (holding all other covariates fixed at their median values).

Figure B.6: Marginal effect between expected return and numerical characteristics (continued)



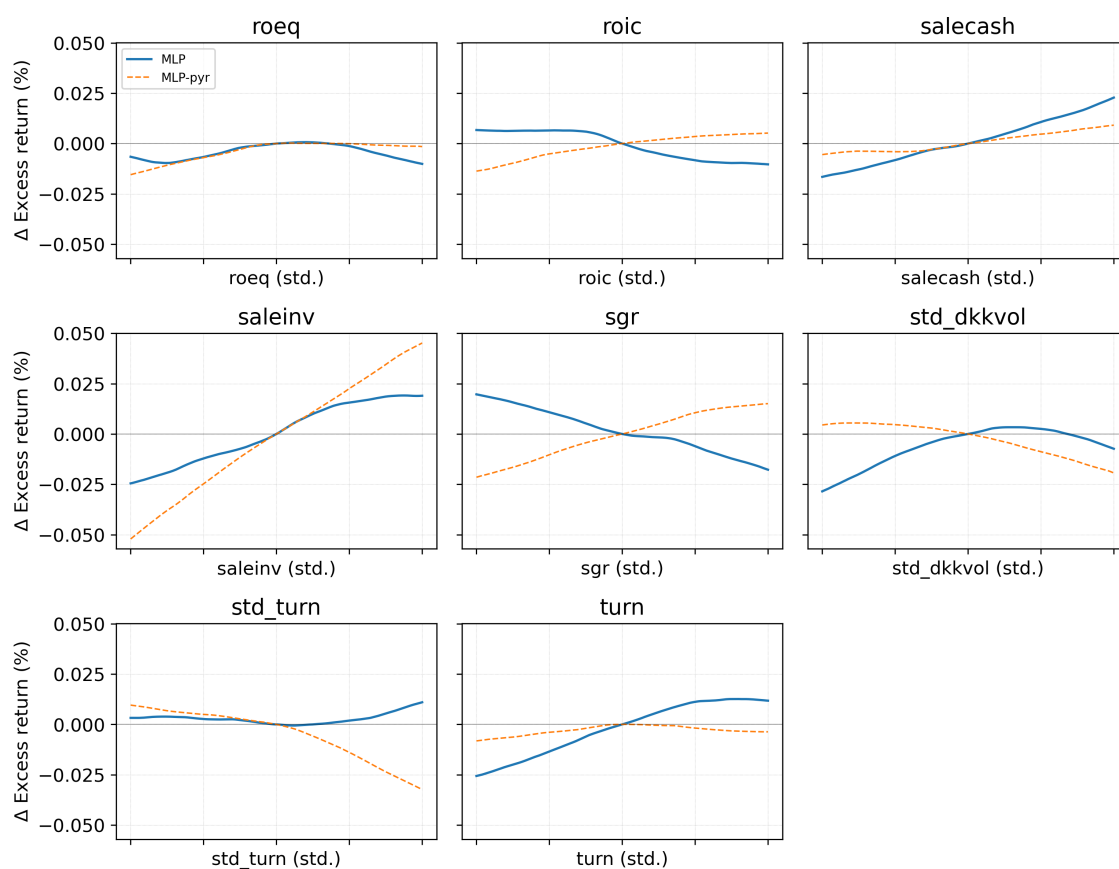
NOTE: Expected change in monthly excess returns (vertical axis) to the individual characteristics (holding all other covariates fixed at their median values).

Figure B.6: Marginal effect between expected return and numerical characteristics (continued)



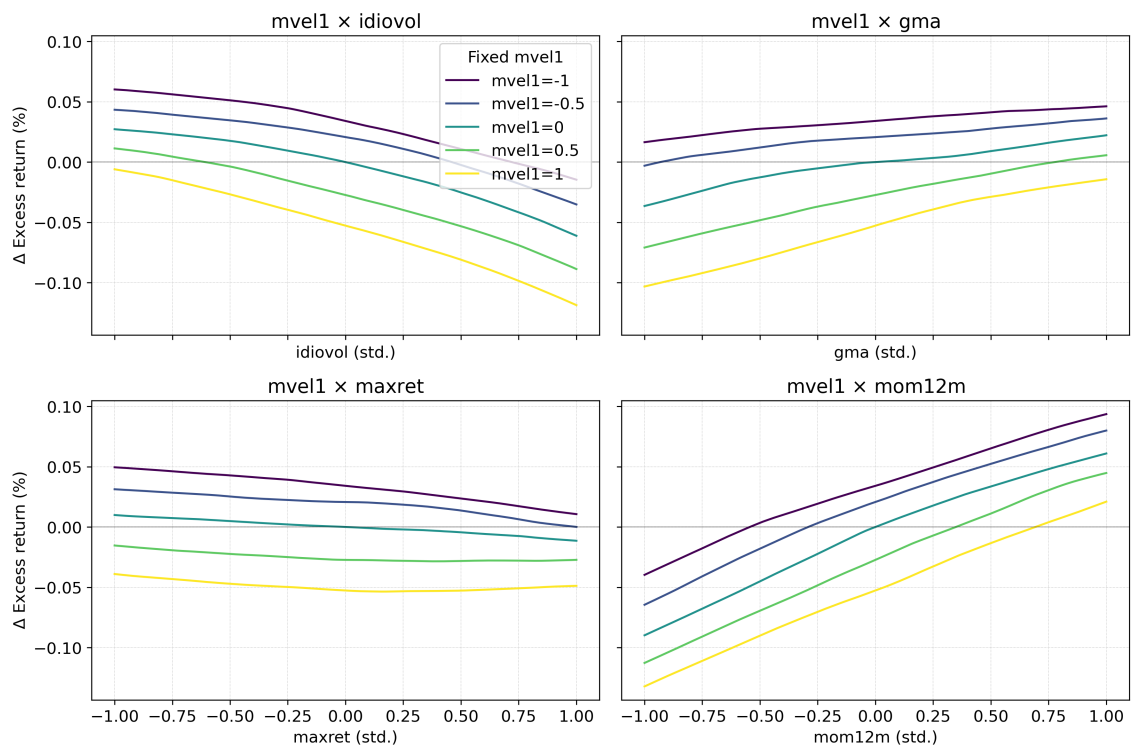
NOTE: Expected change in monthly excess returns (vertical axis) to the individual characteristics (holding all other covariates fixed at their median values).

Figure B.6: Marginal effect between expected return and numerical characteristics (continued)



NOTE: Expected change in monthly excess returns (vertical axis) to the individual characteristics (holding all other covariates fixed at their median values).

Figure B.7: Marginal effect between expected return and select characteristic interactions (MLP-Pyr)



NOTE: Expected change in monthly excess returns (vertical axis) to the individual interactions effects for $mvel1$ with $idiovol$, gma , $mom12m$, $maxret$, in our MLP-Pyr model (holding all other covariates fixed at their median values). For interactions in our MLP model see Figure 4.7